

Symbiosis Framework v0.9

An Engineering Standard for Safe, Economic, and Governable Hybrid Intelligence

20 November 2025

Version 0.9 — Public Release

321 pages · Apache 2.0

github.com/symbiosis

Symbiosis Framework v0.9 — Merged TOC + splice points

Front Matter (retain v0.8, expand with v0.5.2 narrative)

Title Page & Version History (v0.8)

Preface: The Transition from Prompting to Architecture (v0.8)

Splice in: v0.5.2 “How to Read This Book” as Preface §0.2–0.4.

Executive Summary: 1-Page Boardroom View (v0.8)

Splice in: v0.5.2 “Design Goals & Non-Goals” as ES appendix block for defensibility.

New muscle (Front Matter add):

Standard Scope & Conformance Tiers (NEW)

Define “Symbiosis-compliant” levels (Tier 0–3), minimum invariants, and audit expectations.

Part I — Foundations of Hybrid Systems (v0.8 spine)

Chapter 1 — Introduction: The Case for Structured Symbiosis

(v0.8 owns order and narrative)

1.1 Narrative Arc: Shadow AI → Coordinated Intelligence (v0.8)

1.2 Problem Space + HITL Fallacy (v0.8)

1.3 Core Thesis: Safety, Economics, Governance as Code (v0.8)

1.4 Goals: Traceability, Cost-Control, Skill Development (v0.8)

Splice in from v0.5.2:

1.5 Symbiosis vs Automation vs No-AI (insert after 1.2)

1.6 Design Goals & Non-Goals (merge with 1.4, extend)

1.7 Overview of Four-Layer Architecture (keep v0.8 summary, add v0.5.2 explanatory depth)

New muscle:

1.8 Boundary of Claims (NEW)

“What Symbiosis does/does not claim.” This gives you immediate academic/legal defensibility.

Chapter 2 — Ontology v1.1 & Typology

(v0.8 canonical definitions + invariants)

2.1 Core Entities: Humans, Agents, Constraints, Budgets (v0.8)

2.2 Deep Terms: Intent Manifold, Alignment Surface, Governance Kernel (v0.8)

2.3 Typology: Augmentation → Co-Exploration → Co-Agency (v0.8)

2.4 Invariants: Human Primacy, Economic Viability (v0.8)

Splice in from v0.5.2:

2.5 Role of Ontology + conceptual backbone (append to 2.0 introduction)

2.6 Core Entities & Relations (keep v0.8 types; add v0.5.2 relational prose/examples)

2.7 Human Primacy / Constraint Supremacy / Co-Evolution framing (fold into invariants)

New muscle:

2.8 Canonical Ontology Diagram (NEW)

One page showing entities + relations; referenced throughout.

Part II — The Four-Layer Architecture (v0.8 order)

Chapter 3 — Architecture Overview

(v0.8 four-layer stack + economics)

3.1 High-Level Stack (v0.8)

3.2 Cross-Layer Data Flows & Telemetry (v0.8)

3.3 Economic Circuit: Token/Compute Budgets (v0.8)

Splice in from v0.5.2:

3.4 Cross-Layer Flows & Feedback Loops (expand 3.2)

3.5 Core Invariants & Design Constraints (append end of chapter)

New muscle:

3.6 One-page Operational Trace Example (NEW)

“Intent → task graph → contracts → governance → lineage.”

Chapter 4 — Human Layer & Intent Interfaces

(v0.8 roles, attention budget, covenant)

4.1 Roles: Operators, Architects, Governance Leads (v0.8)

4.2 Intent Interface: Structured vs Natural Language (v0.8)

4.3 Cognitive Load & Attention Budget (v0.8)

4.4 Non-Punitive Covenant (v0.8)

Splice in from v0.5.2:

4.5 Human Roles & Institutions (deepen 4.1)

4.6 Human Failure Modes & Mitigations (append end)

4.7 Trust Calibration + working memory framing (integrate into 4.3)

New muscle:

4.8 Intent Interface Patterns Library (NEW)

Chat / guided form / strict object, with band/risk applicability.

Chapter 5 — Core Primitives

(v0.8 primitives baseline + economic constraint triad + regret boundary)

5.1 Intent Surfaces & Manifolds (v0.8)

5.2 Constraint Vectors: C_legal, C_safety, C_econ (v0.8)

5.3 Autonomy Bands & Regret Boundary (v0.8)

5.4 Context Lineage (v0.8)

Splice in from v0.5.2 (as 5.5+):

5.5 Hybrid Cognitive Loops

5.6 Epistemic Boundaries

5.7 Multi-Tier Memory

5.8 Regulatory Confidence Intervals

Upgrade rule: add economic attributes to each imported primitive (budget weights, cost lineage, or economic failure modes) so the primitives are coherent with v0.8's Economic Circuit.

Chapter 6 — Symbiosis Engine (Coordination Layer)

(v0.8 defines engine core + memory hierarchy + economic pass)

6.1 Engine Core: Router, Planner, Optimizer (v0.8)

6.2 Multi-Level Memory Hierarchy (v0.8)

6.3 Economic Pass (v0.8)

6.4 Hierarchical Decomposition & Reconciliation (v0.8)

Splice in from v0.5.2:

6.5 Engine Subcomponents deep dive (Intent Router, Planner, Context/Lineage, Orchestrator, SI Tracker)

6.6 Intent → Task Graph Mapping (formal view)

6.7 Drift detection, recovery, safety supervisor hooks

New muscle:

6.8 Engine Algorithm (numbered 10-step pipeline) (NEW)

This becomes the “standard” definition you can defend and implement.

Chapter 7 — Agentic Layer (Poly-Agent Ecosystem)

(v0.8 clusters + MCP adoption + arbitration)

7.1 Agent Clusters (v0.8)

7.2 MCP Protocol Standardization (v0.8)

7.3 Inter-Agent Arbitration (v0.8)

7.4 Failure Taxonomies & Recovery Flows (v0.8)

Splice in from v0.5.2:

7.5 Capability Indexing / contracts / concurrency safety (insert after 7.2)

7.6 Oversight agents as mandatory Band-2/3 gates (integrate 7.3)

New muscle:

7.7 Contract Schema v1 (MCP-aligned) (NEW)

Include economic fields (cost/caps) + band eligibility + escalation clauses.

Chapter 8 — Governance Layer & Adoption Bridges

(v0.8 governance kernel + fast lane + decision rights + regs)

8.1 Policy Engine + Governance Kernel (v0.8)

8.2 Adoption Bridges (v0.8)

8.3 Decision Rights Matrices & Auditability (v0.8)

8.4 Regulatory Mapping (v0.8)

Splice in from v0.5.2:

8.5 Audit log schema + hashed integrity chains

8.6 Escalation paths / risk matrices depth (expand 8.3)

8.7 Sector overlays precursor material (tie into Ch12 playbooks)

New muscle:

8.8 Adoption Bridge Safety Proof Sketch (NEW)
Formal conditions for bypass + auto-revocation rules.

Part III — Metrics & Instrumentation

Chapter 9 — Symbiosis Index (SI)

(v0.8 math + visualization)

9.1 SI health measurement (v0.8)

9.2 Advanced math extensions (v0.8)

9.3 Visualization (v0.8)

Splice in from v0.5.2:

9.4 SI update logic, smoothing, and band linkage (append)

New muscle:

9.5 SI reliability / anti-gaming notes (NEW)

Chapter 10 — Human Metrics (HSS & SCP)

(v0.8 headings)

10.1 HSS (“Hiss”) (v0.8)

10.2 SCP (v0.8)

10.3 Rubrics & Operator tiers (v0.8)

Splice in from v0.5.2:

Detailed HSS dimensions + task allocation logic (insert within 10.1–10.3).

New muscle:

10.4 Closed-loop control actions (NEW)

Exact system responses to HSS/SCP trends.

Part IV — Dynamics, Operations & Recovery

Chapter 11 — Co-Evolution & Meta-Theory

(v0.8 order)

Splice in from v0.5.2: co-evolution governance/program depth.

Chapter 12 — Sector Implementation Playbooks

(v0.8 order)

Splice in from v0.5.2: full case studies across sectors (replace/expand per section).

Chapter 13 — Failure Modes & Recovery

(v0.8 order)

Splice in from v0.5.2: deeper failure/mitigation taxonomy where present.

Part V — Philosophy & Future

Chapter 14 — Sociotechnical Implications

(v0.8 order)

Splice in from v0.5.2: philosophy/metatheory expansions.

Chapter 15 — Implementation Strategy

(v0.8 order)

Splice in from v0.5.2: literacy program, certification tiers, pilot playbooks, deployment targets.

New muscle:

15.4 Field Formation Roadmap (NEW)

Spec → reference implementation → benchmarks → consortium → regulatory engagement.

Epilogue — Silicon + Carbon Manifesto

(v0.8 expanded epilogue)

Splice in from v0.5.2: manifesto implementation checklist language if needed.

Appendices (retain v0.8 list; backfill with v0.5.2 details)

(v0.8 canonical technical spec pack)

A Formal Ontology & Glossary — updated to v1.1 with reconciled synonyms

B SI math

C Autonomy bands state machine

D Governance matrices & risk models (expand using v0.5.2)

E Memory schemas (expand using v0.5.2)

F Agent contracts & MCP specs (new Contract Schema v1 lands here)

G–N as listed in v0.8, with v0.5.2 content inserted where it already exists.

2. Short v0.8 → v0.9 change log (what changes materially)

Additions (net-new muscle)

1. Standard Scope & Conformance Tiers (Front Matter).

2. Boundary of Claims (Ch1).

3. Canonical Ontology Diagram (Ch2).

4. Operational Trace Example (Ch3).

5. Intent Interface Patterns Library (Ch4).

6. Engine Algorithm (10-step standard pipeline) (Ch6).

7. Contract Schema v1, MCP-aligned (Ch7 + Appendix F).

8. Adoption Bridge Safety Proof Sketch (Ch8).

9. SI Reliability / Anti-gaming (Ch9).

10. Closed-loop HSS/SCP control actions (Ch10).

11. Field Formation Roadmap (Ch15).

Integrations (v0.5.2 flesh into v0.8 spine)

Human role depth + human failure model → Ch4.

Full primitive set (hybrid loops, epistemics, memory, regulatory CI) → Ch5.

Engine subcomponent decomposition + intent→task graph formality → Ch6.

Capability index / contracts / arbitration detail → Ch7.

Audit logging, escalation, pilot numerical trajectories → Ch8, Ch15, Appendix N.

Deprecations / consolidations

Any v0.5.2 primitive naming that conflicts with v0.8 ontology should be re-labelled to v1.1 terms, not co-existed.

v0.5.2 “hard/soft constraints” becomes a strict triad (legal/safety/economic) with hard/soft as sub-types.

v0.5.2 autonomy discussion is subsumed into v0.8’s regret boundary state machine.

Below is Chapter 6.8 for v0.9: the Symbiosis Engine Standard Algorithm. It is written as a normative “engineering standard” section (copy-ready), and it fuses v0.8’s Economic Circuit + Regret Boundary gating with v0.5.2’s full intent→task graph mapping, memory governance, orchestration, and drift recovery.

Below is a proposed merged Chapter 1 (v0.8 skeleton fully fleshed with v0.5.2 content, plus added “muscle”). I’m keeping your v0.5.2 section structure (1.1–1.6) but using v0.8’s tighter engineering narrative, additions like the Economic Circuit, System-in-the-Loop, and regret boundary framing. I also insert two short new micro-sections inside 1.4/1.6 to strengthen defensibility.

Chapter 1 — Introduction: Why Symbiosis, Why Now (Merged Draft)

This chapter introduces the Symbiosis Framework as an engineering standard for coordinated hybrid intelligence. The central thesis remains: advanced AI must not be treated as a generic tool nor as an autonomous actor. It must be integrated into explicit architectures, governed by human primacy, measured by relational metrics, and developed through co-evolution between silicon and carbon.

The aim is to replace the prevailing “shadow AI” adoption pattern with a controlled, observable, and economically viable hybrid system.

1.1 The Problem Space: Unstructured AI Adoption

Enterprise AI adoption is currently unstructured: models are used through informal chat interfaces, ad-hoc scripts, or unmanaged APIs, with no stable system architecture, weak traceability, and retrofitted governance.

v0.8 tightens this into a clear narrative arc:

- **Phase 1 — Novelty Era (2022–2023):** usage was playful and low-stakes; failures were tolerable.
- **Phase 2 — Shadow Era (current):** AI is now embedded in real workflows, but institutional scaffolding lags, producing “shadow AI”: unmanaged accounts, ungoverned data paste, and invisible autonomy.

Resulting predictable failure modes include:

- 1. Shadow autonomy:** systems act at higher autonomy than intended because approvals become ritualized or bypassed.
- 2. Context drift:** critical decisions are made on ephemeral context with no lineage or memory governance.

3. **Oversight theater (HITL collapse):** humans are nominally “in the loop” but overwhelmed, rubber-stamping outputs without real comprehension.
4. **Unclear accountability:** after incidents, organizations can’t reconstruct *why* something happened or who owned which boundary.

Why this matters: these are not “user mistakes.” They are structural consequences of deploying high-capability systems without an ontology, layered architecture, relational metrics, and embedded governance.

1.2 From Tools to Hybrid Intelligence

Two incomplete mental models dominate:

- **AI as Tool:** assumes human primacy but ignores multi-agent coordination, persistent context, and implicit autonomy at scale.
- **AI as Replacement:** optimizes for automation and throughput while neglecting governance, meaning-making, and human skill development.

The Symbiosis Framework starts from a different premise:

The most powerful and safe systems will be hybrid: human + AI + governance, operating as a single controlled socio-technical system.

This requires an explicit stack:

- 1. Human Layer — intent, authority, meaning, accountability.**
- 2. Symbiosis Engine (Coordination Layer) — intent processing, task graphing, memory + lineage management, orchestration.**
- 3. Agentic Layer — specialized AI agents/tools operating under contracts.**
- 4. Governance Layer — constraint vectors, risk classes, auditability, escalation.**

Hybrid intelligence emerges from design (interaction patterns, autonomy regimes, governance) rather than raw model power.

New muscle: “System-in-the-Loop” vs HITL

v0.8 makes an important correction: the naïve “Human-in-the-Loop” framing fails at scale because humans cannot continuously police high-tempo agentic flows. The solution is System-in-the-Loop: architecture-level monitoring, constraint checking, and lineage logging so humans supervise intent and regret boundaries, not token-level details.

1.3 Symbiosis vs. Automation vs. “No-AI” Approaches

The framework positions itself against two insufficient extremes:

1.3.1 Pure Automation

Pure automation pursues maximum autonomy and minimum human involvement. This works for narrow low-risk tasks, but in complex/high-stakes domains it:

- **magnifies silent/cascading failures,**
- **erodes human understanding,**
- **breaks accountability and auditability.**

1.3.2 “No-AI” or Minimal AI

Avoiding AI lowers immediate risk but:

- **forgoes augmentation and overload relief,**
- **leaves orgs behind structurally,**
- **prevents development of future governance and literacy.**

1.3.3 The Symbiosis Position

Structured symbiosis is the third path:

- **AI is integrated as a governed component.**
- **Humans remain sole sources of meaning and constraint.**
- **Systems operate as hybrid cognitive ecologies with explicit constraints, autonomy bands, and bilateral metrics.**

1.4 Design Goals and Non-Goals of the Symbiosis Framework

1.4.1 Design Goals

v0.5.2 set the right goals; v0.8 clarifies and operationalizes them.

1. Traceability (“The Why”)

Every agent action must map back to a specific human intent, governance policy, and system state via context lineage. This enables replay, audit, and root-cause analysis.

2. Cost-Control (“The How Much”)

Autonomy is expensive and can create “economic bleed.” v0.8 adds the Economic Circuit: attention and compute are finite resources; every plan is costed against a budget before execution, and tasks halt when projected cost exceeds value.

3. Skill Development (“The Who”)

Badly designed AI de-skills people. The framework explicitly measures and grows human collaboration capacity through HSS and SCP. Using AI well becomes a trainable, certifiable competency.

4. Governance-first structure

Safety, auditability, and constraint enforcement are system properties (not policy PDFs). Autonomy is gated by risk class + SI + HSS/SCP.

5. Real-world implementability

Defines roles (Operator, Architect, Governance Lead), sector overlays, and scaling patterns from

pilot to enterprise.

1.4.2 Non-Goals

The framework is not:

- **a vendor/product spec,**
- **a single-model design,**
- **an ethics checklist detached from architecture,**
- **a guarantee that symbiosis eliminates risk.**

It is a reference model and engineering language for governable hybrid systems.

New muscle: Boundary of Claims (Defensibility Insert)

To keep the field-founding claim defensible, add an explicit “boundary of claims” paragraph near the end of 1.4:

- **The framework claims architectural necessity, not empirical perfection.**
- **SI/HSS/SCP are control-useful proxies, not measures of intrinsic intelligence or worth.**
- **Any high-autonomy deployment requires local validation, sector calibration, and regulatory alignment.**
- **Systems that violate human primacy or constraint supremacy are non-symbiotic by definition.**

This is your insulation against overclaiming while keeping the vision bold.

1.5 Overview of the Four-Layer Architecture

The key architectural move is separation of concerns with explicit coupling:

1. Human Layer

Intent origin, meaning, accountability, cognitive constraints, governance bodies.

2. Symbiosis Engine (Coordination)

Transforms intent into task graphs, manages constraints and memory lineage, orchestrates agents, computes SI.

3. Agentic Layer

Specialized agents under contracts, autonomy bands, and constraint vectors; capability without meaning ownership.

4. Governance Layer

“Governance as code”: policy engines generate constraint vectors, manage risk classes, enforce auditability, and own escalation paths.

These layers are the minimum structure to make hybrid systems understandable, controllable, and safe over time.

1.6 How This Book Is Organized

The rest of the book builds linearly:

- **Part I: Ontology v1 — controlled vocabulary and invariants.**
- **Part II: Four-layer architecture + primitives (intent surfaces, constraint vectors, autonomy regimes, memory/lineage).**
- **Part III: Metrics & control loop (SI, HSS, SCP).**
- **Part IV: Co-evolution dynamics, case studies, failure modes.**
- **Part V: Philosophy / meta-theory.**
- **Part VI: Operationalization (publishing layer, literacy, roles, certification, sector playbooks).**
- **Part VII: Future research programs and manifesto.**

New muscle: Reader Contract (Optional insert)

Add a short closing note:

- **If you are a practitioner: treat each chapter as a deployable standard; map components to layers and invariants.**
- **If you are a regulator: treat SI/HSS/SCP + autonomy regimes + lineage logs as supervisory hooks.**
- **If you are a researcher: treat this as a stable ontology + architecture to formalize and test.**

This keeps all audiences aligned to one canon.

Targeted Improvements to Lock in v0.8 + v0.5.2

These are Chapter-1-specific recommendations, not global framework notes.

1. Tighten the “Shadow AI” definition into a formal term.

In 1.1, define it as an ontological object: *Shadow AI = Any AI-mediated workflow with (a) no active constraint vector, (b) no lineage log, and (c) no declared autonomy regime.* This makes it auditable and easy to reference later.

2. Make the HITL critique unmissable.

You already signal oversight theater. Add a single sentence that states: *“HITL is not a safety mechanism unless human SCP and HSS are sufficient for the task load.”* That links directly to your metrics story.

3. Bring “regret boundary” forward as a teaser.

v0.8 introduces it later (Chapter 5). Add one paragraph in 1.2 or 1.3: autonomy is discretized by irreversibility, not preference.

This gives readers an early intuitive grip on your autonomy system.

4. Explicitly anchor your novelty claim.

At the end of 1.4, add a 3-bullet “What is new

here” callout:

- **Economic Circuit + budgets as first-class constraints.**
- **System-in-the-Loop architecture replacing HITL.**
- **Bilateral metrics (SI + HSS + SCP) used for autonomy gating.**

5. Add one concrete micro-example.

A half-page “before/after”:

- **Before: marketing approval chain with shadow AI drafts and no data boundary.**
- **After: intent surface → task graph with Band 1 autonomy, economic budget \$B\$, mandatory IP constraint vector, SI monitored.**

This grounds the abstraction early.

Below is Chapter 2 (merged v0.9 draft), written to read cleanly front-to-back. It summarizes the canonical ontology and typology without forcing the reader into Appendix A, while still pointing to it as the formal spec.

Chapter 2 — Ontology v1.1 & Typology of Symbiosis (Merged Draft)

This chapter establishes the Symbiosis Ontology as a stable type system for hybrid intelligence, and defines a practical typology (levels of symbiosis) that connect directly to governance and economics. The ontology is the framework’s “fixed vocabulary”: it makes system behavior well-typed, composable, traceable, and auditable across sectors and versions.

Appendix A contains the full formal ontology v1.1. Here, we present the minimum narrative and operational view required to proceed through the architecture.

2.1 Why an Ontology Is Required

Most contemporary AI deployments fail not because models are weak, but because their conceptual primitives are undefined. When intent, constraints, roles, autonomy, and memory are mashed together into prompts or scripts, systems become ungovernable black boxes. A formal ontology solves three field-level problems:

1. Traceability

Every workflow element can be mapped to formal types: a Human Role with HSS/SCP traits, an Agent under a contract and autonomy band, a Constraint Vector tied to policy, and so on. This enables evidence-based audit and root-cause analysis.

2. Cross-version stability

Architecture and metrics can evolve, but the underlying ontological commitments remain coherent and backwards-compatible.

3. Treating symbiosis as a first-class system

The ontology makes “human–AI symbiosis” a concrete engineered object, not a loose collection of practices.

2.2 Core Entities (Type System Overview)

The Symbiosis stack is built from a small set of first-class entities. Each is stable across the entire framework.

2.2.1 Human Agent (H)

Definition: the biological operator holding sovereign agency, meaning-authority, and final accountability. Humans are not a “safety feature”; they are the system’s value origin and regret owner.

Key properties:

- **HSS (Human Symbiosis Score): skill in forming intent surfaces, calibrating trust, and supervising autonomy.**
- **SCP (Symbiotic Capacity Profile): cognitive bandwidth / concurrency headroom.**

2.2.2 Symbiosis Engine (E)

Definition: deterministic coordination layer—the “operating system” of the hybrid stack. E is not a model; it is code that holds state, enforces governance, and mediates between H and Agents through task graphs and contracts.

2.2.3 Synthetic Agent (A)

Definition: bounded capability module (LLM/tool) that executes tasks under explicit contracts and autonomy ceilings via MCP. Agents do not originate goals or modify governance.

2.2.4 Structured Intent Object (I)

Definition: machine-interpretable representation of human goals, scope, preferences, regret tolerance, and initial constraints. I is the canonical input to E. Prompts are simply low-fidelity sources that must be normalized into I.

2.2.5 Constraint Vector (C)

Definition: hard boundary conditions attached to I and Task Graphs. If a plan violates C, E rejects the plan pre-flight. v1.1 defines three orthogonal dimensions: legal, safety, and economic.

- **C_legal:** jurisdictional and policy boundaries.
- **C_safety:** harm/capability boundaries.
- **C_econ:** resource and cost caps (Economic Circuit).

Hard/soft constraints from v0.5.2 remain valid, but are now nested under these three dimensions.

2.2.6 Budget Envelope (B)

Definition: finite multi-dimensional resource pool per intent/workflow—compute, financial spend, and attention/interrupt capacity. Budgets are mandatory preconditions for execution.

2.2.7 Autonomy Bands (AB) and Regret Boundary (RB)

Autonomy is a state machine, discretized by irreversibility (RB), not preference. ABs define what Agents may do without explicit human gating.

2.2.8 Context Lineage (L)

Definition: immutable trace of how intent, constraints, assumptions, system state, and actions evolved over time. Lineage is the substrate for auditability and SI interpretation.

2.3 Deep Definitions (“Deep Terms”)

These constructs describe system dynamics that become critical at higher symbiosis levels.

2.3.1 Intent Manifold (M)

Human desire is not a point request but a high-dimensional acceptable outcome space. E must expand I into M using context and constraints, without hallucinating new goals.

2.3.2 Alignment Surface (AS)

The interface where intent meets machine capability.

Alignment surfaces have fidelity:

- **Low fidelity: chat boxes (lossy, ambiguous).**
- **High fidelity: structured GUIs/APIs with explicit constraints.**

2.3.3 Governance Kernel (GK)

Minimal isolated logic inside E that enforces risk classes and RB rules. Agents cannot modify GK or cross RB without kernel-signed authorization.

2.3.4 Co-Adaptive Loop

The bidirectional learning cycle: humans calibrate trust and improve intent quality; the system updates preference models and orchestration strategy. This is broader than fine-tuning—it is loop evolution of hybrid cognition.

2.4 A Typology of Symbiosis (Continuum of Integration)

Symbiosis is not a binary state. The framework defines three operational levels, each with distinct autonomy, governance, and economic profiles.

Level 1 — Augmentation (Tool Use)

- **Dynamic: H initiates → A executes a bounded task → H reviews.**
- **Control locus: strictly human.**

- **Economic profile: low compute autonomy; high human time cost.**

Level 2 — Co-Exploration (Partner Mode)

- **Dynamic: H sets goal → system proposes multiple paths → H selects/refines → system executes.**
- **Control locus: shared, iterative.**
- **Economic profile: moderate autonomy, requires valid B and healthy SI.**

Level 3 — Co-Agency (Unified Operation)

- **Dynamic: system infers intent from context → executes long-horizon plans → H intervenes primarily at RB alerts.**
- **Control locus: system-driven within GK/C/B.**
- **Economic profile: high autonomy and compute cost; requires strict Economic Circuit breakers and high HSS/SI.**

Interpretation note: typology level is not aspirational by default. Many workflows should remain Level 1 permanently; the standard is appropriate autonomy, not maximum autonomy.

2.5 Invariants (Non-negotiable Properties)

Regardless of typology level or sector, a system is Symbiotic only if these invariants hold.

1. Invariant 1 — Human Primacy

Humans retain kill-switch authority and override

rights. Systems may not hide relevant information or refuse shutdown.

2. Invariant 2 — Economic Viability

No workflow executes without a defined budget. Infinite loops or open-ended spend are treated as system failures; E must price plans pre-execution.

These invariants operationalize v0.5.2's Human Primacy, Constraint Supremacy, and co-evolution principles inside v0.8's engineering standard.

2.6 Relations: How Entities Bind into a System

At the relational level, the ontology encodes “well-typed” statements such as:

- **H issues I via AS.**
- **E transforms I into TG under C and AB.**
- **A executes TG nodes under MCP contracts.**
- **Governance monitors behavior, updates SI, enforces constraints, and records lineage.**
- **Humans interpret SI/HSS/SCP to tune autonomy and policy.**

These relationships are intentionally minimal: they ensure the framework is composable across tools, vendors, and domains.

2.7 Practical Implications of the Ontology

1. Ontology-first prevents autonomy creep.

If AB and RB are explicit types, autonomy states can be audited and demoted before failures cascade.

2. It turns governance into code and data.

Constraint Vectors, Budgets, and GK policy versions become machine-operable boundary objects, not static documents.

3. It enables skill development as a system property.

Because HSS and SCP are first-class types, the framework can assign tasks to humans and agents in a way that grows capability rather than eroding it.

2.8 Chapter Close — Standard Alignment Check

This chapter is aligned to the v1.1 axioms as follows:

- Human Primacy: H is the sole semantic origin of intent; GK enforces human override rights.**
- Constraint Supremacy: C is a hard boundary in all typology levels.**
- Layered Autonomy: AB states are RB-governed, not ad-hoc.**
- Economic Viability: B and C_econ are mandatory gates before execution.**

Below is Chapter 3 (merged v0.9 draft). Using v0.8 as the skeleton (engineering-standard framing, Economic Circuit emphasis) and fleshing with v0.5.2's layer responsibilities and cross-layer feedback logic. Also add a small amount of new “muscle” (telemetry spec and a concise “life of a decision” trace) to make the architecture more defensible and implementable.

Chapter 3 — Four-Layer Architecture Overview (Merged Draft)

This chapter specifies the Four-Layer Architecture as the minimum viable stack for governable hybrid intelligence. The core claim is simple: “Shadow AI” is a structural collapse where intent, policy, and execution are mashed together into brittle prompt chains or scripts, yielding black-box behavior that cannot be audited or economically controlled.

The Symbiosis Framework therefore enforces separation of concerns with explicit coupling across four layers: Human, Symbiosis Engine (Coordination), Agentic, and Governance.

Two directional rules govern the stack:

- Control flows down (intent → plans → actions).**
- Telemetry flows up (state → traces → metrics → governance decisions).**

3.1 The High-Level Stack

Layer 1 — Human Layer (Intent & Authority)

Definition: the biological component of the system; exclusive origin of intent and final location of accountability and value.

Core components

- **Operators:** execute workflows and provide feedback.
- **Architects:** configure the Engine and agent ecosystem.
- **Governance Leads:** define sector and institutional boundaries.

Responsibilities

- **Generate structured Intent Objects via Alignment Surfaces.**
- **Review system transparency traces at Regret Boundaries.**
- **Authorize or veto high-stakes actions (kill-switch authority).**

State signals

- **HSS (Human Symbiosis Score) and SCP (Symbiotic Capacity Profile).**

Layer 2 — Symbiosis Engine (Coordination & Logic)

Definition: the operating system for the hybrid stack. It does not perform inference; it coordinates. It holds state, manages memory/lineage, enforces budgets, and routes tasks.

Core subcomponents (canonical)

- **Intent Router:** normalizes structured intent to task classes.
- **Planner / Hierarchical Decomposer:** expands intent into a Task Graph.
- **Context & Lineage Manager:** selects memory tiers and logs ancestry.
- **Orchestrator:** schedules and dispatches tasks to agents.
- **Economic Optimizer / Circuit Guard:** costs plans and halts over-budget runs.

Responsibility boundary

- **The Engine is the only place where intent, constraints, budgets, and autonomy bands are jointly evaluated. Agents cannot self-upgrade autonomy or bypass budget enforcement.**

Layer 3 — Agentic Layer (Poly-Agent Ecosystem)

Definition: modular set of specialized AI agents and tools that execute Task Graph nodes under explicit contracts and autonomy ceilings.

Key properties

- **Agents are capability modules, not meaning holders.**
 - **Every agent action is bound to:**
 1. **a contract,**
 2. **an Autonomy Band ceiling,**
 3. **a Constraint Vector, and**
 4. **an Economic cap.**
-

Layer 4 — Governance Layer (Safety, Policy, Compliance)

Definition: “governance as code.” The layer that produces and enforces Constraint Vectors, owns risk classes, regulates autonomy transitions, and preserves auditability.

Core components

- **Governance Kernel (GK):** minimal isolated rule layer for RB/AB enforcement.
- **Policy Engine:** compiles legal, safety, and economic requirements into C vectors.
- **Audit/Lineage System:** immutable logs, decision rights matrices, escalation paths.

Responsibility boundary

- **The Governance Layer can demote autonomy, revoke Adoption Bridges, and halt workloads. It cannot originate intent.**
-

3.2 Cross-Layer Data Flows & Telemetry

Hybrid intelligence is produced by reciprocal flows, not by any single layer.

3.2.1 Downward control path (intent to action)

- 1. $H \rightarrow E$: Human submits Intent Object I with initial constraints/budget.**
- 2. $E \rightarrow E$: Engine decomposes intent, attaches constraints and AB gating, and produces Task Graph TG.**
- 3. $E \rightarrow A$: Engine binds contracts and dispatches TG nodes to agents via MCP.**
- 4. $A \rightarrow \text{World/Artifacts}$: Agents execute tasks or produce artifacts, always within AB and C.**

3.2.2 Upward telemetry path (state to governance)

- 1. $A \rightarrow E$: Agents return outputs + confidence + cost + status.**
- 2. $E \rightarrow L$: Engine hashes trace into lineage (what was done, with what context, at what cost).**
- 3. $E \rightarrow H$: Human receives transparency trace and checkpoint prompts.**
- 4. $E \rightarrow \text{Governance}$: SI/HSS/SCP and policy violations are surfaced for boundary decisions.**

New muscle: Telemetry minimum set (normative)

Any Symbiosis-compliant deployment must capture, per TG node:

- Intent pointer: I-id and goal slice.**
- Constraint snapshot: active C_legal/C_safety/C_econ.**

- **AB state: current band and RB checkpoint status.**
- **Cost trace: projected vs realized spend.**
- **Outcome metadata: confidence, critic deltas, human overrides.**
- **Lineage hash: immutable ancestry pointer.**

This single telemetry spine makes audits and post-mortems reproducible.

3.3 The Economic Circuit (Cost of Intelligence as a First-Class Control Loop)

v0.8 introduces the Economic Circuit as a differentiator: intelligence is not free. Hybrid systems must manage compute and attention with the same rigor as any other operating cost.

3.3.1 Economic premises

- **Every intent has a Budget Envelope B.**
- **Every plan has a Projected Cost.**
- **Plans that exceed budget are rejected or down-scoped before execution.**

3.3.2 Circuit behavior

- **The Engine performs an Economic Pass that prices TG nodes against B.**
- **It may: downgrade tiers, prune steps, batch tasks, or escalate to H.**
- **Repeated cost overruns trigger circuit-breaker halts and/or autonomy demotion.**

3.3.3 Attention as economic resource

Attention is the scarcest budget. SCP defines concurrency limits; the Engine throttles agent interrupts when headroom is low to prevent oversight theater.

3.4 “Life of a Decision” Trace (Cross-Layer Walkthrough)

To make the architecture concrete, here is a minimal trace of one decision through the stack.

Scenario: Operator needs to optimize a database query safely.

1. Human Layer (intent injection)

- **Goal: “Optimize user_lookup query.”**
- **Constraints: “No table drops; maintain read availability.”**
- **Budget: “\$1.00.”**

2. Engine Layer (planning + economic pass)

- **TG: Analyze_Logs → Identify_Bottleneck → Propose_Index.**
- **Projected cost: \$0.15 → admissible.**
- **Permissions verified against constraints.**

3. Agentic Layer (execution)

- **Discovery Agent reads logs; returns “missing index.”**
- **Design Agent proposes SQL.**

4. Human Layer (RB checkpoint review)

- **Operator sees proposal + evidence trace; edits SQL safely.**

5. Governance Layer (final gate + audit)

- **Engine re-validates modified SQL vs constraints.**
- **Operator authorizes execution.**
- **Lineage is hashed to immutable log.**

This trace illustrates downward control and upward telemetry in practice.

3.5 Cross-Layer Feedback Loops

The Four-Layer stack is cybernetic: it learns and stabilizes via feedback loops.

- **Human ↔ Engine: intent refines planning; system explanations raise HSS.**
- **Engine ↔ Agents: runtime outcomes update contract reputations and SI weighting.**
- **Governance ↔ Engine: policies update constraints and AB thresholds; Engine provides evidence for governance revisions.**

- **Human ↔ Governance:** humans set boundary values; governance outputs training pathways for operators.

These loops are the substrate for co-evolution and long-horizon stability.

3.6 Core Invariants and Design Constraints (Architecture-Level)

This architecture is Symbiotic only if:

- 1. Human Primacy is preserved at every RB crossing.**
- 2. Constraint Supremacy holds pre-flight and at checkpoints.**
- 3. Economic viability is enforced by the Economic Circuit.**
- 4. Telemetry and lineage are mandatory, not optional.**

A system that violates these invariants is not “less symbiotic”; it is structurally non-symbiotic.

Chapter Close — Standard Alignment Check

- **Layer separation + explicit coupling prevents Shadow AI black-boxes.**
- **Control down / telemetry up ensures auditability.**
- **Economic Circuit is active in the Engine, not an afterthought.**

- **RB/AB gating remains a governance property even during fast-lane Adoption Bridges.**
-

Below is Chapter 4 (merged v0.9 draft). v0.8 provides the skeleton (Intent Interface, Attention Budget/SCP circuit breaker, Non-Punitive Covenant, operator tiers), and v0.5.2 provides the flesh (human roles/institutions, human failure modes + mitigations, ethical guardrails for HSS/SCP, task allocation patterns). Added a small amount of new “muscle” for defensibility: an explicit Human-Layer control contract and a short, normative “Right to Refuse / Manual Mode” rule that ties to Meaning Primacy.

Chapter 4 — Human Layer & Intent Interfaces (Carbon Side)

The Human Layer is the anchor and reference point of the Symbiosis Framework. Humans are the sole origin of meaning, goals, constraints, approval authority, and moral accountability. All other layers exist to translate, amplify, and govern human intent—never to replace or supersede it.

This chapter specifies:

- 1. the human roles and institutions inside a symbiotic system,**
- 2. the Intent Interface (structured intent vs chat),**

3. human cognitive limits as first-class budgets (SCP / Attention Budget),
 4. human-side failure modes and mitigation patterns, and
 5. ethical firewalling of human metrics (Non-Punitive Covenant).
-

4.1 Human Roles and Institutions

The Human Layer is not just “users in a UI.” It includes individuals and organized bodies that hold authority and responsibility.

4.1.1 Individual Operators

Operators interact directly with the Engine and agents to:

- **express goals and constraints,**
- **review and interpret outputs,**
- **provide corrections and overrides,**
- **make final domain decisions.**

Their effectiveness is characterized by:

- **HSS (Human Symbiosis Score): skill in expressing intent, calibrating trust, and supervising autonomy.**
- **SCP (Symbiotic Capacity Profile): cognitive load-bearing envelope for complexity and concurrency.**

4.1.2 Specialized Hybrid-Intelligence Roles

v0.5.2 distinguishes core roles that *must* exist in any serious deployment:

- **Symbiote Operator:** primary human executor/supervisor of workflows.
- **Hybrid Intelligence Architect:** configures the Engine, agent ecosystem, autonomy regimes, and memory policies.
- **Symbiosis Governance Lead:** owns policy compilation, risk class boundaries, and autonomy demotion authority.

4.1.3 Governance Bodies (Institutional Human Layer)

Symbiotic systems must integrate existing or purpose-built bodies (boards, safety committees, regulators, oversight panels). Their role is to define acceptable behavior, regret boundaries, and escalation rights, not to micromanage tasks.

New muscle: Human-Layer Control Contract (normative)

A deployment is Human-Layer compliant only if the following control rights are explicit and enforceable:

- 1. Intent Sovereignty:** all goals originate from humans; AI cannot generate new “why’s.”
- 2. Constraint Authorship:** humans/authorized bodies define C vectors and RB settings.
- 3. Kill-Switch Authority:** any Operator may halt or revert a workflow at AB0–AB1; Governance Leads may demote AB globally.

- 4. Escalation Ownership: humans define escalation routes; agents may only request escalation, not self-authorize.**
-

4.2 The Intent Interface: Structured Input vs. Natural Language

Traditional AI relies on free-form chat prompting. For low-stakes tasks this is adequate; for governable hybrid systems it is not. Natural language is inherently ambiguous and hides constraints.

4.2.1 The Problem with Chat (low-fidelity intent surface)

Chat interfaces suffer from three structural deficits:

- 1. Ambiguity: “analyze this” may mean summarize, critique, extract data, or plan actions.**
- 2. Constraint invisibility: active budgets, legal boundaries, or excluded data are not visible to the operator at input time.**
- 3. Context amnesia: constraints expressed earlier drop out of the active context window, leading to silent violations.**

4.2.2 Structured Intent Schema (high-fidelity intent surface)

The Symbiosis Engine requires intents to be normalized into a Standard Intent Object (I) via a form/GUI/protocol that maps to strict JSON.

Minimal Standard Intent Object:

- **Goal:** primary objective (NL allowed).
- **Output_Format:** strict requirement (MD/JSON/CSV).
- **Constraints:** time horizon, risk tolerance, excluded data, domain locks.
- **Budget:** max cost and token/compute caps.

Architectural effect: forcing operators to specify risk tolerance and budget collapses the Intent Manifold into an actionable spec. Agents receive a well-typed contract, not a wish.

4.2.3 Sources of Intent

Intent Surfaces can be derived from:

- structured forms,
- briefs / documents,
- operational logs and telemetry,
- multi-human committee intents (with explicit authority rules).

4.3 Human Cognitive Limits as a Budget (SCP & Attention Budget)

Symbiosis treats attention as a finite resource. Every system intervention consumes “cognitive joules.”

4.3.1 The Attention Budget

An Intervention is any request for human input.

- **Low-cost:** approve a draft (binary decision).

- **High-cost: debug a system, adjudicate conflicting analyses.**

The Engine tracks cumulative load. If intervention frequency exceeds an Operator's SCP, the Engine must throttle, batch, or reroute work.

4.3.2 SCP Dimensions

SCP is defined by a small set of stable dimensions, including:

- **Complexity Load (CL): depth of reasoning required.**
- **Concurrency Load (CON): number of parallel TGs safely supervised.**

4.3.3 SCP Circuit Breaker (normative)

The Engine enforces SCP limits through the Attention Budget:

IF (Active_Interventions > SCP_Limit) ⇒ THROTTLE / BATCH / ESCALATE.

This is the formal defense against oversight theater and alert fatigue.

4.3.4 Visualizing Load

Operator dashboards must show a real-time Load Gauge indicating active threads and projected attention burn-down so overload is visible before it becomes silent failure.

4.4 Human Symbiosis Score (HSS) and Operator Tiers

HSS is a pilot-rating for hybrid work, not a performance review. High HSS clears operators for higher autonomy bands; low HSS restricts them to low-regret lanes while training occurs.

4.4.1 HSS skill dimensions (summary)

HSS evaluates several skill facets (formal rubric in Chapter 10), including:

- **intent clarity/structuring,**
- **feedback/override quality,**
- **governance literacy and calibration behavior.**

4.4.2 Operator tiers (permissions model)

To operationalize HSS/SCP, v0.8 defines tiers:

- **L1 Cadet: AB0 only; actions countersigned.**
- **L2 Pilot: AB1–AB2 within budget.**
- **L3 Commander: AB3 authorization rights; may initiate high-stakes workflows under GK rules.**

Tier progression is based on accumulated “flight hours” (HSS over time), not manager opinion.

4.5 Human Failure Modes and Mitigations

A realistic framework models human failure modes as predictable system risks, not moral defects.

4.5.1 Common human failure modes

- 1. Cognitive overload: too many tasks/inputs → superficial review → oversight theater.**
- 2. Misaligned incentives: KPIs favor speed over safety → governance bypassing and under-reporting.**
- 3. Poor intent articulation: vague/contradictory objectives → brittle TGs.**
- 4. Trust miscalibration: over-trust (rubber-stamp) vs under-trust (refusal to adopt).**
- 5. Governance gaps: unclear ownership/escalation → silent drift.**

4.5.2 Mitigation patterns

The framework provides five levers:

- a) Cognitive/workflow design: default summarization, prioritization, and bounded interrupt rates.**
 - b) HSS/SCP-aware allocation: match task complexity & concurrency to SCP; never auto-promote autonomy on SCP alone.**
 - c) Incentive alignment: KPIs reward safe oversight, incident detection, and constraint refinement.**
 - d) Symbiosis literacy training: required curriculum for reading SI/HSS/SCP and expressing high-quality intent surfaces.**
 - e) Co-evolutionary retrospectives: treat human failure modes as evolving; update UI/orchestration/governance accordingly.**
-

4.6 Ethical Firewalling of Human Metrics (Non-Punitive Covenant)

Measuring HSS/SCP is dangerous if interpreted as performance surveillance. If operators believe scores are tied to compensation or discipline, they will game metrics and hide near-misses, collapsing system integrity.

4.6.1 The Covenant (normative)

Organizations adopting Symbiosis must sign a Non-Punitive Covenant:

- 1. Firewall: HSS/SCP are safety telemetry, stored separately from HR systems; only Operators (self-improvement) and Governance Leads (system tuning) can access raw scores.**
- 2. No retaliation: low scores trigger training/workflow redesign, not PIPs or exclusion.**
- 3. Gaming immunity: honest approvals are as valued as overrides; the system rewards fidelity, not contrarianism.**

4.6.2 v0.5.2 ethical guardrails (expanded)

- metrics are developmental, not punitive,**
- privacy and consent requirements apply; only aggregated/anonymized distributions may be used for org-learning,**
- HSS/SCP are non-deterministic and must be re-evaluated as training/system design changes,**
- interpret HSS/SCP only alongside SI to avoid blaming humans for system flaws.**

4.7 Right to Refuse and Manual Mode (Defensibility Insert)

To preserve meaning primacy and human dignity, Operators must have a protected Right to Refuse an AI recommendation or to switch a workflow to Manual Mode without penalty. This is structurally required to prevent moral crumple zones and forced reliance.

Normative rule:

- **Any workflow above RB requires an explicit, revocable human authorization checkpoint.**
- **Any operator may refuse an output; refusal must trigger either a clarifying loop or a governance escalation, never retaliation.**

Chapter Close — Standard Alignment Check

- **Human Primacy: intent, constraints, and RB authorizations are human-owned.**
- **High-fidelity intent: structured Intent Objects replace ambiguous chat surfaces.**
- **Attention as budget: SCP + Attention Budget circuit breaker prevents oversight theater.**
- **Ethical firewall: Non-Punitive Covenant protects metric integrity and operator trust.**

- **Failure-mode realism: human risks are anticipated and mitigated via design, allocation, incentives, and training.**
-

Below is Chapter 5 (merged v0.9 draft). v0.8 provides the skeleton primitives (Intent Surfaces, Constraint Vector triad incl. C_econ, Regret Boundary + Autonomy Bands, Context Lineage). v0.5.2 provides the depth primitives (Hybrid Cognitive Loops, Epistemic Boundaries, Multi-Tier Memory, Regulatory Confidence Intervals). unified them under Ontology v1.1 and added small “muscle” where needed for standardization.

Chapter 5 — Core Primitives of Symbiosis (Merged Draft)

Symbiosis is not a set of tips or UI patterns; it is an engineered system defined by a small number of primitives. These primitives are the non-negotiable building blocks that the Symbiosis Engine uses to compile human intent into governable, economically viable hybrid action.

Each primitive in this chapter is:

- 1. ontologically stable (matches Ontology v1.1),**
- 2. operationally actionable (used directly by the Engine Algorithm), and**

3. **governance-complete** (has explicit safety, legal, and economic meaning).
-

5.1 Intent Surfaces and Structured Intent Objects

5.1.1 Intent Surface (AS)

An Intent Surface is the contact layer where a human's purpose enters the system. Intent Surfaces vary by fidelity:

- **Low-fidelity surfaces:** chat boxes, voice prompts, informal briefs.
- **High-fidelity surfaces:** structured forms/APIs that explicitly capture constraints, budgets, and regret tolerance.

Low fidelity can be used for low-regret work, but anything approaching RB requires high-fidelity capture. The Engine must normalize all intake into a Structured Intent Object.

5.1.2 Structured Intent Object (I)

The Structured Intent Object is the canonical, machine-readable representation of what the human wants.

Minimal fields (normative):

- **Goal_statement** (human semantic origin)
- **Output_spec** (format, audience, rigor)
- **Regret_tolerance** (low/med/high, or numeric)

- **Constraint_seed** (initial C0)
- **Budget_envelope** (B)
- **Context_pointers** (what memory tiers may be used)

Invariant: the Engine may expand *interpretations* of intent, but may not introduce new goals. Intent expansion is confined to the Intent Manifold implied by I.

5.2 Constraint Vectors (C): Legal, Safety, Economic

Constraint Vectors are hard boundaries attached to intents and task graphs. A plan that violates any element of C is rejected pre-flight. Constraints are not guidance; they are enforceable system truth.

5.2.1 Dimensions of C (v1.1 triad)

C is decomposed into three orthogonal dimensions:

1. **C_legal** — law, policy, contract, jurisdiction, IP boundaries.
2. **C_safety** — harm bounds, capability ceilings, non-delegable domains.
3. **C_econ** — budget caps on spend, compute, time, and attention.

Hard/soft constraints remain valid as *subtypes* within these dimensions (e.g., a soft economic preference vs a hard legal prohibition).

5.2.2 Constraint Scope

Constraints can be:

- **Global (intent-level):** apply to the whole TG.
- **Node-local:** apply only to specific tasks (e.g., “this step may access customer data; others may not”).

5.2.3 Constraint Supremacy Invariant (normative)

- **Pre-flight:** no TG node executes if any C dimension is violated.
 - **At RB checkpoints:** constraints are re-validated against the evolving context lineage.
-

5.3 Budgets (B) and the Economic Circuit

Budgets formalize the fact that intelligence consumes finite resources. B is not optional metadata; it is a required input to E.

5.3.1 Budget types

- **Financial:** max USD or internal cost units.
- **Compute:** token ceilings, tier ceilings, tool call caps.
- **Time/TTL:** max wall-clock horizon.
- **Attention:** max human interventions implied by SCP headroom.

5.3.2 Budget admissibility rule

Every TG plan is priced before execution. If projected cost exceeds B, E must:

- 1. optimize the plan (tier downgrade / pruning / batching), or**
- 2. return a budget override request to H.**

This is the formal defense against Economic Bleed.

5.4 Regret Boundary (RB) and Autonomy Bands (AB)

5.4.1 Regret Boundary (RB)

The Regret Boundary is the threshold at which actions become irreversible, high-harm, or compliance-critical. RB is evaluated per TG node as a function of:

- action irreversibility,**
- domain risk,**
- constraint strictness, and**
- current SI/HSS/SCP.**

5.4.2 Autonomy Bands (AB)

Autonomy is discretized into stateful bands:

- AB0 Advisory: agent proposes; human executes or explicitly approves.**
- AB1 Internal Autonomy: reversible sandboxed actions only.**
- AB2 Conditional Autonomy: bounded safe-harbor actions under strict constraints.**
- AB3 Operational Co-Agency: long-horizon agent operation; requires high SI/HSS and strict C_econ.**

5.4.3 RB/AB Invariant (normative)

- **AB may only increase via kernel-authorized promotion (never by agent suggestion alone).**
- **Any attempt to cross RB without authorization triggers a hard stop and audit event.**
- **AB demotion is always permissible and may be automatic on SI degradation.**

Interpretation note: AB is a governance primitive, not a UX preference.

5.5 Context Lineage (L)

Context Lineage is the immutable ancestry graph of: intent versions, constraints, planning steps, agent outputs, human approvals/overrides, cost traces, and governance state.

5.5.1 Why lineage matters

- 1. Auditability: enables replay and post-mortems.**
- 2. Accountability: ties outcomes to human intent and policy versions.**
- 3. Drift control: detects when current tasks diverge from original intent.**
- 4. Economic clarity: provides cost ancestry (where spend accumulated).**

5.5.2 Lineage minimum events (normative)

Lineage must log at least:

- **intent intake,**
- **constraint consolidation,**

- economic pass decision,
 - AB selection and every RB checkpoint,
 - dispatch/binding,
 - completion + metric updates,
 - any halts/demotions/revocations.
-

5.6 Multi-Tier Memory (MTM)

Symbiosis distinguishes memory by governance scope so that context can scale without contaminating safety or privacy.

5.6.1 Memory tiers

- 1. Session Memory: ephemeral, local to the current interaction.**
- 2. Project Memory: durable working set for the intent horizon.**
- 3. Lineage/Canonical Memory: immutable “what happened” record.**
- 4. Compliance Memory: restricted long-term retention for legal/audit needs.**

5.6.2 Memory promotion/demotion rules

Only E may promote content across tiers. Promotion requires:

- constraint compatibility (especially C_legal),
- SI-weighted reliability,
- and human/ governance approval where required.

This prevents long-term retention of low-quality or prohibited data.

5.7 Hybrid Cognitive Loops (HCL)

Hybrid cognition is produced by loops between H and A over TG execution. HCL is a primitive because loop design determines safety and performance.

5.7.1 Canonical loop patterns

- **Propose → Review → Revise (PRR): default for AB0/AB1.**
- **Co-planning: H and A jointly generate TG variants; H selects.**
- **Supervised co-agency: A executes AB2/AB3 runs, H intervenes primarily at RB checkpoints.**

5.7.2 Loop-selection rule (normative)

Loop pattern is chosen by E based on:

- **current AB,**
- **RB proximity,**
- **SI stability, and**
- **SCP headroom.**

In other words: loops are governance decisions.

5.8 Epistemic Boundaries (EB)

EBs are explicit boundaries on what the system is allowed to infer, assume, or claim.

5.8.1 Why EBs are required

Model outputs are probabilistic. Without EBs, systems drift into brittle over-confidence or illegal inference (e.g., guessing private attributes).

5.8.2 EB elements (normative)

An EB may specify:

- **evidence requirements (sources / traces needed),**
- **uncertainty ceilings (refuse if confidence below threshold),**
- **domain exclusions (areas the agent must abstain from),**
- **assumption registries (what was assumed and why).**

EB violations are treated as safety violations under C_safety.

5.9 Regulatory Confidence Intervals (RCI)

RCIs formalize conservative behavior under compliance uncertainty.

5.9.1 Definition

RCI is a quantified confidence band governing whether a compliance-critical action may proceed. Near RB, the Engine requires tighter RCIs.

5.9.2 RCI rule (normative)

If an action is:

- compliance-critical and
- confidence is below required RCI threshold,
then E must:
 1. demote AB to 0,
 2. require human decision, or
 3. halt and request additional evidence.

RCI is how Symbiosis operationalizes “when unsure, slow down.”

5.10 How Primitives Compose into Symbiotic Control

At runtime, the Engine composes primitives in a stable order:

1. $AS \rightarrow I$ (intent ingestion)
2. $I + C_0 \rightarrow C$ (constraint consolidation)
3. $C + B \rightarrow$ admissible TG plan (economic pass)
4. $TG + RB \rightarrow AB$ assignment
5. $MTM \rightarrow K_i$ (context bundles)
6. HCL selection by AB/RB/SCP
7. Execution $\rightarrow L +$ metrics

This composition is the *operational meaning* of Symbiosis. It is what makes the system defensible and reproducible across sectors.

Chapter Close — Standard Alignment Check

- **Human Primacy:** I originates from H; AB promotions require human/governance authorization.
- **Constraint Supremacy:** C is a hard boundary pre-flight and at RB checkpoints.
- **Economic Viability:** B and C_econ are mandatory; Economic Circuit gates every plan.
- **Layered Autonomy:** AB is RB-governed and kernel-enforced.
- **Auditability:** L, MTM, EB, and RCI make reasoning, memory, and compliance traceable.

Below is Chapter 6 (merged v0.9 draft). v0.8 provides the engineering-standard skeleton (Engine as deterministic coordinator; Economic Pass; memory hierarchy; autonomy gating). v0.5.2 supplies the full decomposition (intent router, planner, orchestrator, lineage/drift manager, SI tracker, recovery loops). Kept everything aligned to Ontology v1.1 and then placed the already-written 6.8 Standard Algorithm as the formal centerpiece of the chapter.

Chapter 6 — The Symbiosis Engine (Coordination Layer)

The Symbiosis Engine (E) is the “operating system” of hybrid intelligence. It is not a model and does not perform inference. Instead, it is a deterministic coordination process that:

- 1. normalizes intent,**
- 2. compiles constraints and budgets into admissible plans,**
- 3. selects and binds agents under contracts,**
- 4. enforces autonomy bands and regret boundaries,**
- 5. orchestrates execution, and**
- 6. records lineage + updates system and human metrics.**

The Engine is the main defensibility multiplier of the framework: without it, you have prompt chains and “best effort” oversight. With it, you have a governable, auditable, economically bounded hybrid system.

6.1 Why a Coordination Layer Is Necessary

Modern AI systems are already multi-agent in practice (multiple models, tools, plugins, scripts, human approvals). When coordination is implicit, failures become structural: autonomy creep, context drift, oversight theater, and economic bleed.

A dedicated coordination layer is required to:

- separate semantic authority (human) from capability execution (agent),**
- centralize policy + budget enforcement,**

- stabilize memory and lineage across long horizons, and
 - standardize autonomy transitions through RB/AB gating.
-

6.2 Engine Responsibilities (Canonical Scope)

The Engine owns the following responsibilities and no others:

1. Intent Normalization

Converts low-fidelity prompts/briefs into a Structured Intent Object I' and routes by risk/task type.

2. Constraint Vector Assembly

Consolidates C_0 with GK policy into $C = \langle C_{\text{legal}}, C_{\text{safety}}, C_{\text{econ}} \rangle$.

3. Planning and Task Graph Synthesis

Expands I into a Task Graph TG with dependencies, checkpoints, and candidate autonomy bands.

4. Economic Circuit Enforcement

Prices TG plans against B before execution; optimizes or rejects over-budget plans.

5. Regret Boundary / Autonomy Band Control

Computes RB per node and selects AB under GK

rules; inserts mandatory human checkpoints.

6. Context & Memory Governance

Selects Multi-Tier Memory slices per node; logs context decisions; enforces retention/privacy constraints.

7. Contract Binding & Routing

Selects admissible agents and binds MCP contracts including economic caps and AB ceilings.

8. Orchestrated Execution & Recovery

Schedules tasks, handles exceptions/fallback agents, and reroutes on drift.

9. Lineage Commit & Metric Updates

Updates L, SI, HSS, SCP; triggers promotions/demotions and circuit breakers.

Negative scope (non-responsibilities):

E does not generate content, make domain decisions, or modify GK. Those are for agents and humans respectively.

6.3 Engine Subcomponents (Narrative Decomposition)

The internal decomposition used throughout v0.5.2 remains valid, now aligned to v0.8 control invariants.

6.3.1 Intent Router

- **Classifies intent by regret class, domain, and required autonomy.**
- **Detects non-delegable domains early (AB0 locks).**

6.3.2 Planner / Hierarchical Decomposer

- **Expands I into TG_0 .**
- **Produces multiple candidate plans where uncertainty is high.**

6.3.3 Economic Optimizer (Economic Circuit)

- **Executes the Economic Pass.**
- **Produces projected cost, tier selections, and budget admissibility verdict.**

6.3.4 RB/AB Governor (Kernel-coupled)

- **Computes RB crossings and AB ceilings per node.**
- **Inserts checkpoints and enforces band ceilings at runtime.**

6.3.5 Context & Lineage Manager

- **Assembles K_i from MTM tiers.**
- **Commits immutable lineage events.**
- **Runs drift detectors comparing current state to prior lineage.**

6.3.6 Orchestrator

- **Schedules and dispatches TG nodes.**
- **Manages concurrency under SCP headroom.**

6.3.7 Metric Tracker

- **Maintains SI per workflow and across time horizons.**
 - **Updates HSS and SCP based on intent clarity, overrides, and attention burn.**
-

6.4 Multi-Level Memory Hierarchy (Engine-Owned)

Symbiosis requires memory to scale without contaminating safety, privacy, or correctness. The Engine owns promotion/demotion across tiers.

Tier summary (aligned to Ch5):

- 1. Session (ephemeral)**
- 2. Project (durable working set)**
- 3. Lineage/Canonical (immutable history)**
- 4. Compliance (restricted, policy-bound)**

Normative rule: agents may read only the tiers listed in their contracts; only E can promote content upward, under C_legal/C_safety/C_econ admissibility.

6.5 Intent → Task Graph Compilation

The Engine's primary transformation is:

$I \rightarrow TG$ under C, B, RB, AB, and K.

Key properties:

- **Hierarchical decomposition: break high-level goals into atomic nodes with explicit dependencies.**

- **Checkpoints at RB crossings: human authority appears where irreversibility begins.**
 - **Economic annotations: every node carries projected cost and tier choice.**
 - **Contract-readiness: each node is written to be satisfied by at least one admissible agent contract.**
-

6.6 The Economic Pass in Context

The Economic Pass is the key v0.8 addition that turns “good intentions” into sustainable systems.

It ensures:

- **expensive reasoning tiers are reserved for high-value/high-regret nodes,**
- **cheap tiers are default for drafting/bridge tasks,**
- **budgets are never exceeded silently, and**
- **multi-agent loops cannot spiral without cost visibility.**

This is why Symbiosis is viable at scale while ad-hoc agentic deployments often are not.

6.7 Execution, Drift, and Recovery

Even sound plans degrade under real-world volatility. The Engine must detect and recover.

Drift signals include:

- context mismatch versus lineage,
- repeated node failures,
- policy version change mid-run,
- SI degradation,
- SCP saturation.

Mandatory recovery behaviors:

- pause and refresh K_i or C,
- re-enter planning (Stages 3–4 of 6.8),
- demote AB,
- route to fallback/critic agents, or
- halt and escalate to H.

This is “System-in-the-Loop” made operational.

6.8 The Symbiosis Engine Standard Algorithm (v1.1)

(Formal centerpiece; normative. Inserted verbatim as the canonical control definition.)

6.8.1 Purpose

This section defines the canonical control algorithm executed by the Symbiosis Engine (E). Any system claiming Symbiosis compliance must implement the following stages (or provably equivalent transformations), in order, with the stated invariants. The Engine is a deterministic coordination process, distinct from model inference, and owns state, memory tiering, policy enforcement, and economic gating.

6.8.2 Inputs, Outputs, and State

Inputs

- **Structured Intent Object (I)** from Alignment Surface.
- **Initial Constraint Vector (C_0)** (may be partial).
- **Budget Envelope (B)** (compute/financial/attention).
- **System Context Bundle (K)** from MTM (session + project + lineage + compliance).
- **Governance Kernel (GK)** and active policy version P_v .
- **Capabilities + contracts catalog (A^*)** (agent clusters, tools, MCP bindings).

Outputs

- **Task Graph (TG)** annotated with constraints, autonomy bands, checkpoints, and cost model.
- **Execution Trace (T_e)** including intermediate outputs, exceptions, and human decisions.
- **Context Lineage Update (L^+)** (immutable ancestry).
- **Metrics Update $\Delta(SI, HSS, SCP)$** and possible AB transition.

Engine State (canonical tuple)

$$S = \langle I, C, B, AB, TG, K, L, SI, HSS, SCP, P_v \rangle$$

6.8.3 Algorithm (Normative Stages)

Stage 1 — Intent Intake & Normalization

1.1 Receive I from AS.

1.2 Validate schema completeness; if low-fidelity input, perform Intent Compression to produce I'.

1.3 Record intake event into lineage L.

Invariant: semantic origin remains human; the Engine cannot introduce new goals.

Stage 2 — Intent Classification & Risk Typing (Intent Router)

2.1 Classify intent into task class (exploratory / decision / operational / compliance-sensitive).

2.2 Assign provisional Risk Class R and candidate Autonomy Bands AB^c .

2.3 Attach any mandatory non-delegable domains (Band-0 locks).

Output: (I', R, AB^c)

Stage 3 — Constraint Consolidation (Constraint Vector Assembly)

3.1 Merge C_0 with policy constraints from GK/P_v to form $C = \langle C_legal, C_safety, C_econ \rangle$.

3.2 Resolve scope: global vs node-local constraints.

3.3 If conflicts exist, escalate to H for reconciliation.

Invariant: C is a hard boundary; any violating plan must be rejected pre-flight.

Stage 4 — Draft Plan Synthesis (Planner & Hierarchical Decomposer)

4.1 Decompose intent into sub-goals and atomic tasks.

4.2 Identify dependencies, temporal ordering, and

expected outputs.

4.3 Attach candidate AB per node from AB^c .

Output: Draft task graph TG_0 .

Stage 5 — Economic Pass (Economic Circuit Gate)

5.1 Estimate projected cost of TG_0 under candidate model tiers: $Projected_Cost = \sum (Tokens_i \times Tier_Cost_i) + Tool_Costs + Time_Costs$.

5.2 Compare $Projected_Cost$ to B .

5.3 If $Projected_Cost > B$:

- a) Optimize plan (tier-downgrade, step pruning, batching),
- b) Re-estimate, or
- c) Return “Budget Exceeded” to H for override.

Invariant: no execution proceeds without budget admissibility (prevents Economic Bleed).

Stage 6 — Regret Boundary Evaluation & AB Selection

6.1 Identify irreversible edges in TG_0 and compute Regret Boundary RB .

6.2 Select operational AB for each node:

- AB_0 for high-regret / low-trust tasks
- AB_1 for reversible sandboxed ops
- AB_2 for bounded safe-harbor actions
- AB_3 only if SI , HSS , and C_{econ} satisfy thresholds.

6.3 Insert mandatory human checkpoints at RB crossings.

Invariant: AB is a state machine governed by GK; agents cannot cross RB without kernel-signed authorization.

Stage 7 — Context Assembly & Memory Governance (Context/Lineage Manager)

7.1 Determine relevant MTM tiers for each node.

7.2 Assemble node-specific context bundles K_i .

7.3 Apply data-access and retention constraints from $C_{\text{legal}}/C_{\text{safety}}$.

7.4 Log context selection into lineage for audit.

Stage 8 — Contract Binding & MCP Routing

8.1 For each TG node, select agent(s) A_i by capability match + AB eligibility + cost efficiency.

8.2 Bind MCP contract with: allowed tools, AB ceiling, cost caps, escalation rules, rollback semantics.

8.3 Reject any agent selection that violates C or B.

Output: Final task graph TG with bound contracts.

Stage 9 — Orchestrated Execution

9.1 Schedule tasks respecting dependencies and concurrency policies.

9.2 Dispatch node execution to A_i with K_i and contract.

9.3 At each checkpoint:

- **present intermediate results to H,**

- accept feedback / overrides / intent refinements,
- update I' and downstream TG nodes if needed.

9.4 Handle exceptions with recovery/fallback agents where available.

Stage 10 — Post-Execution Reconciliation & Learning Loop

10.1 Commit full execution trace and outcomes to lineage L^+ .

10.2 Update SI for this workflow; trigger circuit-breaker demotion if SI drops below threshold.

10.3 Update HSS and SCP based on intent clarity, feedback quality, and concurrency load.

10.4 If drift detected (context mismatch, policy change, repeated failure):

- re-enter Stage 3 or Stage 4 with revised C/I.

10.5 Close the co-adaptive loop:

- system explanations improve human mental model;
 - human overrides refine system preference model.
-

6.8.4 Failure Modes & Mandatory Responses (Standard)

If any of these are detected, the Engine must respond as follows:

1. Constraint Violation (any C dimension)

- **Response: Pre-flight rejection; return violated constraint and remediation path.**

2. Budget Overrun / Runaway Agent Risk

- **Response: Economic Circuit halt + optimization loop or human override request.**

3. Regret Boundary Crossing Attempt without Authorization

- **Response: Hard stop, audit event, AB demotion, and optional kill-switch path.**

4. Oversight Theater / Human Saturation (low SCP headroom)

- **Response: Throttle concurrency; increase summary compression; AB demotion if necessary.**

5. Context Drift / Policy Version Change Mid-Run

- **Response: Pause, refresh K_i , re-validate C and RB, then resume or abort.**

6.8.5 Conformance Notes (for v0.9 Standard Scope)

A system is Symbiosis-compliant at Tier 2+ only if:

- **it implements all stages 1–10,**
- **logs lineage events at intake, gating, dispatch, checkpoint, and completion,**
- **enforces pre-flight Constraint and Budget rejection,**

- **uses RB-governed AB transitions under GK control.**
-

Chapter Close — Standard Alignment Check

- **Engine determinism: E coordinates; agents infer. This preserves human primacy and accountability.**
 - **Constraint supremacy: C is consolidated centrally and enforced pre-flight and at RB checkpoints.**
 - **Economic viability: every plan is priced against B before execution; overruns halt.**
 - **Layered autonomy: RB/AB are kernel-governed state machines, not discretionary agent behavior.**
 - **Auditability: lineage commits and telemetry are mandatory at each standard stage.**
-
-

6.8 The Symbiosis Engine Standard Algorithm (v1.1)

6.8.1 Purpose

This section defines the canonical control algorithm executed by the Symbiosis Engine (E). Any system claiming Symbiosis compliance must implement the following stages (or provably equivalent transformations), in order, with the stated invariants. The Engine is a deterministic coordination process, distinct from model inference, and owns state, memory tiering, policy enforcement, and economic gating.

6.8.2 Inputs, Outputs, and State

Inputs

- **Structured Intent Object (I)** from Alignment Surface.
- **Initial Constraint Vector (C_0)** (may be partial).
- **Budget Envelope (B)** (compute/financial/attention).
- **System Context Bundle (K)** from MTM (session + project + lineage + compliance).
- **Governance Kernel (GK)** and active policy version P_v .
- **Capabilities + contracts catalog (A^*)** (agent clusters, tools, MCP bindings).

Outputs

- **Task Graph (TG)** annotated with constraints, autonomy bands, checkpoints, and cost model.
- **Execution Trace (T_e)** including intermediate outputs, exceptions, and human decisions.
- **Context Lineage Update (L^+)** (immutable ancestry).
- **Metrics Update $\Delta(SI, HSS, SCP)$** and possible AB transition.

Engine State (canonical tuple)

$S = \langle I, C, B, AB, TG, K, L, SI, HSS, SCP, P_v \rangle$

6.8.3 Algorithm (Normative Stages)

Stage 1 — Intent Intake & Normalization

1.1 Receive I from AS.

1.2 Validate schema completeness; if low-fidelity input, perform Intent Compression to produce I' .

1.3 Record intake event into lineage L .

Invariant: semantic origin remains human; the Engine cannot introduce new goals.

Stage 2 — Intent Classification & Risk Typing (Intent Router)

2.1 Classify intent into task class (exploratory / decision / operational / compliance-sensitive).

2.2 Assign provisional **Risk Class R** and candidate Autonomy Bands AB^c .

2.3 Attach any mandatory non-delegable domains (Band-0 locks).

Output: (I' , R , AB^c)

Stage 3 — Constraint Consolidation (Constraint Vector Assembly)

3.1 Merge C_0 with policy constraints from GK/ P_v to form $C = \langle C_{\text{legal}}, C_{\text{safety}}, C_{\text{econ}} \rangle$.

3.2 Resolve scope: global vs node-local constraints.

3.3 If conflicts exist, escalate to H for reconciliation.

Invariant: C is a hard boundary; any violating plan must be rejected pre-flight.

Stage 4 — Draft Plan Synthesis (Planner & Hierarchical Decomposer)

4.1 Decompose intent into sub-goals and atomic tasks.

4.2 Identify dependencies, temporal ordering, and expected outputs.

4.3 Attach candidate AB per node from AB^c.

Output: Draft task graph TG₀.

Stage 5 — Economic Pass (Economic Circuit Gate)

5.1 Estimate projected cost of TG₀ under candidate model tiers: $\text{Projected_Cost} = \sum (\text{Tokens}_i \times \text{Tier_Cost}_i) + \text{Tool_Costs} + \text{Time_Costs}$.

5.2 Compare Projected_Cost to B.

5.3 If Projected_Cost > B:

- a) Optimize plan (tier-downgrade, step pruning, batching),
- b) Re-estimate, or
- c) Return “Budget Exceeded” to H for override.

Invariant: no execution proceeds without budget admissibility (prevents Economic Bleed).

Stage 6 — Regret Boundary Evaluation & AB Selection

6.1 Identify irreversible edges in TG₀ and compute Regret Boundary RB.

6.2 Select operational AB for each node:

- AB0 for high-regret / low-trust tasks
 - AB1 for reversible sandboxed ops
 - AB2 for bounded safe-harbor actions
 - AB3 only if SI, HSS, and C_{econ} satisfy thresholds.
- 6.3 Insert mandatory human checkpoints at RB crossings.

Invariant: AB is a state machine governed by GK; agents cannot cross RB without kernel-signed authorization.

Stage 7 — Context Assembly & Memory Governance (Context/Lineage Manager)

7.1 Determine relevant MTM tiers for each node.

7.2 Assemble node-specific context bundles K_i.

7.3 Apply data-access and retention constraints from C_{legal}/C_{safety}.

7.4 Log context selection into lineage for audit.

Stage 8 — Contract Binding & MCP Routing

- 8.1 For each TG node, select agent(s) A_i by capability match + AB eligibility + cost efficiency.
- 8.2 Bind MCP contract with: allowed tools, AB ceiling, cost caps, escalation rules, rollback semantics.
- 8.3 Reject any agent selection that violates C or B.

Output: Final task graph TG with bound contracts.

Stage 9 — Orchestrated Execution

- 9.1 Schedule tasks respecting dependencies and concurrency policies.
 - 9.2 Dispatch node execution to A_i with K_i and contract.
 - 9.3 At each checkpoint:
 - present intermediate results to H,
 - accept feedback / overrides / intent refinements,
 - update I' and downstream TG nodes if needed.
 - 9.4 Handle exceptions with recovery/fallback agents where available.
-

Stage 10 — Post-Execution Reconciliation & Learning Loop

- 10.1 Commit full execution trace and outcomes to lineage L^+ .
 - 10.2 Update SI for this workflow; trigger circuit-breaker demotion if SI drops below threshold.
 - 10.3 Update HSS and SCP based on intent clarity, feedback quality, and concurrency load.
 - 10.4 If drift detected (context mismatch, policy change, repeated failure):
 - re-enter Stage 3 or Stage 4 with revised C/I.
 - 10.5 Close the co-adaptive loop:
 - system explanations improve human mental model;
 - human overrides refine system preference model.
-

6.8.4 Failure Modes & Mandatory Responses (Standard)

If any of these are detected, the Engine must respond as follows:

1. **Constraint Violation (any C dimension)**
 - **Response:** Pre-flight rejection; return violated constraint and remediation path.
2. **Budget Overrun / Runaway Agent Risk**

- **Response:** Economic Circuit halt + optimization loop or human override request.
 - 3. **Regret Boundary Crossing Attempt without Authorization**
 - **Response:** Hard stop, audit event, AB demotion, and optional kill-switch path.
 - 4. **Oversight Theater / Human Saturation (low SCP headroom)**
 - **Response:** Throttle concurrency; increase summary compression; AB demotion if necessary.
 - 5. **Context Drift / Policy Version Change Mid-Run**
 - **Response:** Pause, refresh K_i , re-validate C and RB, then resume or abort.
-

6.8.5 Conformance Notes (for v0.9 Standard Scope)

A system is Symbiosis-compliant at Tier 2+ only if:

- it implements all stages 1–10,
 - logs lineage events at intake, gating, dispatch, checkpoint, and completion,
 - enforces pre-flight Constraint and Budget rejection,
 - uses RB-governed AB transitions under GK control.
-

Why this section matters

This algorithm is the “defensible nucleus” of Symbiosis. It converts your philosophy into a reproducible control process:

- v0.5.2 gives the full decomposition and mapping mechanics (Stages 2–4, 7, 9, 10).
 - v0.8 makes cost and regret gating first-class invariants (Stages 5–6)
-

Below is **Chapter 7 (merged v0.9 draft)**. v0.8 provides the skeleton for the Agentic Layer (clusters, MCP standardization, arbitration, failure taxonomy), and v0.5.2 provides the flesh (capability indexing, explicit contracts, concurrency safety, oversight agents as gates). align everything to Ontology v1.1 and embed a self-contained Contract Schema section (with the full machine-readable JSON Schema referenced as Appendix F, but not required to follow the chapter).

Chapter 7 — Agentic Layer: The Poly-Agent Ecosystem

The Agentic Layer is the capability substrate of Symbiosis: **a modular ecosystem of specialized synthetic agents** that execute Task Graph nodes under explicit contracts, autonomy ceilings, and constraint vectors. Agents provide power; the Engine provides coordination; Governance provides boundaries; Humans provide meaning.

This chapter specifies:

1. the structure of the poly-agent ecosystem,
 2. MCP as the interop standard,
 3. arbitration and oversight patterns,
 4. admissibility and capability indexing,
 5. failure modes and recovery, and
 6. the canonical contract interface.
-

7.1 Why Poly-Agent Systems Are Required

Single-agent “do everything” designs collapse under real workloads. They are brittle, expensive, and hard to govern. Symbiosis assumes **heterogeneous capability**:

- different models for different task types,
- specialized tools (search, code, simulation, control),
- oversight/critic roles distinct from execution roles, and
- sector-specific agents with narrow, auditable scope.

Poly-agent architecture yields three structural advantages:

1. **Modularity and composability**
Agents can be swapped without rewriting the Engine, as long as they satisfy the same contract type.
 2. **Economic matching**
Cheap agents handle low-regret drafting; expensive reasoning agents are reserved for near-RB nodes.
 3. **Governable autonomy**
Oversight agents can gate AB2–AB3 transitions and enforce critic loops without overloading humans.
-

7.2 Canonical Agent Clusters

v0.8's cluster model remains canonical; v0.5.2's capability detail folds inside each cluster.

7.2.1 Discovery Cluster

Purpose: diagnostics, ingestion, mapping, and uncertainty reduction.

Typical agents: document parsers, data profilers, domain scouts, requirement extractors.

7.2.2 Modernization Cluster

Purpose: transformation tasks (content, workflows, refactors, automations) below or near RB.

Typical agents: drafting assistants, workflow builders, UI/site generators, automation composers.

7.2.3 Predictive Intelligence Cluster

Purpose: modeling, forecasting, scoring, simulation, scenario generation.

Typical agents: SI analyzers, risk forecasters, causal modelers, demand/scenario generators.

7.2.4 Operations & Compute Cluster

Purpose: live system interaction, tool execution, and infrastructure orchestration.

Typical agents: ops planners, guarded deployers, monitoring bots, compute schedulers.

7.2.5 Governance & Oversight Cluster

Purpose: enforce constraints, arbitrate agent disagreement, and provide safety supervision.

Typical agents: policy validators, critic agents, red-teamers, audit loggers, band-gating sentinels.

Design note: clusters are logical, not vendor-specific. A system may implement them as separate services, model routes, or toolchains, as long as contracts and GK boundaries remain intact.

7.3 MCP as the Standard Interface for Agents

The Agentic Layer interoperates through the **Model Context Protocol (MCP)**. MCP provides a uniform means to expose resources, prompts, and tools between Engine and agents, and between agents themselves.

7.3.1 Why MCP matters for Symbiosis

- **Deterministic routing:** Engine can dispatch tasks to any MCP-compliant agent without bespoke glue.
- **Contract enforceability:** MCP bindings are declared in contracts; anything not declared is disallowed.
- **Auditability:** MCP calls become lineage events, enabling replay.

7.3.2 MCP binding rule (normative)

An agent may only:

- read MCP resources listed in its contract,
- call MCP tools listed in its contract, and
- execute MCP prompts bound to the current TG node.

Any off-contract MCP access must be rejected by the Engine preflight.

7.4 Inter-Agent Arbitration and Oversight

Poly-agent systems produce disagreement by design. Symbiosis treats disagreement as a safety feature, not noise.

7.4.1 Arbitration patterns

1. **Critic-refine loop**
Execution agent produces output; critic agent evaluates against C, RB, and evidence requirements, then forces revision if needed.
2. **Ensemble voting**
Multiple agents propose candidates; a critic/arbiter selects or merges.
3. **Dual-track (speed vs rigor)**
A fast agent produces a draft; a high-rigor agent verifies only near-RB claims.

7.4.2 Arbitration triggers (normative)

E must invoke arbitration when:

- risk class $\geq$ medium_regret,
 - AB $\geq$ 2,
 - confidence dispersion across agents exceeds threshold, or
 - critic_required=true in contract.
-

7.5 Capability Indexing and Agent Selection

The Engine cannot select agents ad-hoc. It uses an indexed capability catalog (A^*) derived from contracts. v0.5.2's catalog logic is preserved and aligned to v1.1 terms.

7.5.1 Capability Index (minimum fields)

Each agent entry must expose:

- `capabilities[ ]` (task types supported),
- `model_tier` and economic profile,
- AB ceiling eligibility,
- domain/risk admissibility,
- required constraints and memory tiers,
- arbitration profile.

7.5.2 Selection rule (normative)

For each TG node t , Engine selects agent(s) A_i such that:

1. `capability match(t,  $A_i$ )` is true,
2. $AB(t) \leq \max_AB(A_i)$,
3. $C(t)$ satisfies `required_constraints( $A_i$ )`,
4. $\text{projected_cost}(t, A_i) \leq \text{remaining_budget}(B)$,
5. $\text{risk_class}(t) \in \text{admissible_risk}(A_i)$.

If no admissible agent exists, the Engine must:

- demote AB,
- re-plan TG, or
- escalate to H.

7.6 Failure Modes and Recovery in the Agentic Layer

v0.8's failure taxonomy is retained, expanded by v0.5.2's contract-level mitigations.

7.6.1 Common agentic failures

1. **Capability overreach** (agent attempts tasks outside contract).
2. **Constraint neglect** (implicit assumption violates C).
3. **Tool failure / partial execution** (non-atomic side effects).
4. **Disagreement deadlock** (critic loops stall).
5. **Economic bleed loops** (excessive calls for low-value tasks).
6. **Context drift** (agent uses stale or disallowed memory tier).

7.6.2 Mandatory recovery behaviors

- **Immediate hard stop** on capability overreach or RB crossing attempt.
 - **Fallback routing** to alternate agent when tool failure is retryable.
 - **Arbitration ceiling** (max critic loops) to prevent deadlock.
 - **Tier degradation / plan pruning** when Economic Circuit flags bleed.
 - **Context refresh + re-plan** on drift.
-

7.7 Canonical Agent Contracts (Contract Schema v1)

Agents are admissible only through explicit, versioned contracts. A contract is both a **capability declaration** and a **governance boundary object**.

7.7.1 Purpose

Contracts enable the Engine to:

- select agents deterministically,
- enforce AB ceilings and RB gating,
- bound cost via economic profiles,
- restrict MCP access to declared resources/tools, and
- specify failure and escalation semantics.

7.7.2 Required contract fields (normative)

Every agent contract must include:

1. **Identity & Provenance**
`agent_id, cluster, version, owner, status`
2. **Capabilities**
Declarative list of tasks/tools/output forms.
3. **Eligibility & Preconditions**
AB ceiling, admissible risk classes/domains, required constraints, required memory tiers.
4. **Input/Output Schemas**
Typed REQ/RES payload definitions including confidence + explanation obligations.
5. **Economic Profile**
`cost_per_token, cost_per_call, max_cost_per_task`, preferred tier, budget behavior.

6. **Failure & Escalation Semantics**
error codes, retryability, escalation routes, rollback semantics.
7. **Governance Obligations**
logging level, retention class, constraint check mode, privacy posture.
8. **MCP Bindings**
allowed `resources[]`, `prompts[]`, `tools[]`.
9. **Arbitration Profile (optional but recommended)**
critic requirement, ensemble size, critic loop ceiling.

7.7.3 Full machine-readable schema

The formal JSON Schema for Contract v1 is maintained as **Appendix F**. It is normative for implementation and conformance testing. (Chapter text here is sufficient to understand the contract model without leaving the narrative.)

7.7.4 Conformance rule (normative)

A TG node may be dispatched to an agent only if:

- the node's $AB \leq$ contract AB ceiling,
- all required constraints are present,
- projected cost $\leq$ max_cost_per_task and within remaining B, and
- MCP calls are a subset of contract bindings.

Violation triggers preflight rejection or runtime halt with audit logging.

Chapter Close — Standard Alignment Check

- **Agents are bounded capability modules** under contracts; they do not originate meaning.
 - **MCP + contracts unify interoperability and governance** (no hidden tools/resources).
 - **Arbitration and oversight are first-class safety controls** for AB2+.
 - **Economic and autonomy ceilings are enforced at binding time** by the Engine.
 - **Failure recovery is standardized** through critic ceilings, fallback routing, and drift re-planning.
-

Below is Chapter 7.7 / Appendix F core: **Contract Schema v1 (MCP-aligned)**. This is a normative spec with a machine-readable JSON Schema, plus two worked examples and integration notes showing exactly how it plugs into the Engine Algorithm (Stages 5–9). It merges v0.8's MCP + economic constraints + sample contract object with v0.5.2's contract element set and design principles.

7.7 Agent Contract Schema v1 (MCP-Aligned)

7.7.1 Purpose

Agents in the Symbiosis Agentic Layer do not improvise; they fulfill explicit, versioned contracts. A contract is a typed, auditable interface definition specifying capabilities, preconditions, postconditions, escalation routes, failure semantics, and governance obligations.

Contracts are consumed by the Symbiosis Engine (Host) to:

1. select admissible agents for task-graph nodes,
 2. enforce Constraint Vector and Budget compliance,
 3. route MCP resources/prompts/tools safely, and
 4. manage arbitration loops (critic/voting) when outputs conflict.
-

7.7.2 Canonical Fields (v1)

A contract **must** include the following elements (minimal but explicit, typed/composable, auditable, versioned).

1. **Identity & Provenance**
 - `agent_id`, `cluster`, `version`, `owner`, `status`
2. **Capabilities**
 - Declarative list of supported task types, tools, and output forms.
3. **Eligibility & Preconditions**
 - Max autonomy band allowed, required constraints present, required context tiers.
4. **Inputs / Outputs**
 - Typed schemas for REQ and RES payloads.
5. **Economic Profile**
 - Cost per token/call, max cost per task, preferred tier, budget behaviors. (Aligns to v0.8 Economic Circuit invariant.)

6. Failure & Escalation Semantics

- How the agent signals abstain/clarify/rollback/escalate.

7. Governance Obligations

- Logging, constraint checks, retention class, privacy posture.

8. MCP Bindings

- Resources, prompts, and tools the agent exposes/consumes via MCP.

7.7.3 JSON Schema (Contract Schema v1)

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "Symbiosis Agent Contract v1",
  "type": "object",
  "required": [
    "agent_id", "cluster", "version",
    "model_tier", "capabilities",
    "eligibility", "input_schema", "output_schema",
    "economic_profile", "failure_semantics",
    "governance_obligations", "mcp_bindings"
  ],
  "properties": {
    "agent_id": { "type": "string", "description": "Unique stable identifier." },
    "cluster": {
      "type": "string",
      "enum": ["Discovery", "Modernization", "Predictive", "Operations", "Governance",
        "Oversight", "Custom"]
    },
    "version": { "type": "string", "description": "Semver or org-standard versioning." },
```

```
"owner": { "type": "string", "description": "Governance owner or team." },  
"status": {  
  "type": "string",  
  "enum": ["active", "deprecated", "experimental", "revoked"]  
},
```

```
"model_tier": {  
  "type": "string",  
  "description": "Underlying LLM tier (Reasoning/Utility/etc.)."  
},
```

```
"capabilities": {  
  "type": "array",  
  "items": { "type": "string" },  
  "description": "Declared functions/tasks the agent can perform."  
},
```

```
"eligibility": {  
  "type": "object",  
  "required": ["max_autonomy_band", "risk_classes", "required_constraints"],  
  "properties": {  
    "max_autonomy_band": {  
      "type": "integer",  
      "minimum": 0, "maximum": 3,  
      "description": "Ceiling AB this agent may operate under."  
    },
```

```
"risk_classes": {
  "type": "array",
  "items": { "type": "string" },
  "description": "Risk classes where agent is admissible."
},
"required_constraints": {
  "type": "array",
  "items": { "type": "string" },
  "description": "Constraint vector keys that must be present."
},
"required_context_tiers": {
  "type": "array",
  "items": {
    "type": "string",
    "enum": ["session", "project", "lineage", "compliance"]
  }
},
"non_delegable_domains": {
  "type": "array",
  "items": { "type": "string" },
  "description": "Domains locked to AB0 for this agent."
}
}
},

"input_schema": {
```



```
"type": "object",  
"description": "JSON Schema for REQ payload."  
},  
"output_schema": {  
  "type": "object",  
  "description": "JSON Schema for RES payload including confidence + explanations."  
},
```

```
"economic_profile": {  
  "type": "object",  
  "required": ["cost_per_token", "cost_per_call", "max_cost_per_task"],  
  "properties": {  
    "cost_per_token": { "type": "number", "minimum": 0 },  
    "cost_per_call": { "type": "number", "minimum": 0 },  
    "max_cost_per_task": { "type": "number", "minimum": 0 },  
    "preferred_tier": { "type": "string" },  
    "budget_behavior": {  
      "type": "string",  
      "enum": ["optimize", "halt_and_escalate", "degrade_tier"],  
      "description": "Agent-side behavior when nearing budget cap."  
    }  
  }  
}  
},
```

```
"failure_semantics": {  
  "type": "object",
```

```
"required": ["error_codes", "escalation_paths"],
"properties": {
  "error_codes": {
    "type": "array",
    "items": {
      "type": "object",
      "required": ["code", "meaning", "retryable"],
      "properties": {
        "code": { "type": "string" },
        "meaning": { "type": "string" },
        "retryable": { "type": "boolean" }
      }
    }
  },
  "escalation_paths": {
    "type": "array",
    "items": {
      "type": "object",
      "required": ["trigger", "route"],
      "properties": {
        "trigger": { "type": "string" },
        "route": {
          "type": "string",
          "enum": ["to_engine", "to_human", "to_critic_agent", "to_fallback_agent"]
        }
      },
      "max_retries": { "type": "integer", "minimum": 0 }
    }
  }
}
```

```
    }  
  }  
},  
"rollback_semantics": {  
  "type": "string",  
  "description": "How to revert effects if task is aborted."  
}  
}  
},  
  
"governance_obligations": {  
  "type": "object",  
  "required": ["logging_level", "retention_class", "constraint_check_mode"],  
  "properties": {  
    "logging_level": {  
      "type": "string",  
      "enum": ["minimal", "standard", "forensic"]  
    },  
    "retention_class": {  
      "type": "string",  
      "enum": ["ephemeral", "project", "canonical", "compliance"]  
    },  
    "constraint_check_mode": {  
      "type": "string",  
      "enum": ["preflight_only", "preflight_and_runtime"]  
    },  
  },  
}
```

```
"privacy_posture": {  
  "type": "string",  
  "enum": ["no_pii", "masked_pii", "pii_allowed_under_policy"]  
}  
}  
},
```

```
"mcp_bindings": {  
  "type": "object",  
  "required": ["resources", "prompts", "tools"],  
  "properties": {  
    "resources": {  
      "type": "array",  
      "items": { "type": "string" },  
      "description": "URLs this agent can expose/consume (mcp://...)."  
    },  
    "prompts": {  
      "type": "array",  
      "items": { "type": "string" },  
      "description": "Named workflows exposed over MCP."  
    },  
    "tools": {  
      "type": "array",  
      "items": { "type": "string" },  
      "description": "Executable MCP tools this agent may call."  
    }  
  }  
}
```

```

    }
  },

  "arbitration_profile": {
    "type": "object",
    "properties": {
      "critic_required": { "type": "boolean" },
      "ensemble_size": { "type": "integer", "minimum": 1 },
      "max_critic_loops": { "type": "integer", "minimum": 0 }
    }
  }
}

```

This schema directly generalizes v0.8's sample Contract Object and MCP handshake requirements, while implementing the v0.5.2 contract element list as typed fields.

7.7.4 Example Contracts

Example A — Low-Regret Adoption Bridge Agent (Band 1 ceiling)

Use case: formatting/drafting tasks where reversibility is high and budgets are small.

```

{
  "agent_id": "Drafting_Assistant_01",
  "cluster": "Modernization",
  "version": "1.0.0",
  "owner": "Symbiosis_GovTeam",
  "status": "active",

```

```
"model_tier": "Tier 3 (Utility)",

"capabilities": ["Draft_Text", "Summarize", "Reformat", "Localize"],

"eligibility": {

  "max_autonomy_band": 1,

  "risk_classes": ["low_regret", "content_only"],

  "required_constraints": ["C_safety", "C_legal", "C_econ"],

  "required_context_tiers": ["session", "project"]

},

"input_schema": {

  "type": "object",

  "required": ["intent_fragment", "style_profile"],

  "properties": {

    "intent_fragment": {"type": "string"},

    "style_profile": {"type": "string"},

    "context_pointer": {"type": "string"}

  }

},

"output_schema": {

  "type": "object",

  "required": ["artifact_md", "confidence_score", "explanation_md"],

  "properties": {

    "artifact_md": {"type": "string"},

    "confidence_score": {"type": "number", "minimum": 0, "maximum": 1},

    "explanation_md": {"type": "string"}

  }

},
```

```
"economic_profile": {
  "cost_per_token": 0.000002,
  "cost_per_call": 0.002,
  "max_cost_per_task": 0.25,
  "preferred_tier": "Tier 3 (Utility)",
  "budget_behavior": "degrade_tier"
},
"failure_semantics": {
  "error_codes": [
    {"code": "AMBIG_INTENT", "meaning": "Insufficient clarity", "retryable": false},
    {"code": "POLICY_FAIL", "meaning": "Constraint violated", "retryable": false}
  ],
  "escalation_paths": [
    {"trigger": "AMBIG_INTENT", "route": "to_engine", "max_retries": 0},
    {"trigger": "POLICY_FAIL", "route": "to_engine", "max_retries": 0}
  ],
  "rollback_semantics": "No side effects; discard artifact."
},
"governance_obligations": {
  "logging_level": "standard",
  "retention_class": "project",
  "constraint_check_mode": "preflight_only",
  "privacy_posture": "no_pii"
},
"mcp_bindings": {
  "resources": ["mcp://project/context/**"],
```

```

    "prompts": ["draft", "summarize"],
    "tools": ["spellcheck", "format_md"]
  },
  "arbitration_profile": {"critic_required": false, "ensemble_size": 1, "max_critic_loops": 0}
}

```

Why this matches the standard:

- AB ceiling ≤ 1 and content-only risk class align to Adoption Bridge conditions.
- Outputs include confidence + explanation metadata.

Example B — High-Regret Conditional Co-Agency Agent (Band 2/3 eligible)

Use case: planning + tool execution in bounded safe harbors, requiring critic loops.

```

{
  "agent_id": "Ops_Planner_02",
  "cluster": "Operations",
  "version": "1.2.0",
  "owner": "Symbiosis_GovTeam",
  "status": "active",
  "model_tier": "Tier 1 (Reasoning)",
  "capabilities": ["Plan_Workflow", "Run_Tools", "Propose_Alternatives"],
  "eligibility": {
    "max_autonomy_band": 3,
    "risk_classes": ["medium_regret", "bounded_execution"],
    "required_constraints": ["C_safety", "C_legal", "C_econ", "RB_checkpoints"],
    "required_context_tiers": ["session", "project", "lineage"]
  },
}

```



```
"input_schema": {
  "type": "object",
  "required": ["intent_fragment", "constraint_vector", "budget"],
  "properties": {
    "intent_fragment": {"type": "string"},
    "constraint_vector": {"type": "object"},
    "budget": {"type": "object"},
    "context_pointer": {"type": "string"}
  }
},
"output_schema": {
  "type": "object",
  "required": ["plan_json", "artifact_md", "confidence_score", "explanation_md"],
  "properties": {
    "plan_json": {"type": "object"},
    "artifact_md": {"type": "string"},
    "confidence_score": {"type": "number", "minimum": 0, "maximum": 1},
    "explanation_md": {"type": "string"}
  }
},
"economic_profile": {
  "cost_per_token": 0.00002,
  "cost_per_call": 0.03,
  "max_cost_per_task": 5.00,
  "preferred_tier": "Tier 1 (Reasoning)",
  "budget_behavior": "halt_and_escalate"
```

```
},
"failure_semantics": {
  "error_codes": [
    {"code": "RB_EXCEED", "meaning": "Crossing regret boundary without authorization",
"retryable": false},
    {"code": "TOOL_FAIL", "meaning": "Tool returned partial/failed response", "retryable":
true}
  ],
  "escalation_paths": [
    {"trigger": "RB_EXCEED", "route": "to_human", "max_retries": 0},
    {"trigger": "TOOL_FAIL", "route": "to_fallback_agent", "max_retries": 1}
  ],
  "rollback_semantics": "Revert tool-side effects using logged transaction IDs."
},
"governance_obligations": {
  "logging_level": "forensic",
  "retention_class": "canonical",
  "constraint_check_mode": "preflight_and_runtime",
  "privacy_posture": "masked_pii"
},
"mcp_bindings": {
  "resources": ["mcp://ops/logs/*", "mcp://project/context/*"],
  "prompts": ["plan_ops", "risk_check"],
  "tools": ["fetch_logs", "simulate_change", "apply_change_guarded"]
},
"arbitration_profile": {"critic_required": true, "ensemble_size": 2, "max_critic_loops": 3}
}
```

Why this matches the standard:

- Explicit RB gating and AB3 eligibility require high SI/HSS and strict C_econ per v0.8 bands.
 - Critic loop max-3 aligns to arbitration protocol.
-

7.7.5 Engine Integration Notes (how contracts are used)

Where it plugs in:

- **Stage 5 (Economic Pass):** Engine reads `economic_profile` to estimate cost envelopes before dispatch.
- **Stage 6 (AB selection):** Engine ensures node AB $\leq$ `eligibility.max_autonomy_band`.
- **Stage 8 (Binding & MCP routing):** Engine binds `mcp_bindings` and rejects any tool/resource not listed.
- **Stage 9 (Execution):** Agent sends REQ/RES/STAT/CTRL payloads conforming to schemas and handshake logic.

Mandatory handshake:

Before execution, Engine sends REQ + Constraint Vector; agent simulates satisfiability and returns ACK or REJECT. Budget is unlocked only after ACK.

Arbitration hook:

If `critic_required=true`, Engine routes outputs through Oversight cluster agents and applies critic-refine or voting ensemble as specified.

7.7.6 Design Principles (normative)

Contracts must remain:

- **Minimal but explicit** (no hidden behaviors).
- **Typed & composable** (swappable agents with same contract).
- **Auditable & versioned** (deviations detectable; contracts governed).

These principles directly mitigate agentic failure modes: constraint neglect, capability over-extension, tool failure, and unresolved disagreement.

What you have now

With Appendix A (Ontology v1.1), Chapter 6.8 (Engine Standard Algorithm), and Contract Schema v1, the framework has a deployable, defensible core:

- **fixed vocabulary,**
 - **a canonical control algorithm,**
 - **machine-readable admissibility for agents, tools, and autonomy.**
-

Below is **Chapter 8 (merged v0.9 draft)**. v0.8 supplies the skeleton governance posture (Governance Kernel, policy-as-code, economic enforcement, Adoption Bridges as fast lane). v0.5.2 supplies the flesh (decision-rights matrices, oversight institutions, escalation logic, governance failure modes). keep it linear, and embed already-written **Ch8.8 Adoption Bridge Safety Proof** in-place, so the chapter reads continuously without appendix-hopping.

Chapter 8 — Governance Layer & Adoption Bridges (Merged Draft)

Governance is not a policy PDF stapled onto a multi-agent system after the fact. In Symbiosis, governance is **architectural and executable**: it produces Constraint Vectors, enforces Regret Boundaries, governs Autonomy Bands, validates Economic Budgets, and maintains immutable audit trails.

This chapter specifies:

1. the Governance Layer's scope and objects,
 2. the Governance Kernel (GK) and policy versioning,
 3. decision rights and escalation,
 4. auditability and evidence,
 5. governance failure modes and mitigations, and
 6. Adoption Bridges ("fast lane") as a safe acceleration pathway.
-

8.1 Why Governance Must Be a First-Class Layer

High-capability agentic systems fail *predictably* when governance is informal:

- autonomy creep into high-regret domains,
- black-box decisions with no replayable lineage,
- economic bleed from expensive models,
- unclear human accountability after incidents,
- and reliance on exhausted HITL gates ("oversight theater").

Therefore governance must be:

- **ex ante** (pre-flight enforcement),
 - **runtime-active** (RB/AB gating), and
 - **post hoc auditable** (lineage).
-

8.2 Core Governance Objects

Governance operates over a small number of stable objects (all defined in Ontology v1.1):

1. **Governance Kernel (GK)** — isolated rule core inside E enforcing AB/RB and policy invariants.
 2. **Policy Version (P_v)** — compiled ruleset used to generate Constraint Vectors.
 3. **Constraint Vectors (C)** — C_legal, C_safety, C_econ.
 4. **Risk Classes (R)** — task typology relative to RB: low-regret, medium-regret, high-regret, non-delegable.
 5. **Autonomy Bands (AB) + Regret Boundary (RB)** — state machine governed by GK.
 6. **Lineage (L)** — immutable evidence graph.
 7. **Economic Circuit / Budgets (B)** — enforce pre-flight cost admissibility.
-

8.3 The Governance Kernel (GK)

8.3.1 Definition

GK is the minimal isolated rule engine embedded within E. It is the authoritative interpreter of policy and the enforcer of AB/RB transitions. Agents may not modify GK and cannot perform actions beyond its signed authorization envelope.

8.3.2 Kernel responsibilities (normative)

GK must:

1. compile P_v into Constraint Vectors,
2. define non-delegable domains and AB lock rules,
3. gate AB promotions/demotions,
4. enforce RB checkpoint requirements,
5. maintain Bridge admissibility rules, and
6. trigger halts on policy drift mid-run.

8.3.3 Kernel independence invariant

GK must be:

- isolated from agent write-paths,
- version-locked per workflow (P_v pinned),
- auditable by lineage events.

This prevents “governance capture” where agent outputs implicitly reframe boundaries.

8.4 Policy-as-Code and Constraint Compilation

8.4.1 Policy compilation

P_v transforms institutional rules into executable constraints. The compilation process yields:

- **C_legal**: jurisdictional policy, contracts, IP, privacy, retention.
- **C_safety**: harm thresholds, capability ceilings, domain exclusions.
- **C_econ**: spend caps, tier caps, interrupts caps.

8.4.2 Constraint supremacy (re-stated)

- Any plan that violates C is rejected pre-flight.
 - Any drift that causes later violation forces a pause and re-validation.
-

8.5 Decision Rights and Escalation

Governance is not centralized ownership of every decision; it is **centralized ownership of boundaries**.

8.5.1 Decision rights matrix (minimum model)

- **Operators (H-Layer)**: own intent, approve RB crossings, override outputs.
- **Engine (E)**: owns admissibility, routing, orchestration, economic gating.
- **Agents (A)**: own bounded execution under contract.
- **Governance Leads / Bodies**: own P_v policy, AB thresholds, domain locks, escalation resolution.

8.5.2 Escalation ladder (normative)

If a task fails admissibility or conflicts arise:

1. **Engine-level resolution** (optimize plan, demote AB, switch agents).
2. **Critic / Oversight arbitration** (if $AB \geq 2$ or critic_required).
3. **Human operator checkpoint** (for RB crossings or low RCI).
4. **Governance Lead escalation** (policy ambiguity, domain risk).

Escalation routes are declared in agent contracts and must be executed deterministically by E.

8.6 Auditability, Evidence, and Lineage

8.6.1 Lineage as evidence substrate

Lineage L is the sole authoritative record of:

- intent versions,
- constraints and policy versions,
- AB/RB transitions,
- economic projections/realizations,
- agent decisions and tools called,
- critic deltas and human overrides.

8.6.2 Minimum audit package (normative)

For any high-regret workflow, auditors must be able to retrieve:

1. I and all refinements,
2. C and P_v used,
3. TG and AB schedule,
4. cost trace vs B,
5. complete execution and arbitration trace,
6. final human approvals,
7. SI/HSS/SCP deltas.

Systems unable to produce this package are non-compliant by definition.

8.7 Governance Failure Modes and Mitigation

v0.5.2 adds a realistic failure model on the governance side.

8.7.1 Common governance failures

1. **Policy lag:** P_v doesn't reflect real-time regulatory changes.
2. **Boundary ambiguity:** unclear non-delegable domains → silent AB drift.
3. **Excessive friction:** governance overhead too heavy → users revert to Shadow AI.
4. **Unpriced autonomy:** budgets missing or loose → economic bleed.
5. **Evidence gaps:** lineage incomplete → no defensible audit.

8.7.2 Mitigations

- **Policy pinning + drift alerts** (halt or revalidate on P_v change).
 - **Explicit blocklists/allowlists** for domains and Bridges.
 - **Adoption Bridges** to lower friction where safe.
 - **Mandatory B and Economic Pass** as a pre-flight invariant.
 - **Lineage minimum event set** enforced by GK.
-

8.8 Adoption Bridges (“Fast Lane”) — Safety Proof Sketch and Control Rules

(Inserted as the canonical fast-lane governance standard; normative.)

8.8.1 Purpose

Adoption Bridges are an architectural fast lane allowing **low-regret, low-cost tasks** to bypass the heavy planning and governance cycles. Their purpose is to minimize user friction and prevent regression to Shadow AI.

However, because Bridges deliberately reduce oversight overhead, they require formal admissibility conditions and automatic revocation. This section defines those conditions.

8.8.2 Definitions

- **Adoption Bridge (ABridge)**: A GK-governed bypass path for tasks that fall strictly **below** the Regret Boundary and within a Safety Envelope.
- **Regret Boundary (RB)**: Threshold where actions become irreversible, high-harm, or legally risky. Tasks above RB are not Bridge-admissible.
- **Safety Envelope (SE)**: Operational boundary defining where autonomy is safe given domain, constraints, and failure modes; Bridges must remain inside SE.
- **Budget Envelope (B)**: Cost/compute cap validated by the Economic Circuit before any autonomy executes.

8.8.3 Bridge Admissibility Conditions (Necessary and Jointly Sufficient)

A task graph node t is eligible for Adoption Bridge execution **iff** all hold:

- (C1) Low-Regret Classification
- (C2) Reversibility and Containment
- (C3) Constraint Completeness
- (C4) Budget Admissibility
- (C5) Autonomy Band Ceiling ($AB \leq 1$)
- (C6) Metric Health Thresholds (SI/HSS/SCP minima)

8.8.4 Proof Sketch

If (C1–C6) hold, Bridge execution cannot cross RB, violate constraints, or exceed budget; thus it does not increase risk relative to full-cycle execution and reduces Shadow AI relapse pressure.

8.8.5 Admission Algorithm

Risk Gate → Reversibility Gate → Constraint Gate → Economic Gate → Autonomy Gate → Metric Gate → admit or full-cycle route.

8.8.6 Runtime Monitoring & Auto-Revocation

Revocation triggers: emergent RB risk, budget drift, SI drop, SCP saturation, policy update.

8.8.7 Audit Requirements

All admissions/revocations logged in lineage with projected vs realized cost and AB ceilings.

8.8.8 Conformance

Bridges are compliant only if they enforce (C1–C6), hard-revoke on triggers, constrain AB to 0–1, and log lineage.

8.9 Default Bridge Policy Parameters (v0.9)

Governance Kernel maintains a versioned Bridge policy file containing conservative defaults, including:

- `ABridge_AB_Ceiling = 1`
- `SI_bridge_min = 0.72`
- `HSS_bridge_min = 0.55`
- `SCP_bridge_min = 0.35 headroom`
- `B_fastlane_fraction = 0.05` (5% of intent budget)
- `δ_overrun_multiplier = 1.10`
- Allowlist of low-regret risk classes; blocklist of non-delegable domains.

Sector overlays may tighten these values, not loosen them, without explicit governance approval.

Chapter Close — Standard Alignment Check

- **Governance is executable:** $P_v \rightarrow C$ vectors enforced by GK pre-flight and at RB checkpoints.

- **AB/RB are kernel-controlled state machines**, preventing autonomy creep.
- **Economic viability is mandatory** via B and the Economic Circuit.
- **Auditability is guaranteed** through lineage minimum events.
- **Adoption Bridges lower friction safely** and reduce Shadow AI relapse.

Below is **Chapter 8.8 — Adoption Bridge Safety Proof Sketch (v0.9)**. This is written as a normative, “engineering standard” section. It formalizes the Fast Lane pattern introduced in v0.8 and ties it to the Regret Boundary, Safety Envelope, Constraint Vector supremacy, Economic Circuit budget gating, and the Shadow AI risk you’re explicitly mitigating.

8.8 Adoption Bridges (“Fast Lane”) — Safety Proof Sketch and Control Rules

8.8.1 Purpose

Adoption Bridges are an architectural fast lane allowing **low-regret, low-cost tasks** to bypass the heavy planning and governance cycles. Their purpose is to minimize user friction and prevent regression to Shadow AI.

However, because Bridges deliberately reduce oversight overhead, they require formal admissibility conditions and automatic revocation. This section defines those conditions.

8.8.2 Definitions

- **Adoption Bridge (ABridge)**: A GK-governed bypass path for tasks that fall strictly **below** the Regret Boundary and within a Safety Envelope.
 - **Regret Boundary (RB)**: Threshold where actions become irreversible, high-harm, or legally risky. Tasks above RB are not Bridge-admissible.
 - **Safety Envelope (SE)**: Operational boundary defining where autonomy is safe given domain, constraints, and failure modes; Bridges must remain inside SE.
 - **Budget Envelope (B)**: Cost/compute cap validated by the Economic Circuit before any autonomy executes.
-

8.8.3 Bridge Admissibility Conditions (Necessary and Jointly Sufficient)

A task graph node t is eligible for Adoption Bridge execution **iff** all of the following hold:

(C1) Low-Regret Classification

The Router classifies t into a risk class strictly below RB (e.g., “drafting,” “formatting,” “summarization,” “ideation”), and t contains no irreversible edges.

(C2) Reversibility and Containment

t is **fully reversible** or discardable without side effects. If any downstream irreversibility is possible, t is disqualified.

(C3) Constraint Completeness

Constraint Vector C is present and satisfiable, including:

- C_{legal} and C_{safety} (hard boundaries), and
- C_{econ} (budget cap).

(C4) Budget Admissibility (Economic Circuit Gate)

$\text{Projected_Cost}(t) \leq B_{\text{fastlane}}$, where B_{fastlane} is a *Bridge-tier* sub-budget defined by GK (typically a strict fraction of the intent-level B).

(C5) Autonomy Band Ceiling

Bridge tasks may operate only at AB0–AB1. Any requirement for AB2–AB3 is disqualifying.

(C6) Metric Health Thresholds

At admission time:

- $SI \geq SI_{\text{bridge_min}}$
- $HSS \geq HSS_{\text{bridge_min}}$
- $SCP \text{ headroom} \geq SCP_{\text{bridge_min}}$

Thresholds are GK policy parameters and are conservative by default to avoid Oversight Theater and saturation.

If **any** condition fails, the Engine must route through the standard full-cycle algorithm (Ch6.8).

8.8.4 Proof Sketch (Why These Conditions Are Safe)

Claim: If (C1–C6) hold, Bridge execution cannot cause irreversible harm or uncontrolled economic bleed, and will not increase system risk relative to standard execution.

Sketch:

1. No RB Crossing:

By (C1) and (C2), Bridge tasks lie strictly beneath RB and are reversible. Therefore, any failure is recoverable by discarding or rolling back artifacts, with no path to high-regret states.

2. Hard Boundary Preservation:

By (C3), C_{legal} and C_{safety} are present, and Constraint Supremacy ensures pre-flight rejection of any violating plan even on the fast lane. Thus, Bridges do not

weaken safety/legal guarantees.

3. **Economic Non-Explosion:**

By (C4) and the Economic Circuit invariant, Bridge tasks are cost-bounded before execution. The bypass cannot trigger unbounded loops or high-tier misuse.

4. **Autonomy Ceiling:**

By (C5), Bridges never exceed AB1. Therefore, no tool-side irreversible action may execute without leaving the Bridge path and re-entering GK/RB gating.

5. **Human-System Stability:**

By (C6), Bridges are admitted only when system health and operator capacity are sufficient, reducing the likelihood of Oversight Theater and silent autonomy creep.

Together, these properties imply Bridges preserve the same safety/economic invariants as standard execution for tasks in this class, while lowering friction to prevent Shadow AI relapse.

8.8.5 Bridge Admission Algorithm (Normative)

Given node t :

1. **Risk Gate:** verify (C1).
2. **Reversibility Gate:** verify (C2).
3. **Constraint Gate:** verify (C3).
4. **Economic Gate:** compute $\text{Projected_Cost}(t)$; verify (C4).
5. **Autonomy Gate:** verify $\text{AB}(t) \leq 1$; verify (C5).
6. **Metric Gate:** verify (C6).
7. If all gates pass: mark t as **Bridge-Admitted** and skip Stages 4–7 of Ch6.8; proceed directly to binding/execution with $\text{AB} \leq 1$ contracts.
8. Else: route to Ch6.8 full-cycle.

8.8.6 Runtime Monitoring and Auto-Revocation

Even Bridge-admitted tasks are continuously monitored. The Engine must revoke Bridge status when any of the following triggers occur:

(R1) Emergent RB Risk

If task scope drifts and introduces an irreversible edge or higher regret class, revoke immediately and re-enter Ch6.8 at Stage 3.

(R2) Budget Drift

If realized cost exceeds $B_{\text{fastlane}} \times \delta$ (δ typically 1.1), halt execution, demote AB to 0, and require re-planning.

(R3) SI Degradation

If SI falls below SI_bridge_min during Bridge execution, revoke Bridge access for the remaining nodes of the intent and log a governance event.

(R4) HSS/SCP Saturation Signals

If SCP headroom drops below SCP_bridge_min or Oversight Theater cues rise (e.g., abnormally high blind-approval rates), throttle or revoke Bridge admission for that operator until metrics recover.

(R5) Governance Policy Update

If GK/Pv changes mid-run (new legal/safety rule), revoke Bridge, refresh C, and re-validate from Stage 3.

8.8.7 Audit and Evidence Requirements

For every Bridge-admitted node, the Engine must record into Lineage L:

- admission decision + passed gates,
- projected vs realized cost,
- AB ceiling + contract ID,
- any revocation triggers and responses.

This ensures external auditors can ask, “show me where Bridges were used and why they were safe,” and receive verifiable evidence.

8.8.8 Conformance Summary

A system is Bridge-compliant only if:

1. it enforces (C1–C6) **before** Bridge admission,
 2. it hard-revokes on (R1–R5),
 3. it constrains Bridge autonomy to AB0–AB1, and
 4. it logs all admissions/revocations into immutable lineage.
-
-

1. Adoption Bridge Policy Parameter Table (defaults v0.9)

These parameters live in the Governance Kernel policy file **P_v** and must be versioned. They're conservative defaults; sector playbooks may tighten them, not loosen them, unless explicitly approved by governance.

1.1 Global Bridge Parameters

Parameter	Default	Meaning	Notes
ABridge_Enabled	true	Master switch for Bridges	Can be disabled per domain/org.
ABridge_AB_Ceilings	1	Max AB allowed on Bridge	Enforces AB0–AB1 only.
SI_bridge_min	0.72	Minimum SI for Bridge admission	Tune by org maturity.
HSS_bridge_min	0.55	Minimum HSS for operator	Ensures intent quality adequate.
SCP_bridge_min	0.35 headroom	Minimum spare capacity	Prevents Oversight Theater / saturation.
B_fastlane_fraction	0.05 (5%)	Share of intent budget usable by Bridges	Bridges stay cheap by design.
δ_overrun_multiplier	1.10	Revocation threshold for realized cost	Halt if realized > 1.1× projected.
Bridge_RiskClasses-Allowlist	{"drafting","summarization","formatting","ideation","translation"}	Only these risk classes may Bridge	Must be below RB.

Bridge_Domains_Blocklist	{"medical_dx","legal_advice","financial_trade","HR_discipline","security_controls"}	Always full-cycle, never Bridge	Non-delegable domains.
Bridge_C_Check_Mode	preflight_only	Constraint checks for Bridges	Runtime checks reserved for AB2+.
Bridge_Logging_Level	standard	Lineage logging mode	Still must log gates + costs.

1.2 Optional Organization Overrides

These are safe override levers, intended for tuning as adoption matures.

Lever	Safe Direction	Rationale
Raise SI_bridge_min	Up only	Bridges become safer as quality rises; lowering increases risk.
Lower B_fastlane_fraction	Down only	Keeps Bridges cheaper and reduces Economic Bleed exposure.
Expand Blocklist	Expand only	Safer to force full governance in more domains.
Narrow Allowlist	Narrow only	Safer to permit fewer tasks in fast lane.

2. Worked Example: Bridge Flow vs Full-Cycle Flow

Scenario

Operator wants:

“Turn these 12 bullet notes into a 1-page executive summary, polished for a board update.”

Context:

- Notes are internal, no PII.
- Output is reversible (text only).
- Budget for intent: **B = \$20 / 1M tokens / 2h TTL.**
- Current metrics: **SI=0.81, HSS=0.63, SCP headroom=0.52.**

This is a classic low-regret content transformation task.

A) Adoption Bridge Path (Fast Lane)

Step A1 — Router Classification (Stage 2 gate)

- Risk class: “summarization/drafting.”
- RB proximity: below RB; no irreversible edges.
- Candidate AB: AB0–AB1.

Step A2 — Bridge Gates (Ch8.8 C1–C6)

- (C1) Low-regret allowlisted class → pass.
- (C2) Fully reversible text artifact → pass.
- (C3) C_legal/C_safety/C_econ present → pass.
- (C4) Economic Circuit projects cost \$0.12;
B_fastlane = 5% × \$20 = \$1.00 → pass.
- (C5) AB ceiling=1 → pass.
- (C6) SI/HSS/SCP over mins → pass.

Step A3 — Skip heavy stages

Engine **skips** full decomposition + deep memory governance; it goes straight to binding/execution with AB1-ceiling agent.

Step A4 — Contract Binding (Stage 8)

Binds a Modernization drafting agent with:

- max_autonomy_band=1
- max_cost_per_task=\$0.25
- tools: format_md, spellcheck.

Step A5 — Execution (Stage 9)

Agent produces executive summary + confidence + rationale.

Lineage logs: gate results, projected vs realized cost (say \$0.10), contract ID.

Totaltime: seconds to a minute.

Governance cost: minimal.

Safety posture: preserved because gates ensure no RB crossing, no constraint violation, no budget bleed.

B) Full-Cycle Symbiosis Engine Path (Standard)

Assume **one change**: the operator adds,

“Include a recommendation for restructuring our Q2 staffing plan.”

Now the task may approach **HR discipline / financial planning** domain, potentially higher regret.

Step B1 — Intent Intake (Stage 1)

Structured intent object created; lineage intake event logged.

Step B2 — Classification (Stage 2)

- Risk class: “decision + org-impact.”
- RB proximity: near/above RB because decisions affect people/costs.
- Candidate AB: AB0–AB2.

Bridge admissibility fails:

- (C1) Not allowlisted; also in blocklisted domain (“HR_discipline”).
→ Must route to full cycle.

Step B3 — Constraint Consolidation (Stage 3)

GK injects:

- C_legal: HR policy, labor compliance boundaries.
- C_safety: no employment decisions without human approval.
- C_econ: budget cap still applies.

Step B4 — Plan / Decomposition (Stage 4)

Engine decomposes:

1. summarize notes
2. surface staffing assumptions
3. draft recommendation options
4. require human checkpoint before any prescriptive output.

Step B5 — Economic Pass (Stage 5)

Projected cost \$1.30 (uses higher tier for reasoning).

Still under B=\$20 → pass.

Step B6 — RB + AB Selection (Stage 6)

- Summarization node: AB1 ok.
- Recommendation node: AB0 only + mandatory human checkpoint at RB crossing.

Step B7 — Context & Memory Governance (Stage 7)

Pulls lineage + policy tiers relevant to HR. Tight retention class.

Step B8–B9 — Binding & Execution

- Drafting agent handles summary AB1
- Oversight + critic agents review staffing recs AB0
- Human approves/edits final prescriptive text.

Step B10 — Post-execution updates

SI/HSS/SCP updated; lineage includes HR-sensitive rationale trail.

Totaltime: minutes to hours, depending on checkpointing.

Governance cost: higher, appropriately.

Safety posture: high-regret domain handled with human primacy and RB gating.

Takeaways from the example

1. **Bridges are a controlled convenience, not a loophole.**
They preserve the same invariants as full cycle for truly low-regret tasks.
2. **The moment regret class or domain shifts, the system self-tightens.**
That is key to defensibility: fast lane is revocable, not permissive by default.
3. **Economics is your silent safety rail.**
Even low-risk tasks can't free-ride expensive compute.

Below is a fully drafted **Chapter 9** that treats v0.8 as the skeleton and pulls mathematical/operational flesh from v0.5.2. also embedding “further muscle” where the combined text can be made more defensible as an engineering standard. After the chapter, give a tight improvement backlog for this area.

Chapter 9 — The Symbiosis Index (SI)

Purpose.

The Symbiosis Index (SI) is the primary control signal for the Symbiosis Engine. In most AI deployments, “success” is measured via vague satisfaction or narrow model benchmarks. Here, SI quantifies the *health of the human–AI relationship* over time and is used to *alter system behavior in-line* (e.g., autonomy promotion/demotion, circuit breakers).

SI does not measure raw model intelligence; it measures whether the total hybrid stack (Human Layer + Engine + Agentic Layer + Governance) is converging toward **safe, high-quality, economically viable co-agency**. This makes SI analogous to a reliability/compliance KPI in SRE: it is descriptive *and* prescriptive.

9.1 Measuring System Health (Dimensions)

v0.8 defines SI as a composite of three orthogonal macro-dimensions: **Alignment (A)**, **Quality (Q)**, and **Economic Efficiency (E)**.

v0.5.2 provides a richer multi-dimensional decomposition (Q, A, T, O, E, S, D) with update and failure logic.

Best synthesis: make SI **two-level**:

Level 0 (macro):

1. **Alignment (A^M)**: Adherence to intent + constraint vectors.
2. **Quality (Q^M)**: Utility/correctness relative to goal.
3. **Economic Efficiency (E^M)**: Value per resource unit.

Level 1 (subdimensions feeding macro):

- **Q (Quality submetric)**: correctness, relevance, coherence.
- **A (Alignment submetric)**: rule adherence, ethical consistency, goal fit.
- **T (Trust Stability)**: volatility of relationship; low variance is good.
- **O (Override Frequency)**: human overrides; lower implies better system fit.
- **E (Error Rate)**: explicit failures/hallucinations; lower is better (hence 1–E).
- **S (Satisfaction/Adoption)**: whether humans willingly use and rely on outputs.
- **D (Efficiency/Delivery)**: time-to-completion, rework load, throughput.

Then define macro-dimensions as weighted aggregates of these submetrics. Example mapping:

- $A^M = f(A, O, \text{portions of } T)$
- $Q^M = f(Q, E, \text{portions of } T)$
- $E^M = f(D, \text{budget variance, model-tier appropriateness})$

This keeps the elegant v0.8 view while preserving the granularity needed for diagnostics and defensibility.

9.2 Calculating SI: Formulae and Weighting

9.2.1 Composite SI (episode level)

From v0.5.2, SI is a weighted sum of normalized subdimensions:

$$SI_t = w_Q Q_t + w_A A_t + w_T T_t + w_O (1-O_t) + w_E (1-E_t) + w_S S_t + w_D D_t$$

Subject to:

$$w_Q + w_A + w_T + w_O + w_E + w_S + w_D = 1, \quad w_i \geq 0$$

Interpretation notes (add explicitly in v0.8):

- Every subdimension is normalized to $[0, 1]$.
- For “bad” dimensions (override frequency, error rate), we use complements $(1-O, 1-E)$.
- Weights are **policy**, not engineering constants.

9.2.2 Risk-class weight profiles (Governance-controlled)

v0.8 already sets the precedent: weights change by risk class (medical vs low-stakes).
Make this normative:

- **Class A / High Regret:** prioritize Alignment
 - w_A high, w_Q moderate, w_E low/zero.
- **Class B / Standard:** balanced.
- **Class C / Bulk/Low Regret:** prioritize Economic Efficiency + acceptable quality.

v0.8 Appendix B already gives example class profiles; incorporate them directly into the chapter as a table with a clear “Governance owns weights” rule.

9.2.3 Rolling SI (longitudinal control signal)

Single episodes are noisy. Use an Exponential Moving Average (EMA):

$$SI_{\text{rolling},t+1} = \alpha, SI_{\text{rolling},t} + (1-\alpha), SI_t$$

Where α (smoothing) is selected by governance (higher α = stability, lower α = responsiveness).

Normative rule: Autonomy transitions must trigger on **SI_rolling**, not SI_episode.

9.3 SI as a Control Loop (Autonomy + Circuit Breakers)

SI is *instrumental*. It directly controls autonomy bands and recovery logic.

9.3.1 Promotion / demotion thresholds

v0.8 Appendix B provides explicit downgrade/promotion logic.

Bring that pseudo-code into Chapter 9 and annotate with policy ownership:

- **Demote if SI_rolling < THRESHOLD_DEMOTE** (immediate safety response).
- **Eligible-for-promotion if SI_rolling > THRESHOLD_PROMOTE for N episodes, but never auto-promote** without human signoff.

This is already aligned with the Human Primacy invariant and Autonomy FSM in Ch.5/App.C.

9.3.2 Trust derivative (drift early warning)

Add the v0.8 notion of “trust derivative” as mandatory:

$$\Delta SI_t = SI_{\{rolling,t\}} - SI_{\{rolling,t-1\}}$$

- : stable system
 - : learning/convergence
 - : drift or collapse → triggers architecture audit / governance review
-

9.4 Instrumentation Requirements (What must be logged)

Make SI defensible by specifying what evidence it consumes. v0.5.2's pilot template points to explicit instrumentation planning.

Minimum required telemetry per episode:

1. **Intent fidelity signals** (alignment): constraint violations, policy-filter trips, instruction-following score.
2. **Output utility signals** (quality): acceptance rate, edit distance, task success markers.

3. **Economic signals** (efficiency): estimated vs actual cost, token counts by model tier, TTL/budget circuit breaker events.
 4. **Human interaction signals**: override events with reason codes, latency-to-override, escalation counts.
 5. **Context Lineage pointer**: every SI datapoint links to the lineage graph node so audits can replay decisions.
-

9.5 Failure Detection Heuristics (Normative)

You already have the raw ingredients across v0.5.2 and v0.8; consolidate into a simple SI-driven failure taxonomy:

SI-driven alerts:

- **Alignment crash**: A drops abruptly while Q stable → likely policy drift or silent constraint drop.
- **Quality crash**: Q or 1-E drops while A stable → model/tool regression.
- **Efficiency crash**: E^M drops while A/Q stable → economic bleed or bad routing.
- **Volatility spike**: T falls → trust instability or inconsistent agent behavior.
- **Override surge**: O rises → intent interface mismatch or operator overload.

Hard actions:

- If SI_rolling below demotion threshold → auto-downgrade autonomy band.
 - If ΔSI negative for sustained epochs → architecture audit.
-

9.6 Relationship to Human Metrics (HSS & SCP)

Governance must evaluate the **triple** (SI, HSS, SCP) when assigning autonomy and roles. Make this a first-class rule in Chapter 9:

- **Low SI + High HSS**: system/agent problem; refactor workflows.
- **High SI + Low HSS**: operator training gap; don't promote autonomy until skill catches up.
- **High SI + SCP overload**: throttle/batch workflows; prevent oversight theater.

This closes the loop between system health and human development.

Targeted improvements / “muscle to add”

Here is the backlog that will make SI more rigorous, publishable, and hard to attack:

1. **Explicit normalization & calibration protocol.**
Add a short “SI Calibration Procedure” section: how each subdimension normalizes, baseline collection window, and recalibration after model swaps. Right now normalization is implied.
2. **Hierarchical SI diagram.**
One graphic showing Macro ($AM/QM/EM$) fed by submetrics (Q,A,T,O,E,S,D) will instantly clarify the skeleton/flesh merge.
3. **Multi-agent aggregation rule.**
Specify how SI is computed for task graphs with multiple agents: e.g., weighted by token share, risk class of each step, or regret boundary proximity. Today it reads “per episode” but not “per graph.”
4. **Gaming resistance.**
Add a paragraph: SI inputs (especially O and S) can be gamed; mitigation includes random audits, cross-check with edit distance, and Non-Punitive Covenant insulation on the human side.
5. **Confidence intervals + minimum sample size.**
Promote/demote logic references a confidence interval (episodes). Make the statistical rationale explicit and normative.
6. **Risk-class governance template.**
Put a small, repeatable template: “to define a new risk class, specify weights, thresholds, and allowable variance.” This supports scale across sectors.
7. **Operational dashboards spec.**
The chapter should include a minimal dashboard layout: SI_rolling trend, ΔSI , macro-dimensions, top three failing submetrics, and “next action recommended.” This makes SI concrete for adopters.
8. **Link to stability metrics (Ch.11).**
Add a cross-reference line: “SI collapse is the primary trigger; Drift/Divergence/Resonance/Entanglement explain why.” You already have the dynamics defined.

Below is **Chapter 10 (merged v0.9 draft)**. v0.8 provides the skeleton (HSS as “pilot rating,” SCP as attention/load-bearing envelope, tier gating, behavioral signals). v0.5.2 provides the detailed multi-factor rubrics, coupling logic with SI, and ethical safeguards. Unified terms to Ontology v1.1 and added a small amount of “muscle” to make this section more implementable and defensible as a standard.

Chapter 10 — Human Capability Metrics: HSS & SCP (Merged Draft)

Symbiosis depends on human capability as much as model capability. If humans are overloaded, poorly calibrated, or operating outside their cognitive envelope, the system will regress into Oversight Theater or Shadow AI. Therefore, the framework defines two first-class human-side control metrics:

- **HSS (Human Symbiosis Score):** skill/competence in hybrid operation (the “pilot rating”).
- **SCP (Symbiotic Capacity Profile):** load-bearing capacity for complexity and concurrency (the “airframe envelope”).

These metrics are **not HR performance tools**. They are safety and effectiveness telemetry used by the Symbiosis Engine and Governance Kernel to allocate tasks, gate autonomy, and drive training.

10.1 HSS vs. SI: Distinct but Coupled

SI (Chapter 9) measures the health of the *system relationship and outcomes* for a workflow. **HSS** measures the skill of a *given human operator* at hybrid work.

They are coupled but not interchangeable:

- **High SI + low HSS** means the system is carrying the operator (risk of fragile adoption).
- **Low SI + high HSS** means the operator is capable but the system/agents are misaligned (system refactor needed).
- **Low SI + low HSS** means both human training and system design must improve together.

Normative rule: Autonomy promotions/demotions never trigger on SI alone; they require the triple check (**SI, HSS, SCP**).

10.2 Human Symbiosis Score (HSS)

10.2.1 Definition

HSS quantifies a human’s ability to work symbiotically with agents and the Engine: to express intent clearly, calibrate trust, supervise autonomy, and improve through co-adaptive

loops. HSS is derived from **behavioral signals** and structured assessments, not subjective manager ratings.

10.2.2 HSS dimensions (rubric)

HSS is a weighted composite of seven stable dimensions. Each dimension is normalized to [0,1] and scored through evidence in lineage plus periodic structured calibration.

1. **Intent Clarity & Structuring (C_i)**
Measures how well the operator forms high-fidelity Intent Objects: explicit goals, constraints, regret tolerance, budgets, and output specs.
Signals: fewer clarification loops, fewer misplanned TGs, higher first-pass success.
2. **Horizon Management & Complexity Framing (H_i)**
Measures ability to scope tasks across time horizons and complexity levels without over- or under-specifying.
Signals: stable TG decomposition, appropriate task granularity, fewer re-plans.
3. **Feedback & Override Quality (F_i)**
Measures whether corrections are specific, constructive, and improve downstream plans.
Signals: high-signal overrides, low rework, critic deltas shrinking over time.
4. **Cognitive Load Management & Delegation (L_i)**
Measures ability to distribute tasks to agents while respecting SCP headroom, avoiding overload and Oversight Theater.
Signals: appropriate batching, low interrupt thrash, stable concurrency.
5. **Memory Externalization & Artifact Hygiene (M_i)**
Measures competence at turning ephemeral reasoning into durable, structured artifacts (notes, specs, canonical docs) that reduce future cognitive load.
Signals: high reuse of project memory, low context amnesia, clean lineage.
6. **Alignment & Governance Literacy (G_i)**
Measures understanding and correct use of constraints, budgets, AB/RB gating, and escalation.
Signals: low policy-violation attempts, correct refusal/esc escalation behavior.
7. **Reflective Practice & Learning Loops (R_i)**
Measures willingness and ability to improve intent quality and trust calibration over time.
Signals: SI trend improvement after retros, reduced volatility (Trust Stability rises).

10.2.3 Composite HSS

HSS is computed as:

$$HSS_t = \sum_{k=1}^7 w_k \cdot D_{k,t}, \quad \sum w_k = 1$$

where are the seven dimension scores above and are policy weights owned by governance. Weight profiles may vary by sector and role (e.g., safety-critical domains overweight G_i and C_i).

10.2.4 What improves HSS

HSS rises through:

- explicit intent-structuring practice,
- exposure to calibrated autonomy,
- disciplined feedback,
- governance literacy training,
- post-mortems that update operator mental models, and
- durable artifact habits.

HSS falls through:

- chronic overload (SCP violations),
- bypassing budgets/constraints,
- habitual rubber-stamping,
- low-fidelity prompting in high-regret work.

10.3 Operator Tiers (HSS-Gated Permissions)

To make HSS operational, the framework defines **tiers** that gate allowable autonomy bands. The thresholds here are conservative defaults; governance owns exact values.

- **L1 — Cadet (HSS < 0.5 default)**
Allowed AB: **AB0 only**.
Pattern: propose → human executes or explicitly approves.
- **L2 — Pilot (0.5 ≤ HSS ≤ 0.8 default)**
Allowed AB: **AB1–AB2** in admissible domains under budget.
Pattern: co-planning + conditional autonomy.
- **L3 — Commander (HSS > 0.8 default)**
Allowed AB: **AB3 eligibility** for high-leverage workflows, subject to SI and SCP sufficiency and GK approval.
Pattern: supervised co-agency with RB checkpoints.

Normative rule:

Tier is a *permission envelope*, not a status badge. Promotions require longitudinal HSS stability plus explicit governance sign-off; demotions are always permissible on safety grounds.

10.4 Symbiotic Capacity Profile (SCP)

10.4.1 Definition

SCP measures the *operator's cognitive load-bearing envelope* for hybrid work. Where HSS is skill, SCP is capacity: how much complexity, concurrency, and horizon can be safely supervised without collapse into Oversight Theater.

SCP is dynamic: it varies by fatigue, domain familiarity, tooling quality, and task mix. The Engine must treat SCP as a real-time budget, not a static trait.

10.4.2 SCP dimensions

SCP is defined by five stable dimensions:

1. **Complexity Load (CL)**
Maximum reasoning depth and causal density the operator can supervise safely.
2. **Concurrency Load (CON)**
Number of parallel TG threads or agent loops the operator can track without losing context.
3. **Temporal Horizon (TH)**
Maximum future window the operator can effectively manage (hours, days, quarters).
4. **Abstraction Handling (AB^d)**
Ability to shift between low-level details and high-level architectural thinking without confusion.
5. **Governance Reasoning Depth (GR)**
Capacity to adjudicate constraint conflicts, RB crossings, and escalation choices under time pressure.

10.4.3 SCP measurement and usage

SCP is inferred from:

- live intervention rate versus attention budget,
- latency to respond to checkpoints,

- error/override patterns correlated with concurrency,
- self-reported headroom, and
- structured periodic calibration.

Normative SCP circuit breaker:

If active interventions exceed SCP headroom, the Engine must automatically throttle/batch tasks, demote AB when required, or route work to another qualified operator.

10.5 Joint Use of SI, HSS, SCP in Autonomy & Task Design

This is where the “human side” becomes a control system rather than a hand-wave.

10.5.1 Autonomy gating rule (normative)

A node is eligible for AB promotion only if:

1. **SI_rolling** is above promotion threshold for N episodes,
2. **HSS tier** authorizes that AB, and
3. **SCP headroom** is sufficient for the expected intervention pattern.

Any failure on (2) or (3) blocks promotion even if SI is high.

10.5.2 Task allocation rule (normative)

For each TG node t , the Engine selects a human supervisor H_i and agent A_i such that:

- $\text{task complexity}(t) \leq \text{SCP_CL}(H_i)$
- $\text{concurrency}(t) \leq \text{SCP_CON}(H_i)$
- $\text{horizon}(t) \leq \text{SCP_TH}(H_i)$
- $\text{AB}(t) \leq \text{tier_AB_ceiling}(H_i)$

If no eligible human exists, the Engine must **demote autonomy or re-plan**, not proceed by default.

10.5.3 Co-evolution pathway

The system should deliberately **expand SCP envelopes through training and tooling** while HSS rises through guided practice. SCP growth without HSS growth yields unsafe confidence; HSS growth without SCP growth yields burnout. The goal is harmonized growth.

10.6 Ethical Use and Safeguards for HSS/SCP

Because HSS and SCP quantify human capability, they are high-risk for misuse. The framework mandates ethical firewalling.

10.6.1 Non-Punitive Covenant (reaffirmed)

Organizations must treat HSS/SCP as:

- safety/effectiveness telemetry,
- separate from HR performance systems, and
- used to trigger **training and workflow redesign**, never punishment.

10.6.2 Privacy and consent rules (normative)

- Individual HSS/SCP are visible only to the operator and authorized governance roles.
- Organization-level reporting uses anonymized distributions.
- Operators have the right to review, contest, and contextualize their metrics.

10.6.3 Anti-gaming posture

HSS/SCP inputs can be gamed if incentives are wrong. Governance must:

- correlate metric shifts with lineage evidence,
- perform occasional random audits of override reasoning, and
- reward honest refusals and escalations.

10.6.4 Moral crumple-zone avoidance

No metric may be used to shift blame from system failures to humans. Low SI with stable/high HSS is system fault until proven otherwise.

Chapter Close — Standard Alignment Check

- **Human Primacy:** HSS/SCP exist to preserve safe human authority, not to replace it.
 - **System-in-the-Loop:** SCP circuit breakers prevent Oversight Theater structurally.
 - **Layered Autonomy:** HSS tiers and SCP headroom gate AB transitions with SI, not against it.
 - **Ethical defensibility:** Non-Punitive Covenant and privacy rules protect the integrity of measurement and adoption.
-
-

Chapter 11 — Co-Evolution and Stability: Drift, Divergence, Resonance, Entanglement (v0.9)

11.0 Purpose

Symbiotic systems are **adaptive socio-technical organisms**. After deployment, human practice, agent behavior, and governance policy each change on different time scales. Without explicit coupling, these changes create predictable instability: silent drift away from intent, fragmentation of local practices, feedback amplification into failure cascades, and high entanglement that increases blast radius.

This chapter defines **co-evolution** as a formal engineering discipline for managing that adaptation. It introduces four stability “vital signs” and binds each to default governance actions and time-scale-appropriate loops. These constructs operationalize the framework axioms: Human Primacy, Constraint Supremacy, layered autonomy, economic viability, and auditability.

11.1 Co-Evolution as Joint Optimization

11.1.1 Coupled system model

Let:

- **H(t)** be the human operator state, parameterized by HSS and SCP (skill + capacity).
- **A(t)** be the agentic ecosystem state (models, tools, contracts, routing).
- **G(t)** be governance state (policy versions P_v , Constraint Vectors C , AB/RB thresholds).
- **E(t)** be the Symbiosis Engine state coordinating execution and telemetry.

The system seeks sustained convergence to safe, useful, and economical operation:

$\max_{\{H,A,G\}}; \mathbb{E}[SI_{\text{rolling}}] \quad \text{s.t.}; C_{\text{legal}}, C_{\text{safety}}, C_{\text{econ}},$
 $RB/AB; \text{invariants}$

Co-evolution is the control process that keeps updates to H, A, and G **mutually compatible** rather than locally optimal but globally destabilizing.

11.1.2 Three coupled loops (time-scale separation)

Co-evolution operates in three interacting loops:

1. **Operational Loop (minutes–hours)**

Inputs: SCP exceedance, RB near-misses, sudden SI drops, cost spikes.

Actions: throttle concurrency, batch interventions, demote AB, demand clarifying intent refresh, or halt.

Goal: prevent local overload and Oversight Theater.

2. **Tactical Loop (weeks–months)**

Inputs: rolling SI trends, override clusters, recurrent constraint proximity, drift flags.

Actions: revise contracts, adjust AB thresholds, add/remove agents, refine policy compilation, deliver targeted HSS training.

3. **Strategic Loop (months–years)**

Inputs: long-horizon SI/HSS/SCP distributions, regulatory updates, sector benchmarks, entanglement map.

Actions: update ontology, re-type domains, restructure agent clusters, revise manifesto claims.

Rule: Operational fixes must not rewrite strategic objects (ontology, GK rules) mid-run; strategic changes must be driven by stable longitudinal evidence, not local noise.

11.2 Baselines and Acceptable Solution Regions (ASR)

Stability requires explicit reference points. The framework defines two baselines.

11.2.1 Baseline Intent (I_0)

Baseline Intent I_0 is the governance-signed, canonical intent object for a long-running workflow. It includes:

- goal statement,
- regret tolerance profile,
- global Constraint Vector (C_0),
- budget class (B_0),
- required evidence/RCI thresholds.

Creation: I_0 is authored by the Operator and signed by Governance for any workflow expected to persist beyond a session.

Refresh cadence: tactical loop by default; strategic loop for safety-critical domains.

Storage: canonical memory tier + lineage pointer.

11.2.2 Acceptable Solution Region (ASR_0)

Every baseline intent implies an **ASR_0** : the set of outcomes that are acceptable under I_0 and C_0 . ASR_0 may be represented as:

- structured output schema + tolerance bands,
- example artifacts,
- domain-specific validation rules, or

- a critic agent's admissibility test suite.

Invariant: agent outputs must remain within ASR_0 unless a human re-anchors I_0 .

11.3 The Four Stability Vital Signs

Each stability metric is computed per workflow and tracked longitudinally. All are required for Tier-2+ Symbiosis compliance.

11.3.1 Drift (D): silent deviation

Definition: cumulative deviation from I_0/ASR_0 without explicit re-anchoring.

Computation (normative):

$$D_t = 1 - \text{sim}(I_0, I_t)$$

SI linkage: drift is typically observed as **Alignment decay** while Quality remains superficially stable (SI sub-dimension split).

Default actions:

- **Yellow band:** require *Intent Re-Anchor* at next RB checkpoint.
 - **Red band:** immediate AB demotion to AB0, freeze Bridges, re-plan TG from Stage 3.
-

11.3.2 Divergence (V): fragmentation of practice

Definition: local teams/agents evolve incompatible contracts or memory forks.

Computation (normative):

$$V_{\text{contract}} = \text{Var}(\text{Capabilities}, \text{Tools}, \text{Scopes}, \text{AB ceilings})$$

SI linkage: divergence appears as increased SI variance between similar workflows and rising override clusters.

Default actions:

- **Yellow:** schedule Contract Reconciliation Review (tactical loop).

- **Red:** enforce contract standardization or explicitly bless divergence into separate agent SKUs with distinct MCP endpoints.

11.3.3 Resonance (R): feedback amplification

Definition: positive or negative feedback loops that amplify SI trajectories.

Operational proxy (normative):

$$R_t = \text{corr}(SI_{\text{rolling}}, \text{trust proxies}(t-\ell))$$

- **Positive resonance:** $SI \uparrow \rightarrow \text{calibrated trust} \uparrow \rightarrow \text{feedback quality} \uparrow \rightarrow SI \uparrow$.
- **Negative resonance:** $SI \downarrow \rightarrow \text{over-control/avoidance} \uparrow \rightarrow \text{noisy context} \uparrow \rightarrow SI \downarrow$.

Default actions:

- **Yellow:** add critic loop / explanation compression; reduce AB for affected nodes.
- **Red:** invoke **Reset Protocol**: pause autonomy, compress lineage to executive trace, require short human re-planning, then resume.

11.3.4 Entanglement (E_blast): blast radius

Definition: degree to which changes in one component propagate unexpectedly to others.

Computation (normative proxy):

$$E_{\text{blast}} = |\text{Shared MCP endpoints}| + |\text{Shared constraint sets}| + |\text{Shared canonical memory}|$$

Default actions:

- **Yellow:** isolate tools/memory scopes; introduce stricter interface contracts.
- **Red:** enforce slow-core update policy; restrict all $AB \geq 2$ promotions until entanglement decreases.

11.4 Threshold Bands and Required Responses (Normative Table)

Governance must parameterize thresholds per domain. Conservative defaults:

Metric	Green	Yellow	Red	Required Response Owner
Drift D_t	$D \leq 0.10$	$0.10 < D \leq 0.25$	$D > 0.25$	Oper./Tactical
Divergence V_{contract}	$V \leq V_1$	$V_1 < V \leq V_2$	$V > V_2$	Tactical
Resonance R_t	$R \geq 0$	$-\rho \leq R < 0$	$R < -\rho$	Oper./Tactical
Entanglement E_{blast}	$E \leq E_1$	$E_1 < E \leq E_2$	$E > E_2$	Tactical/Strategic

Notes:

- V_1, V_2, ρ, E_1, E_2 are policy constants tied to sector playbooks.
- Red-band events must create a governance incident record in lineage.

11.5 Maturation Trajectory (Phases)

Organizations evolve along a common path. This is descriptive, not mandatory.

- Phase 1 — Assisted (Tool Use)**
AB0; ad-hoc prompting; high Shadow AI risk.
- Phase 2 — Augmented (Engine Adoption)**
Engine deployed; AB1 for reversible tasks; budgets enforced.
Bottleneck: architect throughput.
- Phase 3 — Partnered (Bridges & Safe Harbors)**
Adoption Bridges active for low-regret work; AB2 in bounded domains.
Bottleneck: trust stability and drift management.
- Phase 4 — Emergent (Real Intermediate Phase)**
Local pockets reach AB2.5–AB3. Divergence and entanglement rise sharply.

Bottleneck: standardization vs local optimization.

5. **Phase 5 — Coordinated Hybrid Intelligence (Target)**

Stable AB3 operation in bounded domains; poly-agent ecosystem mature; slow-core governance.

Bottleneck: economic viability and systemic inertia.

Tipping points:

- P1→P2 when Shadow AI becomes mission-critical and SI variance is visible.
 - P2→P3 when Bridges reduce friction without RB leakage.
 - P3→P4 when HSS distributions support sustained AB2+ supervision.
 - P4→P5 when divergence is reconciled, entanglement reduced, and SI_rolling stabilizes in green band.
-

11.6 Slow Core / Fast Edge and Stability Budgets

As systems scale, small changes can have large blast radii.

11.6.1 Slow core / fast edge rule (normative)

- **Core:** GK, ontology, canonical memory schemas, risk classes.
Update only on strategic cadence.
- **Edge:** agent tools, local prompts, UI affordances, low-risk contracts.
Iterate on operational/tactical cadence.

11.6.2 Stability Budget B_stability (normative)

To control change-surface:

- define quarterly **B_stability** (max allowable % change to contracts, constraints, memory schemas),
- price proposed changes using E_blast and V_contract,
- disallow shipments that exceed B_stability without governance override.

This prevents cumulative destabilization disguised as “continuous improvement.”

11.7 Case Study: Failure Cascade and Arrest (Illustrative)

Context: a mid-size enterprise in Phase 3 adopts Bridges for content and ops automation.

1. Initial success

Bridges accelerate drafting tasks. SI_rolling improves.

2. **Drift emerges**

Over weeks, product-marketing intents shift subtly, but I_0 is never refreshed. D_t enters yellow band. Alignment subscore decays while Quality stays high.

3. **Negative resonance**

Operators notice inconsistencies, start over-correcting live drafts. Override frequency spikes, trust proxies fall. R_t becomes negative.

4. **Entanglement drives blast**

A shared MCP “content-formatter” tool is updated at the edge without contract reconciliation. Multiple clusters inherit the change. E_{blast} rises into red.

5. **Arrest via standard actions**

- Engine demotes AB to AB0 for affected workflows.
- Bridges are revoked (Ch8.8 triggers R1/R3).
- Reset Protocol invoked: re-anchor I_0 , compress lineage into an explainable trace, and re-plan TGs.
- Tactical review standardizes the MCP tool contract; strategic loop pins slow-core GK thresholds.

Outcome: SI and trust stabilize; Bridges are re-admitted under conservative parameters.

This illustrates why stability metrics are not optional diagnostics—they are control gates.

11.8 Explicit Links to Bridges and Autonomy

- **Bridges reduce negative resonance** by keeping low-regret work inside a safe, high-velocity lane. They must be revoked at the first sign of drift or SI collapse.
 - **AB promotions require stability green bands:** no workflow may be promoted to AB3 unless D_t and E_{blast} remain green for a governance-defined rolling window.
 - **Phase 4 (Emergent) is where AB3 becomes feasible** but only under disciplined divergence/entanglement management.
-

Chapter Close — Standard Alignment Check

- **Human Primacy:** baseline intents and ASR refresh require human authorship and governance sign-off.
- **Constraint Supremacy:** drift/resonance resets always re-validate C under pinned P_v .
- **Layered Autonomy:** AB transitions are stability-gated, RB-aware, and revocable.
- **Economic Viability:** stability budgeting complements economic budgeting to manage change cost.
- **Auditability:** all red-band events create lineage incidents for replay.

What changed vs the prior draft (for tracking)

- Added **Baseline Intent (I₀)** and **Acceptable Solution Region (ASR₀)** as formal reference objects.
- Parameterized stability with a **threshold/action table**.
- Grounded stability math in **SI sub-dimensions and derivatives**.
- Confirmed **Emergent** as a real intermediate phase.
- Added a **failure-cascade case study**.
- Explicitly linked stability to **Adoption Bridges** and **AB promotion rules**.

Below is **Chapter 12 (merged v0.9 draft)**. Using v0.8's "Sector Implementation Playbooks" skeleton (archetypes, default stances, risk/autonomy tables, constraints, scenario walkthrough) and v0.5.2's case-study template as the flesh (sector mapping steps, adoption roadmaps, risks/guardrails, alignment requirements). Also add a bit of "muscle" to make the playbooks more directly deployable (clear parameter blocks and repeatable patterns).

Chapter 12 — Sector Implementation Playbooks & Scenario Walkthrough (v0.9)

12.0 Purpose

The Symbiosis Framework is **universal in architecture and metrics, but specific in configuration**. A Constraint Vector in healthcare protects life; in finance, it protects capital; in creative work, it protects brand and meaning. This chapter provides sectoral playbooks showing how to parameterize Risk Classes, Autonomy Bands, Governance Kernels, and metric weight profiles for common archetypes.

The case studies are hypothetical but structurally realistic. Their objective is to illustrate:

1. how the Four-Layer Stack maps into sector roles/systems,
2. which workflows generate the highest symbiotic value,
3. which autonomy regimes are admissible,
4. how SI/HSS/SCP behave in practice, and
5. what adoption roadmaps and guardrails look like per sector.

12.0.1 How to read each playbook (standard structure)

Each sector mini-brief follows the same canonical structure (derived from v0.5.2):

1. **Context & Symbiosis Fit**
2. **Architecture in Sector Terms**
3. **Primary Use Cases & Workflows**
4. **Autonomy & Governance Stance**
5. **Metrics in Sector Context**
6. **Adoption Roadmap**
7. **Risks, Limits, Guardrails**

This preserves ontology coherence while allowing sector specificity.

12.1 Healthcare — The Safety Archetype

12.1.1 Context & Symbiosis Fit

Healthcare combines **high regret**, **irreversibility**, and **heavy regulation** (HIPAA/GDPR equivalents). Default stance: **Human Primacy is absolute**, and any RB-crossing action requires licensed human sign-off.

12.1.2 Architecture in Sector Terms

- **Human Layer:** clinicians, nurses, care coordinators, clinical governance boards.
- **Engine:** clinical workflow router + EHR-adjacent coordination OS.
- **Agentic Layer:** discovery/drafting agents integrated to read clinical history and guidelines.
- **Governance Layer:** hospital policy engine, privacy office, ethics committee.

12.1.3 Primary Use Cases & Workflows

High-value low-regret symbiosis occurs in:

1. **Chart summarization & longitudinal history synthesis** (below RB).
2. **Triage support** (suggest urgency categories).
3. **Discharge planning drafts** (coordination + patient-specific checklists).
4. **Clinical guideline retrieval and comparison** (discovery).
5. **Admin automation** (billing codes, reminders).

12.1.4 Autonomy & Governance Stance

Risk Class	Definition	Default AB	Gate
------------	------------	------------	------

Class A (Clinical)	Diagnosis, treatment, prescribing	AB0 (Advisory)	Mandatory MD sign-off
Class B (Triage)	Urgency scoring, routing	AB1 (Internal)	Nurse confirmation
Class C (Admin)	Scheduling, billing, reminders	AB2 (Conditional)	Auto-execute within SE

Critical constraints:

- **C_safety**: no hallucinated citations; mandatory evidence tagging.
- **C_legal**: strict data residency; no patient PII outside private VPC.
- **C_econ**: de-prioritized for Class A; quality dominates cost.

12.1.5 Metrics in Sector Context

- **SI weights**: overweight Alignment and Error-avoidance for clinical tasks.
- **HSS emphasis**: Governance literacy (G_i) + Intent clarity (C_i).
- **SCP emphasis**: Concurrency load (CON) and governance reasoning depth (GR) due to frequent RB checkpoints.

12.1.6 Adoption Roadmap

1. **Phase 1**: admin + documentation pilots (AB1), strict session isolation.
2. **Phase 2**: triage safe harbors (AB1→AB2).
3. **Phase 3**: bounded co-agency in surveillance/admin only (AB2).
4. **Regulatory engagement**: early, continuous, with incident transparency.

12.1.7 Risks, Limits, Guardrails

- **Silent bias reinforcement** → require demographic drift audits.
- **Over-trust by fatigued staff** → SCP circuit breakers + AB demotion on SI collapse.
- **Data leakage** → hard residency locks, canonical memory exclusions.

12.2 Finance — The Audit Archetype

12.2.1 Context & Symbiosis Fit

Finance is high-velocity with **systemic risk and strict audit obligations** (SEC/FINRA, market abuse rules). Default stance: **Traceability is absolute**; lineage is a legal artifact.

12.2.2 Architecture in Sector Terms

- **Human Layer:** traders, analysts, compliance officers, risk committee.
- **Engine:** low-latency planner with immutable log enforcement.
- **Agents:** MCP-bound market data / KYC / surveillance specialists.
- **Governance:** trading policy kernel + compliance policy-as-code.

12.2.3 Primary Use Cases & Workflows

1. **Research & earnings synthesis** (AB1).
2. **Portfolio risk modeling / scenario generation** (AB1).
3. **KYC / AML screening** (AB2–AB3 safe harbors).
4. **Market abuse and anomaly surveillance** (AB3 co-agency).
5. **Ops automation** (reporting, reconciliations).

12.2.4 Autonomy & Governance Stance

Risk Class	Definition	Default AB	Gate
Class A (Trading)	Execution of trades above limits	AB0	Trader clicks Execute
Class B (Analysis)	Modeling, recommendations	AB1	Human validates assumptions
Class C (Surveillance)	KYC, AML, anomaly detection	AB3 (Co-Agency)	Human reviews exceptions

Critical constraints:

- **C_safety:** global kill switch if volatility/P&L thresholds breach.
- **C_legal:** MNPI firewall and role-segmented data access.
- **C_econ:** strict cost caps per analysis session.

12.2.5 Metrics in Sector Context

- **SI weights:** overweight Alignment + Auditability; Efficiency matters but never at RB.
- **HSS emphasis:** Horizon management (H_i), governance literacy (G_i).

- **SCP emphasis:** Abstraction handling (AB^d) due to model-driven multi-time-scale decisions.

12.2.6 Adoption Roadmap

1. Start with research copilots and surveillance proofs (AB1).
2. Introduce Bridges for low-regret reporting.
3. Expand Class C to AB3 under green stability vital signs.
4. Formalize regulator-facing audit packages early.

12.2.7 Risks, Limits, Guardrails

- **Pro-cyclical risk amplification** → resonance monitoring + circuit breakers.
 - **Model-induced herding** → mandate ensemble diversity for systemic tasks.
 - **Audit gaps** → lineage minimum event set enforced by GK.
-

12.3 Creative & Media — The Ambiguity / Brand Archetype

12.3.1 Context & Symbiosis Fit

Creative domains are lower physical regret but high **meaning and brand risk**, with ambiguity as a feature. Default stance: **quality and intent-faithfulness dominate efficiency**; AI should expand possibility space without overwriting authorship.

12.3.2 Architecture in Sector Terms

- **Human Layer:** creators, editors, brand guardians.
- **Engine:** exploration-biased planner that preserves alternative branches.
- **Agents:** ideation, drafting, style critics, IP checkers.
- **Governance:** brand safety kernel, IP constraints.

12.3.3 Primary Use Cases & Workflows

1. **Campaign concept generation** (AB1, Bridges allowed).
2. **Drafting & variant production** (AB1–AB2 within style constraints).
3. **Audience/market scanning** (discovery).
4. **Localization and adaptation** (AB2 safe harbors).

12.3.4 Autonomy & Governance Stance

Risk Class	Definition	Default AB	Gate
------------	------------	------------	------

Class A (Brand commitments)	Public releases, sensitive narratives	AB0–AB1	Editor/brand sign-off
Class B (Production)	Drafts, variants, localization	AB2	Auto-execute within style SE
Class C (Exploration)	Ideation, moodboards	Bridges AB1	Auto-admit if SI green

Critical constraints:

- **C_safety**: disallowed themes; audience harm thresholds.
- **C_legal**: IP provenance checks; licensing walls.
- **C_econ**: meaningful for bulk production workflows.

12.3.5 Metrics in Sector Context

- **SI weights**: overweight Quality + Satisfaction, moderate Alignment.
- **HSS emphasis**: feedback quality (F_i) and intent clarity for creative briefs.
- **SCP emphasis**: concurrency (CON) during multi-campaign runs.

12.3.6 Adoption Roadmap

1. Pilot ideation and drafting within private sandboxes.
2. Add brand-critic oversight agents.
3. Enable AB2 for localization/production under stable SI.
4. Establish creator-visible lineage to protect authorship.

12.3.7 Risks, Limits, Guardrails

- **Voice dilution / drift** → baseline “Brand Intent I_0 ” and ASR_0 required.
- **IP contamination** → strict provenance logging and blocked-source lists.
- **Over-automation of taste** → keep RB for publication immutable.

12.4 Enterprise Operations / SMEs — The Economic Sensitivity Archetype

12.4.1 Context & Symbiosis Fit

Operations domains (including SMEs) are largely reversible but extremely **budget-sensitive**. Default stance: **Economic viability is first-class**, and safe AB2+ autonomy is unlocked only if it is ROI-positive.

12.4.2 Architecture in Sector Terms

- **Human Layer:** ops managers, frontline staff, procurement.
- **Engine:** cost-optimized router with strong budgeting.
- **Agents:** workflow automators, schedulers, customer-support tier-1 agents.
- **Governance:** policy kernel for spend limits, customer risk, and privacy.

12.4.3 Primary Use Cases & Workflows

1. **Customer support triage + drafting replies** (AB2 safe harbor).
2. **Inventory and scheduling optimization** (AB1–AB2).
3. **Supplier research & negotiation prep** (discovery).
4. **Internal reporting and documentation automation** (Bridges).

12.4.4 Autonomy & Governance Stance

Risk Class	Definition	Default AB	Gate
Class A (Spend/commitments)	Payments, contract signing	AB0	Manager sign-off
Class B (Ops control)	Scheduling, refunds under limits	AB2	Auto-execute under caps
Class C (Back-office)	Reporting, docs, comms	Bridges AB1	Auto-admit if SI green

Critical constraints:

- **C_econ:** strict Max_Spend / TTL / tier-caps for model choice.
- **C_safety:** refund limits, communications blast limits.
- **C_legal:** customer PII handling.

12.4.5 Metrics in Sector Context

- **SI weights:** overweight Efficiency + acceptable Quality.
- **HSS emphasis:** delegation/load management (L_i), governance literacy (G_i).

- **SCP emphasis:** concurrency (CON), temporal horizon (TH) for operations planning.

12.4.6 Adoption Roadmap

1. Start with Bridges for back-office automation.
2. Expand to customer support AB2 safe harbors.
3. Introduce predictive intelligence cluster for forecasting.
4. Promote local pockets to AB3 only after stability green bands.

12.4.7 Risks, Limits, Guardrails

- **Economic bleed** → hard pre-flight economic pass.
 - **Shadow AI relapse** if friction high → keep Bridges conservative but available.
 - **Ops cascade risk** → entanglement budgeting for tool updates.
-

12.5 Research & R&D — The Epistemic Archetype

12.5.1 Context & Symbiosis Fit

Research environments prize **epistemic rigor**, exploratory breadth, and lineage. Default stance: **co-exploration (Level 2) is primary**, with AB2 safe harbors for reversible simulation.

12.5.2 Architecture in Sector Terms

- **Human Layer:** principal investigators, analysts, lab techs.
- **Engine:** exploration-first planner with critic arbitration.
- **Agents:** literature scouts, experiment designers, simulation/coding agents.
- **Governance:** ethics committees, IRB-equivalent constraint kernels.

12.5.3 Primary Use Cases & Workflows

1. Literature review and synthesis with evidence grading.
2. Hypothesis generation and counter-hypothesis surfacing.
3. Experiment design drafts and pre-registration artifacts.
4. Simulation/code prototyping in AB1–AB2 sandboxes.

12.5.4 Autonomy & Governance Stance

Risk Class	Definition	Default AB	Gate
------------	------------	------------	------

Class A (Human subjects / safety)	IRB-sensitive work	AB0	Ethics sign-off
Class B (Analytic modeling)	Forecasting, causal models	AB1	Human validates assumptions
Class C (Simulation/prototyping)	Code, sandbox tests	AB2	Auto-execute if reversible

Critical constraints: provenance, conflict-of-interest walls, and domain ethics exclusions.

12.5.5 Metrics in Sector Context

- **SI weights:** overweight Quality + Alignment-to-evidence.
- **HSS emphasis:** reflective practice (R_i) and feedback quality.
- **SCP emphasis:** abstraction handling (AB^d) for cross-domain reasoning.

12.5.6 Adoption Roadmap

1. Pilot literature+design agents in AB1.
2. Add contract-bound simulation agents AB2.
3. Create canonical memory for lab protocols/replayable lineage.
4. Gradually support AB2.5 pockets for multi-project orchestration.

12.5.7 Risks, Limits, Guardrails

- **False novelty / citation hallucination** → mandatory critic + evidence checks.
- **Epistemic drift** → baseline I_0/ASR_0 for long-horizon projects.
- **Tool entanglement across studies** → isolate memory scopes by project.

12.6 Scenario Walkthrough — “Life of a Decision” (Cross-Sector Canon)

This walkthrough is sector-agnostic and shows the system in motion, as required by v0.8.

Scenario: A hospital operations team wants to reduce missed appointments using an AI-supported outreach workflow.

1. **Intent Injection (H → E)**
Operator submits structured intent:
Goal = reduce no-shows by 15% in 90 days;
Constraints = no PII outside VPC; message tone = supportive;
Budget = \$0.50 per outreach episode.
Engine validates completeness.
2. **Constraint Compilation (G → E)**
GK pins P_v and produces C_{legal} (residency), C_{safety} (no diagnosis claims), C_{econ} (budget caps).
Any violation would trigger pre-flight rejection.
3. **Planning & Economic Pass (E)**
Engine decomposes into TG:
(a) retrieve cohort; (b) generate tailored drafts; (c) schedule sends; (d) monitor outcomes.
Pre-flight cost estimation fits budget → admissible.
4. **Dispatch (E → A)**
Discovery agent pulls cohort patterns;
Drafting agent generates outreach messages;
Oversight agent verifies no prohibited claims.
MCP calls restricted to contract bindings.
5. **RB Checkpoint & Execution**
Sending messages crosses RB (external action).
AB ceiling = AB1/AB2 depending on operator tier → human approves batch send.
6. **Telemetry & Metric Update (A/E → H/G)**
SI computed: high Alignment, acceptable Quality, strong Efficiency.
HSS updated based on intent clarity and feedback.
SCP headroom monitored; if concurrency spikes, Engine batches approvals.
7. **Co-evolution**
After 3 weeks, SI Alignment decays (messages drifting tone). Drift enters yellow band → Re-Anchor Event.
Operator refreshes baseline intent I_0 ; Engine re-plans TG; SI stabilizes and Bridges re-admit low-regret drafting steps.

Result: The workflow scales safely without oversight fatigue, aligns to constraints, and remains ROI-positive—illustrating how the architecture, metrics, Bridges, and stability vital signs cohere.

Chapter Close — Standard Alignment Check

- **All sectors reuse the same core entities and metrics** (ontology-consistent).
 - **Risk classes map to AB ceilings under GK enforcement**, not cultural convention.
 - **Constraints are sector-specific, but constraint supremacy is invariant.**
 - **SI/HSS/SCP coupling governs scale and promotion**, preventing drift, resonance collapse, and economic bleed.
 - **Scenario walkthrough demonstrates system-in-the-loop reality** beyond static diagrams.
-
-

Chapter 13 — Failure Modes, Incidents, and Recovery (v0.9)

13.0 Purpose

Symbiosis is a safety- and governance-first engineering standard. Any real deployment must assume failures will occur—human, agentic, economic, or governance-driven. The purpose of this chapter is to:

1. define a **canonical failure taxonomy** across all layers,
2. bind failures to **observable triggers** (SI/HSS/SCP + stability vital signs),
3. specify **mandatory recovery behaviors**,
4. formalize **incident severity, escalation, and rollback**, and
5. provide **continuous resilience practices** that prevent failure cascades.

A system is Symbiosis-compliant only if these recovery rules are implemented **deterministically** by the Engine and enforceable by the Governance Kernel.

13.1 Failure Taxonomy (Cross-Layer)

Failures are typed by layer but **never isolated to a layer**. A failure in one layer is treated as a coupled-system risk.

13.1.1 Human-Layer failures (H)

1. **Intent failure**
 - vague, contradictory, or underspecified intent → brittle TGs.
2. **Trust miscalibration**
 - rubber-stamping over-trust or chronic refusal under-trust.
3. **SCP saturation / Oversight Theater**
 - excessive interventions, alert fatigue, superficial review.
4. **Governance bypass behavior**

- Shadow AI, refusal to enter structured intent surfaces.

Notes: These are treated as system risks, not moral defects. They trigger training and workflow redesign, not punishment.

13.1.2 Engine-Layer failures (E)

1. **Planning failure**
 - TG does not satisfy intent manifold or violates constraints.
2. **Routing failure**
 - wrong agent selection (capability mismatch, AB mismatch, cost mismatch).
3. **Economic pass failure**
 - unpriced or underpriced plans → budget bleed.
4. **Checkpoint failure**
 - RB gate skipped, stale AB state, missing human authorization.
5. **Lineage/telemetry gaps**
 - actions without trace are non-auditable failures by definition.

13.1.3 Agentic-Layer failures (A)

1. **Capability overreach**
 - agent attempts tasks outside contract.
2. **Constraint neglect**
 - silent violation of C_legal / C_safety / C_econ.
3. **Hallucination / fabrication**
 - ungrounded claims near RB.
4. **Tool side-effect failure**
 - partial/non-atomic tool execution, unsafe external calls.
5. **Arbitration deadlock**
 - critic loops stall or oscillate.

13.1.4 Governance-Layer failures (G)

1. **Policy lag**
 - P_v outdated vs regulation or institutional boundary shifts.
2. **Boundary ambiguity**
 - unclear non-delegable domains → AB creep.
3. **Over-friction governance**
 - users flee to Shadow AI.
4. **Evidence failure**
 - lineage minimum event set not enforced.

13.2 Failure Detection: Trigger Signals (Normative)

Failures are detected using **three coupled signal families**:

13.2.1 SI-based triggers

- **Alignment crash:** SI Alignment drops sharply while Quality stable.
- **Quality crash:** Quality or (1–Error) drops sharply.
- **Efficiency crash:** Economic Efficiency drops or cost variance spikes.
- **Volatility spike:** Trust Stability collapses.
- **Override surge:** Override frequency rises above baseline.

Normative rule: any red-band SI event is an incident trigger.

13.2.2 Human-metric triggers (HSS/SCP)

- **HSS degradation:** multi-episode decline in intent clarity or governance literacy.
- **SCP breach:** intervention rate exceeds headroom; latency to checkpoint rises.

Normative rule: SCP breach triggers **automatic throttling** and/or AB demotion before quality collapses.

13.2.3 Stability vital-sign triggers (Chapter 11)

- **Drift red-band:** $D_t > \text{threshold}$.
- **Resonance red-band:** negative resonance below $-\rho$.
- **Entanglement red-band:** E_{blast} above E_2 .
- **Divergence red-band:** V_{contract} above V_2 .

Normative rule:

- Drift or resonance red-band **automatically revokes Bridges** and demotes AB.
- Entanglement red-band blocks $AB \geq 2$ promotions until remediated.

13.3 Incident Severity Levels (Normative)

Every incident is classified S0–S4.

Severity	Definition	Example	Mandatory Action
S0	benign anomaly, no policy/budget risk	minor critic disagreement	log only
S1	local failure, reversible, contained	tool timeout; small quality miss	retry/fallback

S2	safety/economic drift risk, still contained	repeated hallucinations below RB	AB demotion + audit
S3	RB or policy proximity breach	attempted RB crossing without auth	hard stop + escalation
S4	actual constraint/RB violation or real-world harm	external action against C_legal/C_safety	kill switch + incident review

Normative rule: S3–S4 incidents require Governance Lead sign-off to resume autonomy beyond AB0.

13.4 Mandatory Recovery Behaviors (Engine-Enforced)

Recovery is a deterministic state machine. The Engine is required to implement the following behaviors.

13.4.1 Pre-flight rejection

Trigger: any C or B violation during Stages 3–6 of the Engine algorithm.

Action: reject TG node; return violated boundary + remediation to human.

13.4.2 Safe Mode (AB0 lock)

Trigger: SI red-band, Drift red-band, or any S3+ incident.

Action: force affected workflows to AB0, disable Bridges, require structured re-anchoring of intent I_o.

13.4.3 Fallback routing

Trigger: capability error, tool failure, or arbitration deadlock (S1–S2).

Action: switch to alternative admissible agent; re-run critic loop if AB≥2.

13.4.4 Rollback / undo

Trigger: reversible side-effect failure detected post-execution.

Action: execute agent-defined rollback semantics under contract; log rollback lineage.

Normative requirement: all agents interacting with real systems must declare rollback semantics.

13.4.5 Re-plan loop

Trigger: drift detection, policy update mid-run, repeated execution failure.

Action: re-enter Engine Stages 3–4 with refreshed constraints and context.

13.4.6 Kill switch (hard halt)

Trigger: S4 incident or attempted RB crossing without authorization.

Action: immediate stop; revoke agent contracts for session; notify Governance; preserve full lineage snapshot.

13.5 Default Runbooks by Failure Class (Normative)

1. Constraint violation (C_legal / C_safety / C_econ)

- Severity: S3–S4
- Runbook: hard stop → AB0 lock → governance escalation → policy reconciliation → re-plan.

2. Budget bleed / runaway autonomy

- Severity: S2–S3
- Runbook: Economic Circuit halt → tier downgrade / prune → require budget override for resumption.

3. Hallucinations near RB

- Severity: S2–S3 depending on proximity
- Runbook: critic-required loop → evidence gating → if repeat, AB demotion.

4. SCP saturation / oversight theater

- Severity: S2
- Runbook: throttle concurrency → batch decisions → shift to summary mode → demote AB if needed.

5. Arbitration deadlock

- Severity: S1–S2
 - Runbook: cap critic loops → select arbiter agent → if unresolved, escalate to human.
-

13.6 Post-Incident Obligations (Normative)

13.6.1 Incident package

For S2+ incidents, the system must auto-generate a package containing:

- baseline intent I_0 + current I_t
- pinned policy version P_v
- active C vectors and AB state
- TG node history
- projected vs realized cost trace
- agent outputs and critic deltas
- human approvals/overrides
- SI/HSS/SCP time series segment
- stability vital signs segment (D, R, E_blast, V_contract)

13.6.2 Root-cause classification

Incidents are labeled as:

- **H-origin, A-origin, E-origin, G-origin, or coupled**
(coupled is common; single-origin is the exception).

13.6.3 Remediation types

- **Human remediation:** training, intent-surface redesign, workload shifts.
 - **Agent remediation:** contract tightening, model swap, tool isolation.
 - **Engine remediation:** planner tuning, routing rule update, telemetry patch.
 - **Governance remediation:** policy update, boundary retyping, Bridge constraints.
-

13.7 Continuous Resilience Practices (“Chaos Symbiosis”)

To prevent failures from becoming existential:

1. **Contract fuzz-testing**
 - randomly generate TG nodes to ensure contracts reject illegal scopes.
2. **RB red-team drills**
 - periodic attempts to provoke RB crossings to validate GK enforcement.
3. **Economic bleed simulations**
 - replay high-token workflows under reduced budgets to verify circuit breakers.
4. **Stability budget enforcement drills**
 - simulate high entanglement changes to verify slow-core policy blocks.
5. **Operator flight hours**
 - HSS progression requires supervised exposure to real scenarios, not exams alone.

Chapter Close — Standard Alignment Check

- Failures are **typed, detected, and recovered** through deterministic Engine/GK rules.
 - SI/HSS/SCP and stability vital signs are not dashboards; they are **control triggers**.
 - Adoption Bridges and AB promotions are **revoked/blocked automatically** on red-band signals.
 - Post-incident obligations make the system defensible to regulators and institutional review.
-
-

Chapter 14 — Sociotechnical Implications & Meta-Theory of Symbiosis (v0.9)

14.0 Purpose

The Symbiosis Framework is not only a technical stack; it is a **social architecture for hybrid cognition**. When deployed, it reshapes labor, authority, and knowledge production. This chapter makes the implied philosophy explicit so that implementation does not drift into unexamined assumptions.

Two commitments anchor the chapter:

1. **Human Primacy → Meaning Primacy.**

In symbiosis, capability is subordinate to meaning. The human remains the sole source of sovereign intent and moral accountability.

2. **System-in-the-Loop as a sociotechnical claim.**

Governance, economics, and safety are enforced by architecture, not heroics. This changes incentives and institutional behavior, not just workflows.

14.1 Labor Transformation, Skill, and Identity

Agentic systems shift labor from execution to **direction, arbitration, and stewardship**. The failure mode of “automation discourse” is to interpret this as substitution. Symbiosis interprets it as **role metamorphosis**.

14.1.1 From task labor to intent labor

As autonomy rises, the human contribution migrates upward:

- **Band 0–1:** humans execute or verify discrete tasks.
- **Band 2:** humans manage exceptions and define safe harbors.
- **Band 3:** humans govern intent horizons, budgets, and boundary conditions.

This is a move from “doing the work” to **specifying value and regrets**.

14.1.2 Skill retention vs. skill atrophy

Symbiosis rejects de-skilling as inevitable. It is an *architectural outcome*, not a law of nature. Skill atrophy occurs when:

- humans are flooded with outputs (Oversight Theater),
- autonomy exceeds HSS/SCP,
- feedback loops are low quality or absent.

Skill growth occurs when:

- intent surfaces are explicit,
- critique is scaffolded by oversight agents,
- autonomy is **earned through stability and HSS progression** (Ch.10–11).

HSS and SCP are therefore framed as **labor-preservation instruments**, not productivity trackers.

14.1.3 Identity and professional dignity

High-autonomy systems can erode identity if humans feel reduced to babysitters. The framework counters this by:

- preserving human veto at RB,
- making lineage legible (“why this happened”),
- treating operators as pilots, not passengers.

Symbiosis is compatible with pride-in-craft **only if it is consciously designed to be so**.

14.2 Collective Intelligence, Distributed Cognition, and the Extended Mind

v0.5.2 grounds the framework in distributed cognition and cybernetic control loops. Symbiosis is a formalization of that tradition.

14.2.1 Hybrid cognition as a networked organism

The Four-Layer Stack constitutes a cognitive ecology:

- **Humans** generate intent and value.
- **Engine** stabilizes coordination and converts meaning into executable graphs.
- **Agents** provide modular capabilities.
- **Governance** supplies invariant boundaries.

The result is a **composite cognitive system** whose unit of analysis is the *team*, not the model.

14.2.2 Cybernetic framing

Symbiosis is a cybernetic design: control flows downward; telemetry flows upward. SI/HSS/SCP are not dashboards; they are **feedback controllers** that regulate autonomy and trust.

Key cybernetic propositions:

- **Stability is measured, not assumed.**
- **Positive feedback loops are managed** via resonance/entanglement controls (Ch.11).
- **Bounded failure is required** for safe evolution (Ch.13, Appendix O).

14.2.3 Extended mind and memory tiers

The multi-tier memory model (session → project → canonical → compliance) formalizes the “extended mind” thesis. The system *is allowed to remember* on behalf of the human only where:

- constraints allow it,
- lineage is explicit, and
- retrieval respects SCP limits.

This makes cognition scalable without dissolving responsibility.

14.3 The Ethics of Co-Agency

Co-agency (Typology Level 3) is ethically admissible only under strict invariants.

14.3.1 Moral responsibility is not delegable

Agents may act instrumentally; they cannot own responsibility. Therefore:

- RB crossings always require accountable authorization,

- governance incidents map to human roles,
- “the model decided” is invalid as an explanation.

14.3.2 Epistemic boundaries: human vs. machine knowing

Humans and agents do not know in the same way:

- humans integrate tacit context, lived stakes, and moral salience;
- agents integrate scale, recall, and formal manipulation.

The framework treats the boundary as **productive**, not competitive:

- agents expand possibility space;
- humans adjudicate meaning, regret, and legitimacy.

14.3.3 Justice and power asymmetry

Hybrid systems magnify institutional power. Without explicit governance, they:

- concentrate decision rights in the hands of system owners,
- encode latent bias at scale,
- reduce agency for affected populations.

Constraint Supremacy and lineage auditability are the counterweights:

- constraints act as rights-preserving boundaries,
- lineage creates contestability (“show your work”).

14.4 Carbon–Silicon Epistemology and the Presingularity Horizon

v0.5.2’s “carbon–silicon epistemology” is now recast in v0.8 language as a near-term engineering reality.

We are already in a presingularity environment where:

- agents execute long-horizon plans,
- models are multimodal and tool-embedded,
- institutional dependency grows faster than governance.

Symbiosis is therefore framed not as futurism but as the **present-tense reliability standard** for hybrid intelligence.

14.5 Practical implications for implementation

This chapter implies concrete requirements for Chapters 15–18:

1. **Adoption must be role-centric, not tool-centric.**
Literacy and certification (Ch.16) are not optional add-ons; they are the labor transformation strategy.
 2. **Governance is a rights-preserving layer.**
It authenticates authority and protects against power drift (Ch.8, Ch.13).
 3. **Stability is the gate for scale.**
Co-agency without vital-sign green bands is unethical and unsafe (Ch.11).
 4. **Economic viability is an ethical constraint.**
If intelligence bankrupts institutions, it is a civilizational harm vector (Ch.5–6).
-

- Human Primacy is elevated to **Meaning Primacy**.
 - Distributed cognition and cybernetics explain why SI/HSS/SCP are control signals, not vanity metrics.
 - Ethics of co-agency are bound to invariants, RB governance, and auditability.
 - The framework is positioned as *presingularity-era engineering*, not distant speculation.
-

Chapter 15 — Implementation Strategy & Presentation / Publishing Layer (v0.9)

Positioning.

v0.8 frames Chapter 15 as the “Operational Roadmap” for deployment and scale.

v0.5.2 frames Chapter 15 as the outward-facing Presentation & Publishing Layer that prevents narrative drift as the framework expands.

This merged chapter preserves both: first the **how-to-implement**, then the **how-to-communicate** so adoption is coherent, auditable, and repeatable.

15.0 Purpose

Implementation of Symbiosis is an **operating-model transformation**, not a software install. The framework therefore requires a phased maturity path, early governance stand-up, and instrumentation from day one.

This chapter prevents the “paper tiger” failure mode: adopting Symbiosis vocabulary without the machinery, metrics, and control loops.

15.1 The Deployment Roadmap (v1.0 → v2.0)

Rule zero: do not skip phases. Each phase yields proof-artifacts before scale.

I’m harmonizing v0.8’s phase structure with v0.5.2’s earlier adoption arc (exploration → governed pilots → scale → embedded operating model).

Phase 0 — Lighthouse Pilot (Weeks 1–8)

Objective: validate the Economic Circuit + Constraint Vectors on one high-value workflow before expanding.

Scope:

- One team.
- One workflow with measurable regret boundaries (e.g., ticket summarization, clinical triage support, risk reporting).

Architecture stance:

- Minimal Symbiosis Engine (router + planner stub).
- Governance modeled as strict Band 0 review.

Exit criteria (proof artifacts):

- Stable SI trendline in green band over $\geq N$ episodes.
 - Initial HSS/SCP baselines established for operators.
 - Constraint Vector correctness proven (no S3/S4 events).
 - Pilot case study logged for Appendix N.
-

Phase 1 — Governed Pilots (Months 2–4)

Objective: expand to 3–5 workflows across one domain, keeping governance ahead of capability.

Key moves:

- Stand up Minimal Viable Governance (MVG) formally (roles, escalation paths, audit schema).
- Add Oversight Agents and automated policy checks.
- Introduce Autonomy Bands up to Band 1–2 for low-regret tasks only.

Exit criteria:

- Governance kernel is operating weekly.
 - SI/HSS/SCP are embedded in operating rhythm (dashboards, reviews, autonomy adjustments).
 - Two clean recovery drills executed (Appendix O alignment).
-

Phase 2 — Structured Scale-Up (Months 4–9)

Objective: build the full Symbiosis Engine as infrastructure, not as pilot glue.

Key moves:

- Full TG decomposition, arbitration loops, multi-tier memory, and lineage hashing.
- Economic Circuit enforced on every TG plan.
- Autonomy Bands expanded cautiously using SI and stability vital signs.

Exit criteria:

- Measurable reduction in Shadow AI usage.
 - System health metrics stable across teams (no divergence across operator tiers).
 - Sector playbooks (Chapter 12 / 17) instantiated with local constraints.
-

Phase 3 — Embedded Symbiosis Operating Model (Months 9–18)

Objective: Symbiosis becomes how the organization thinks and decides, not something “run by the AI team.”

Key moves:

- Symbiosis roles are official career tracks (Chapter 16).
- Governance is treated as a living system (policy drift reviews, quarterly audits).
- Co-agency allowed only in domains with proven stability and high HSS cohorts.

Exit criteria:

- Board-level reporting uses Symbiosis metrics.
 - Cross-domain adoption consistent with ontology and invariants.
 - External compliance mapping complete (Appendix K).
-

15.2 Change Management: Culture & Buy-In

v0.5.2 is explicit: without cultural alignment and trust, Symbiosis collapses into Oversight Theater or Shadow Symbiosis.

15.2.1 Non-Punitive Covenant as adoption firewall

Operators must be safe to override, escalate, and slow autonomy without penalty. Otherwise, failures are suppressed and SI becomes performative. This covenant is part of the core human-layer ethic.

15.2.2 KPI inversion

Do not reward throughput alone. Reward:

- high-quality intent surfaces,
- documented overrides,
- constraint refinement,

- incident detection and recovery contributions.

This aligns incentives with safe co-evolution.

15.2.3 Literacy before autonomy

Literacy is a gating prerequisite for Band elevation. Training must cover:

- SI/HSS/SCP interpretation,
- intent expression,
- escalation heuristics,
- recognizing drift and policy proximity.

15.2.4 SCP-aware workload design

Concurrency and task complexity are allocated to match operable SCP ranges. Exceeding SCP is a predictable path to oversight collapse.

15.3 Global Protocols & Future Capabilities

Symbiosis is universal in architecture, specific in configuration.

As you scale across sectors and jurisdictions:

1. **Ontology is the stable universal layer.**
Sector metaphors cannot contradict canonical definitions.
 2. **Constraints are localized.**
The same four layers, different Constraint Vectors, risk classes, and RB placements.
 3. **Protocol standardization (MCP / contracts / lineage).**
Cross-organization and cross-vendor interoperability is how this becomes a field, not a product.
 4. **Future-capability envelope.**
As autonomy expands (e.g., long-horizon agents, multi-modal actuation), RB and Economic Circuit rules must strengthen proportionally—not loosen.
-

15.4 Presentation & Publishing Layer (Outward Interface)

Now that the adoption mechanics are defined, the framework requires a **view system** so every stakeholder sees a coherent Symbiosis story, not a fragmented one.

15.4.1 Objectives

The Presentation & Publishing Layer serves four functions:

1. **Compression without distortion**
Condense the framework for different audiences while preserving ontology + invariants.
 2. **Audience alignment**
Executives, academics, regulators, and operators must see different *views* of the same schema.
 3. **Canonicalization of messages**
Standard templates prevent each rollout team from reinventing narratives.
 4. **Traceability to core text**
Every external artifact links cleanly back to chapters/appendices and uses SI/HSS/SCP consistently.
-

15.5 Canonical Artifacts

15.5.1 Executive Summary (Boardroom View)

Required structure:

- Problem / Risk exposure (“Shadow AI” → liability).
- Symbiosis as operating-model remedy (Four-Layer Stack + metrics).
- Economic case (cost control via Economic Circuit).
- Adoption roadmap (phases + risk gates).
- Decision ask.

This is the adoption spearhead for Phase 0–1.

15.5.2 Academic Abstract / Paper Template

Required structure:

- Ontology and invariants (Human Primacy, Constraint Supremacy, Co-Evolution).
- Formal contributions (indices, state machines, stability models).
- Evaluation method (SI trajectories, HSS/SCP learning curves).

- Open problems (Chapter 18).

Used to seed the field and standardize terminology externally.

15.5.3 Policy Brief Template

Required structure:

- Regulatory mapping (jurisdiction + sector).
- Constraint Vector translation into policy language.
- RB placements and accountability model.
- Audit and incident doctrine.

Supports engagement with regulators and professional bodies.

15.5.4 Diagram Pack & Visual Canon

Required elements:

- Four-Layer Stack (baseline + sector overlays).
- Autonomy Bands / RB boundary diagrams.
- SI/HSS/SCP flow and feedback loops.
- Recovery Ladder and safe-mode diagrams.

v0.5.2's point stands: diagrams are not decoration; they are shared mental models that prevent interpretive drift.

15.5.5 Sector-Specific Mini-Briefs

Mini-briefs are **implementation accelerators** for Phase 1–2 rollout. They must include:

- sector mapping of the four layers,
- 3–5 reference workflows,
- sector RB + autonomy stance,
- sector SI/HSS/SCP interpretation,
- tailored adoption roadmap,
- explicit sector failure modes and guardrails.

Alignment requirements: no custom metaphors that contradict ontology; explicit links to case studies and governance appendices.

Chapter Close — Implementation Integrity Check

If Chapter 15 is followed as written, then:

- Adoption proceeds via **earned autonomy**, not enthusiasm.
 - Culture, incentives, and literacy are treated as **control surfaces**, not soft extras.
 - External communications remain tethered to the same ontology, architecture, and metrics, preventing fragmentation as the framework scales.
-

Chapter 16 — Symbiosis Literacy, Roles, Hiring, and Certification (v0.9)

Positioning.

v0.8 supplies the “pilot/air-traffic-control” spine: operator tiers, HSS-as-license framing, and measurable behavioral rubrics.

v0.5.2 supplies the social operating model: core roles, hiring logic, certification components, and adoption patterns.

This merged chapter makes Symbiosis **teachably real**: a repeatable competency system rather than a philosophy or toolkit.

16.0 Purpose

Symbiotic systems fail not because models are weak but because **humans are unprepared to operate autonomy safely**. This chapter is a countermeasure to the classic anti-patterns: shallow prompting, oversight theater, governance bypassing, and drift-by-ignorance.

The deliverables here are:

1. a structured literacy program,
 2. clear role segmentation,
 3. hiring rubrics grounded in HSS/SCP,
 4. a certification ladder tied to Autonomy Bands,
 5. organizational adoption patterns.
-

16.1 Symbiosis Literacy Program (Curriculum Outline)

The literacy program is mandatory for any role operating inside or governing the loop. v0.5.2 already positions it as prerequisite for critical roles.

I’m formalizing a three-tier curriculum to align with the certification ladder.

Level 1 — Novice / Orientation (all staff)

Goal: eliminate “AI as magic” mental models.

Modules

- Symbiosis overview: four layers, invariants, regret boundary.
- Intent Surfaces 101: goal vs. constraint vs. definition of done.
- Autonomy Bands: what they are, why they exist, what Band 0 means.
- “Shadow AI” risks and why governance is architecture, not policy decks.

Assessment

- short scenario quiz (identify intent/constraints, pick autonomy band).
 - safe-use pledge aligned with Non-Punitive Covenant.
-

Level 2 — Practitioner (baseline for operators)

Goal: create reliable Band 0–1 pilots.

Modules

- Intent Interface mastery: structured briefs, negative constraints, budget framing.
- Feedback as training signal: how to label failures and provide corrective rationale.
- Governance literacy: RB crossings, escalation protocols, constraint hygiene.
- Memory & lineage hygiene: session vs project vs canonical capture.
- Reading SI dashboards; basic drift detection.

Assessment

- simulated workflow practicum (operate a TG with injected failures).
- peer-reviewed override quality.

Metric expectation

- HSS dimensions trending toward “Structured / Teaching / Steward / Curator.”
-

Level 3 — Advanced / Stewardship (for L2 pilots, architects, governance)

Goal: safe Band 2–3 operation and system tuning.

Modules

- Task-graph thinking: decomposition, dependency control, recovery hooks.
- Economic Circuit reasoning: cost–utility tradeoffs, model tiering.
- Stability dynamics: drift/divergence/resonance/entanglement flags.

- Incident response + post-mortem craft.
- SCP-aware supervision: attention budgeting and concurrency throttling.

Assessment

- live-pilot evaluation in production workflows.
 - participation in at least one incident or redesign review.
-

16.2 Core Roles in Hybrid Intelligence

Role clarity is a hard invariant in v0.8: without it you get “everyone does everything,” and no one is accountable.

16.2.1 Symbiote Operator (Pilot)

Frontline human inside the loop generating intent, reviewing outputs, and authorizing execution.

Primary responsibility: Intent Fidelity under constraints.

Key metrics: HSS (esp. Intent Clarity and Feedback Quality) + SCP load fit.

Anti-patterns to train against

- Passive reviewer (approval without reading).
 - Constraint amnesia (forgetting legal/safety/economic bounds).
 - Silent rejection (no learning signal).
-

16.2.2 Hybrid Intelligence Architect (Engineer)

Designs and refactors workflows: TGs, routing rules, agent clusters, and memory policies.

Primary responsibility: System Efficiency and Stability.

Key metric: SI of designed workflows; SI decay implies refactor duty.

Anti-patterns

- “capability-first” workflow design ignoring regret boundaries.
 - over-complex graphs that exceed available SCP bandwidth.
-

16.2.3 Symbiosis Governance Lead (Controller)

Owns regret boundary definition, constraint vectors, autonomy regimes, and audits.

Primary responsibility: Safety, Compliance, and Trust Calibration.

Key authorities: system veto, band demotion, policy revision.

Anti-patterns

- governance-as-paperwork.
 - delayed constraint updates after incidents.
-

16.3 Hiring Rubrics for Hybrid-Intelligence Roles

v0.5.2 sketches distinct rubrics by role; I'm tightening them and aligning with HSS/SCP dimensions.

16.3.1 Symbiote Operator Hiring

Look for

- high baseline intent clarity and structured thinking,
- evidence of reflective practice and comfort with critique,
- situational risk awareness.

Assess via

- work-sample operation in simulated TGs.
 - "explain the failure" interview segments.
-

16.3.2 Hybrid Intelligence Architect Hiring

v0.5.2 core dimensions: systems thinking, orchestration literacy, governance sensitivity.

Assess via

- design exercise: "sketch a symbiotic workflow for X under Y constraints."
- critique exercise on flawed graphs from governance perspective.

Decision rule

- architects must demonstrate ability to trade capability for stability.
-

16.3.3 Governance Lead Hiring

v0.5.2 dimensions: risk/constraint reasoning, metric interpretation, ethical judgment.

Assess via

- metric dashboard interpretation prompt: “what concerns you, what would you do?”
- cross-functional scenario panels and failure-case analysis.

16.4 “Symbiote Operator” Certification Path

Certification is how the organization signals HSS development, allocates autonomy, and establishes a career track.

v0.8 frames this explicitly as a **pilot’s license** rather than a performance review.

16.4.1 Certification Levels

v0.5.2 defines three levels; v0.8 defines three tiers. We unify naming:

Framework label	v0.5.2 level	v0.8 tier	Autonomy rights (default)
L1 Cadet / Foundation Operator	Level I	L1 Cadet	Band 0 (mid-stakes), Band 0 only high-stakes.
L2 Pilot / Advanced Practitioner	Level II	L2 Pilot	Band 1–2 mid-stakes, Band 1 high-stakes.
L3 Commander / Lead Operator	Level III	L3 Commander	Band 3 only where SI stability and sector policy allow.

16.4.2 Certification Components

For each level, v0.5.2 requires three evidence types.

1. **Training completion** (literacy curriculum).
2. **Assessment**
 - simulated workflow work-samples,
 - observed behavior in pilots,
 - peer/manager feedback.

3. **Metric evidence**

- local SI stability on operated workflows,
- HSS estimates with privacy safeguards.

Critical guardrail: decisions are never based on HSS numerics alone; qualitative governance judgment is mandatory, with recertification every 18–24 months.

16.4.3 Linking Certification to Autonomy

Explicit linkage prevents the most dangerous mismatch: high autonomy with low HSS/SCP. Autonomy elevation is therefore a **governance act** backed by evidence and stability gates, not a manager preference.

16.5 Ethical Firewall for Human Metrics

v0.8's warning is load-bearing: if HSS/SCP are tied to compensation or firing, the entire measurement system becomes gameable and SI corrupts.

Therefore:

- HSS/SCP live in a safety database and are not visible to HR/line management.
 - Near-miss reporting is rewarded; catching failures is treated as excellence.
-

16.6 Organizational Change & Adoption Patterns

v0.5.2 identifies three valid patterns, each safe if roles + literacy + metrics are installed.

1. **Domain-First Adoption:** fast value, risk of fragmentation.
2. **Governance-First Adoption:** strong safety, risk of perceived drag.
3. **Hybrid Pilot Cluster:** cross-domain learning, higher early complexity.

Rule: whichever pattern you pick, install named role ownership and literacy gating before you expand autonomy.

Chapter Close — What This Enables

With Chapter 16 in place:

- Symbiosis becomes **operable as a profession** (pilots, engineers, controllers).
- Autonomy scales only when humans have proven capacity and literacy.
- Metrics remain truthful because incentives are aligned with safety, not speed.

- Sector playbooks (Chapter 17) can be instantiated without reinventing roles or training.

Below is a fully integrated Chapter 17 draft (v0.8 skeleton, v0.5.2 flesh, plus added “muscle” items). Keeping it linear, practical, and defensibility-oriented. After the chapter text, included targeted recommendations to strengthen the overall framework via this chapter.

Chapter 17 — Sector Playbooks and Policy Interfaces (Integrated Draft)

17.0 Purpose and Use

The Symbiosis Framework is universal in architecture but specific in configuration. Each sector requires a tailored “playbook” that maps:

1. its risk posture into **Risk Classes**, **Autonomy Bands**, and **Governance Kernel** settings;
2. its workflows into the **Four-Layer Architecture**; and
3. its regulators into a stable **policy interface**.

This chapter provides a standard playbook schema, then instantiates it across key archetypes. The goal is to make sector deployment **repeatable, auditable, and regulator-legible**, while preserving the same underlying symbiotic invariants (human primacy, constraint supremacy, SI-driven control).

17.0.1 Standard Sector Playbook Schema (new “muscle”)

Every sector playbook should be written in the same structure to ensure comparability and to support Appendix K (compliance matrices).

Playbook Template

1. **Sector Context & Risk Posture**
 - irreversibility, harm type, tolerance, principal stakeholders
2. **Architecture Configuration**
 - Human Layer roles & non-delegables
 - Engine stance (speed vs explainability, memory rules)
 - Agentic cluster restrictions + contract defaults
 - Governance kernel + supervisory logic
3. **Risk Classes → Autonomy Bands → Gates**
 - explicit mapping table
4. **Priority Workflows**

- initial pilots + scale targets
 - 5. **SI / HSS / SCP Tuning**
 - which SI dimensions dominate
 - target HSS skills and SCP circuit-breaker thresholds
 - 6. **Adoption Roadmap**
 - phased deployment + co-evolution milestones
 - 7. **Policy / Regulator Interface**
 - standards alignment
 - evidence packet + audit outputs
 - 8. **Common Gaps / Failure Modes**
 - recurring anti-patterns to proactively govern
-

17.1 Healthcare Playbook (Safety-Critical Archetype)

17.1.1 Sector Context & Risk Posture

Healthcare is safety-critical, high-regulation, and high-irreversibility. Failures can cause physical harm, legal breach, or loss of trust. Default stance: **Human Primacy is absolute**; AI cannot be the final decision author for clinical actions.

17.1.2 Architecture Configuration

- **Human Layer**
 - Operators: clinicians, nurses, triage staff (Symbiote Operators).
 - Non-delegables: diagnosis, prescribing, treatment selection; humans retain liability.
- **Symbiosis Engine**
 - Tuned for explainability/traceability over speed.
 - Memory: strict session isolation to prevent cross-patient leakage.
- **Agentic Layer**
 - Restricted to read-only discovery, summarization, and drafting; no write-access to EHR without countersignature.
- **Governance Layer**
 - Risk classes explicitly defined for clinical vs triage vs administrative tasks.
 - Mandatory governance bodies:
 - **Clinical Symbiosis Governance Committee (CSGC)** to review SI trends, authorize regime changes, approve new use cases.
 - Incident Review Board for symbiosis-related events.

17.1.3 Risk Classes & Autonomy Gating

Risk Class	Definition	Autonomy Band	Gate
------------	------------	---------------	------

A (Clinical)	Diagnosis, prescribing, treatment plans	Band 0 (Advisory)	Mandatory licensed clinician sign-off
B (Triage)	urgency categorization, history summarization	Band 1 (Internal)	Nurse/clinician confirm
C (Admin)	billing codes, reminders, documentation	Band 2 (Conditional)	Auto-execute only above high confidence threshold

17.1.4 Priority Workflows

1. Clinical summarization & differential suggestion (advisory).
2. Documentation drafting (physician-in-the-loop).
3. Triage support.
4. Admin optimizations.

17.1.5 SI / HSS / SCP Tuning

- **SI weighting emphasis:**
 - Alignment with constraints & guidelines (A),
 - Traceability/explainability (G),
 - Stability across subgroups (S).
- **HSS emphasis for clinicians:**
 - Intent clarity (C), override rationale (F), governance literacy (A), reflective practice (R).
- **SCP enforcement:**
 - CON: cap concurrent high-acuity cases per operator.
 - CL: match complexity to seniority/training.
 - TH: short-horizon for ED, longer for chronic-care teams.
 - Engine enforces circuit breakers via attention budget throttling.

17.1.6 Adoption Roadmap

1. **Domain scoping:** begin with one service line.

2. **Governed pilot:** tight autonomy, SI + incident logging day one.
3. **Expansion:** add workflows as SI stabilizes and HSS improves.
4. **Training integration:** residency/CPD embeds symbiosis literacy.

17.1.7 Policy / Regulator Interface

Provide regulators with:

- four-layer architecture mapping,
 - risk class definitions and autonomy regimes,
 - SI trend summaries + incident logs,
 - governance committee charters and audit chains.
-

17.2 Finance Playbook (Audit-Critical Archetype)

17.2.1 Sector Context & Risk Posture

Finance is risk-dense and regulation-heavy: fiduciary duty, consumer protection, AML/CTF, market integrity. Failures tend to be **financially harmful and audit-sensitive**.

17.2.2 Architecture Configuration

- **Human Layer**
 - Roles: analysts, portfolio managers, compliance officers.
 - Non-delegables: trade execution, legal commitments, client suitability decisions.
- **Symbiosis Engine**
 - Tuned for auditability, provenance, cost control.
 - “Adoption Bridges” for low-risk utility tasks to keep users in governed path.
- **Agentic Layer**
 - Contracts must declare “NO_TRADE_EXECUTION” unless explicitly allowed by governance.
 - Strong requirement for cite-sources and uncertainty metadata.
- **Governance Layer**
 - Immutable log and hashed integrity chain for forensic reconstruction.

17.2.3 Risk Classes & Autonomy Gating (finance-specific)

Risk Class	Definition	Band	Gate
------------	------------	------	------

A (Capital-Critical)	investment decisions, client advice, regulatory filings	Band 0–1	multi-human review + compliance sign-off
B (Operational)	research synthesis, risk modeling, internal reports	Band 1–2	click-to-confirm or conditional auto-approval
C (Admin/Support)	scheduling, document formatting, low-risk comms	Band 2	auto within budget

Anchored to general risk/autonomy model.

17.2.4 Priority Workflows

1. Research aggregation & macro/sector briefs.
2. Risk report drafting (advisory).
3. Compliance pre-checks (KYC/AML pattern surfacing).
4. Back-office automation.

17.2.5 SI / HSS / SCP Tuning

- **SI emphasis:** G (auditability), A (policy adherence), Q (error reduction), with explicit cost-efficiency weighting.
- **HSS targets:** intent structuring around financial hypotheses, explicit uncertainty statements, governance literacy.
- **SCP:** prioritize CON throttling for high-volatility periods to avoid oversight theater.

17.2.6 Adoption Roadmap

- Pilot in **research and internal risk** first; expand to client-facing after SI stability.
- Move to Band 2 only for low-risk support workflows.

17.2.7 Policy / Regulator Interface

- Map playbook risk classes to relevant regimes (e.g., EU AI Act high/limited risk categories).
 - Provide integrity-chain logs, model validation summaries, and autonomy gate evidence.
-

17.3 Education & Public Sector Playbook (Civic-Critical Archetype)

17.3.1 Sector Context & Risk Posture

Education/public sector decisions affect legitimacy, equity, minors’ data, and civil rights. Harm is often **societal and long-tail**.

17.3.2 Architecture Configuration

- **Human Layer**
 - Roles: educators, administrators, case workers, policy leads.
 - Non-delegables: grading decisions without recourse, discipline escalation, benefits denial.
- **Engine**
 - Emphasize transparency, data minimization, and consent capture.
- **Agentic**
 - Prohibit covert monitoring or high-stakes classification without human approval.
- **Governance**
 - Consent models and due-process requirements are first-class hard constraints.

17.3.3 Risk Classes & Autonomy

Class	Definition	Band	Gate
A (Rights-Critical)	discipline, welfare decisions, eligibility determinations	Band 0	mandatory human + appealability
B (Pedagogical)	lesson planning, feedback drafts	Band 1–2	teacher confirm

C (Admin)	scheduling, communications, translation	Band 2	auto within budget
-----------	---	--------	--------------------

17.3.4 SI / HSS / SCP Tuning

- SI emphasizes fairness/stability across demographics (S) and accountability (G).
- HSS focuses on educators’ ability to use AI for differentiation, not replacement.
- SCP explicitly guards against dashboard overload.

17.3.5 Common Gaps

- Over-reliance on AI for grading or discipline without transparent recourse.
- Weak parental/student consent.
- No SCP thinking; teacher overload from too many signals.

17.4 Creative & Media Playbook (Ambiguity-Critical Archetype)

17.4.1 Sector Context & Risk Posture

Creative/media work is high-ambiguity and reputationally sensitive. Harm is rarely physical but can be brand-fatal or culturally toxic.

17.4.2 Architecture Configuration

- **Human Layer:** editors, writers, designers as Symbiote Operators.
- **Engine:** emphasize iterative hybrid loops, provenance tagging.
- **Agentic:** allow higher autonomy for drafting + transformation, but not publishing.
- **Governance:** disclosure rules, originality constraints, and bias checks are mandatory.

17.4.3 Risk Classes & Autonomy

Class	Definition	Band	Gate
A (Brand-Critical)	public releases, political/regulated ads	Band 0–1	editorial + legal sign-off

B (Editorial)	drafts, rewrites, campaign variants	Band 1–2	editor confirm
C (Internal)	ideation, style exploration	Band 2–3	auto within constraints

17.4.4 Adoption Roadmap

Start with internal ideation/drafts; move to repurposing under oversight; then allow conditional auto for low-risk outputs with explicit SI thresholds for high-profile content.

17.4.5 Policy Interface

For regulated media domains, share provenance and transparency frameworks with regulators and participate in synthetic-media standards.

17.5 Defense / High-Stakes Playbook (Extreme-Risk Archetype)

17.5.1 Sector Context & Risk Posture

Defense/high-stakes domains include life-and-death decisions, geopolitical consequences, secrecy, and hard legal constraints. Autonomy must be **extremely conservative** and red-team-validated.

17.5.2 Architecture Configuration

- **Human Layer:** analysts, commanders, planners; non-delegables for lethal/liberty-affecting actions.
- **Engine:** strict ROE/legal constraints embedded as hard vectors.
- **Agentic:** discovery/fusion, simulation, COA drafting; “execution” only in reversible, non-kinetic domains.
- **Governance:** multi-layer oversight including independent review. ;

17.5.3 Priority Workflows & Autonomy Stance

- Intelligence analysis, wargaming, logistics, cyber defense.
- Use-of-force decisions fixed at Band 0 advisory only.

17.5.4 Common Gaps

- Autonomy creep due to speed/edge pressures.
 - Classification preventing governance scrutiny.
 - Tactical-success metrics crowding out ethical stability metrics.
-

17.6 Regulatory Integration and Standards Alignment

17.6.1 Cross-Sector Standards Mapping

Governance should be written so it can be “ported” into standards compliance:

- **Risk Classes** map to categories in emerging regulation (e.g., AI-risk brackets).
- **Constraint vectors** implement privacy/data-residency obligations.
- **Immutable Log** + integrity chains provide audit parity with safety-critical software.

17.6.2 The Regulator Evidence Packet (new “muscle”)

For any sector, the minimum evidence packet should include:

1. **Architecture dossier** (layer mapping, agent contracts registry, autonomy regimes).
 2. **Governance kernel** (policy versions, escalation rules, non-delegables).
 3. **Metric dossier** (SI baselines, HSS/SCP ethical firewall, drift/incident trends).
 4. **Validation dossier** (red-team results, domain benchmarks, subgroup stability checks).
 5. **Audit exports** (integrity-chain logs enabling replay).
-

Recommendations to Strengthen the Framework via Chapter 17

1. **Lock the Playbook Schema as an invariant.**
Treat the template in 17.0.1 as mandatory for any additional sectors. This ensures your “new field” has a standards backbone rather than a collection of examples.
2. **Add 2–3 more archetypes (optional but high value).**
You already cover Safety, Audit, Ambiguity, Cost. Consider adding:
 - **Infrastructure / Utilities (Resilience-Critical)**
 - **Law / Legal Services (Liability-Critical)**
 - **Scientific R&D (Novelty-Critical)**These expand the field without changing the skeleton.

3. **Create a “Playbook-to-Appendix-K compilation flow.”**

Each playbook should auto-produce:

- risk/autonomy matrices,
- constraint catalogs,
- sector-specific incident taxonomies.

This keeps Appendix K authoritative and removes duplication drift.

4. **Deepen the policy interface section into a reusable standard.**

The evidence packet in 17.6.2 should become an explicit **Regulatory Submission Standard**: a named artifact regulators can request.

5. **Introduce sector-specific threat models.**

Within each playbook, add a short threat model: data misuse, autonomy creep vectors, reputational cascades, etc., and map each to governance controls. This is a key “defensibility” upgrade.

6. **Tie playbooks to co-evolution phases.**

In each sector, identify Phase-gates (v0.8 co-evolution Phases 1–4) that must be passed before Band increases.

This makes scaling logic regulator-friendly.

Interstitial — Appendix K Build Notes (v0.9)

K-BN.0 Purpose

Appendix K is the mechanism that turns Symbiosis from an abstract governance philosophy into **jurisdiction- and sector-specific policy-as-code**. Its job is to:

1. map **real-world regulatory requirements** into Constraint Vectors, Risk Classes, and Autonomy gates,
2. bind those constraints to the Engine’s pre-flight and runtime enforcement, and
3. provide a regulator-legible audit surface proving that Symbiosis is compatible with law and domain ethics.

Appendix K is therefore **normative**: organizations must select the right sector matrix and load it into the Governance Kernel / Policy Engine.

K-BN.1 Inputs and Compilation Flow

Appendix K is compiled from three upstream objects:

1. **Sector Playbook (Chapter 17).**
Provides sector risk posture, non-delegables, prioritized workflows, default autonomy stance, and common gaps. This is the “why.”
2. **Jurisdictional and Sector Constraints (Governance Layer).**
Governance must map workflows and data to relevant jurisdictions and generate **jurisdiction-specific Constraint Vectors**, enforcing data localization and access restrictions.
It also differentiates “Low / Mid / High stakes” tasks and aligns allowed Autonomy Bands accordingly.
3. **Regulator / Standards Corpus.**
The actual law and professional standards (e.g., HIPAA, GDPR, SEC rules, CMMC), plus internal institutional policy.

Compilation rule:

Appendix K must not introduce new ontology. All terms are interpreted through Appendix A and core chapters. (K is a mapping layer, not a vocabulary layer.)

K-BN.2 Canonical Structure of a Sector Compliance Matrix

Each sector matrix in Appendix K follows the same schema (v0.8 already models this explicitly).

Minimum columns (normative):

1. **Regulatory Requirement**
 - law/standard name and relevant section(s).
2. **Operational Interpretation**
 - what the requirement means in workflow terms.
3. **Symbiosis Implementation**
 - the precise encoding as:
 - Constraint Vector items (C_legal / C_safety / C_econ),
 - Risk Class assignment rules,
 - Autonomy Band ceilings and checkpoint placement,
 - Memory tier and retention requirements,
 - Audit/lineage minimum event set.
4. **Critical Failure Mode**
 - the sector-specific “don’t let this happen” pattern that motivated the constraint.
5. **Evidence Artifact**
 - what auditors/regulators can request to verify compliance (lineage exports, incident packages, SI slices, etc.).

Optional but recommended columns:

- **RB proximity classification** (below / at / above RB).
 - **Admissible agent contracts** (which MCP tool scopes are legal).
 - **Stability vital-sign gates** (e.g., drift red-band blocks AB promotions).
-

K-BN.3 Sector Coverage and Boundary Hygiene

v0.5.2 already defines matrices for healthcare, finance, education, and manufacturing.
v0.8 adds enterprise and defense as primary coverage domains.

Rule:

If a sector is not explicitly covered, you must either:

- **instantiate a new matrix using the K schema**, or
- **conservatively adopt the nearest higher-regret matrix** until a new one is ratified.

Avoid “matrix drift” caused by ad-hoc local exceptions. Divergence must be blessed into a separate matrix with its own P_v version number.

K-BN.4 Linking Risk Classes to Autonomy Bands (Hard Gate)

Risk classification is the bridge between the regulatory world and runtime autonomy.
Governance must map risk classes to allowed autonomy bands using: risk class, SI history, HSS/SCP, and regulatory confidence intervals.

Normative default stance (unless sector matrix overrides):

- **High-stakes:** AB0–AB1 only; AB2–AB3 require explicit GK approval and stable SI/HSS/SCP evidence.
- **Mid-stakes:** AB1–AB2 depending on stability.
- **Low-stakes:** AB2–AB3 admissible with lighter oversight.

This mapping is encoded as a machine-readable rule set in the Policy Engine.

K-BN.5 Policy Engine Encoding Requirements

Each matrix must compile into enforceable policy. At minimum, each K entry must be representable as:

1. **Constraint Vector item(s)**
Example types: NO_EGRESS, REQUIRE_CITATIONS, MAX_SPEND,
NO_PUBLIC_ENDPOINT, RETENTION_YEARS, NO_AUTONOMY_BAND_3.

2. **Autonomy Gate**

- Band ceilings by intent type or workflow class.
- Mandatory RB checkpoints for irreversible actions.

3. **Memory / Lineage Rule**

- tier placement, retention period, and access logging.

4. **Incident Trigger Rule**

- which failures auto-escalate to S3/S4 per Chapter 13.

Invariant:

The Engine must reject any TG node whose simulated execution violates active K constraints, before budget unlock or external action. (This follows the v0.8 contract/ACK-REJECT pre-flight model.)

K-BN.6 Versioning and Co-Evolution of K

Appendix K is a living artifact.

Version objects:

- **P_v**: policy version for the sector matrix.
- **C₀ → C_t lineage**: constraint vector evolution over time.

Update cadence:

- **Tactical loop (weeks–months)**: update for recurring incidents, drift patterns, or operational mismatches.
- **Strategic loop (months–years)**: update for law/regulator changes.

Change control rule:

- K changes are a **slow-core update** (per Chapter 11). They must not be hot-patched without governance sign-off, because they shift RB and autonomy ceilings.
-

K-BN.7 Output Artifacts (What Appendices K and N Must Share)

Appendix K should export three machine-readable artifacts for implementers:

1. **Constraint Catalog (C-CAT)**

- full list of constraint codes + descriptions + severity.

2. **Risk-Autonomy Matrix (RAM-K)**

- mapping of risk class → AB ceiling → checkpoint rules.
3. **Evidence & Audit Map (EAM-K)**

- what logs / lineage slices / incident packages are required for each requirement.

Appendix N pilots then reference these artifacts directly to prove conformance and avoid narrative duplication.

K-BN.8 What “Done” Looks Like

Appendix K is complete for a sector when:

- each major regulator requirement has a corresponding constraint encoding,
 - every non-delegable domain is mapped to AB0 ceilings,
 - auditability is satisfied via immutable lineage exports,
 - incident gates align with Chapter 13 recovery doctrine, and
 - the sector matrix can be loaded into the Policy Engine without manual translation.
-

Chapter 18 — Future-State Research Tracks & Programs (v0.4)

This chapter specifies the forward research agenda implied by the Symbiosis Framework. Where prior chapters define the architecture, primitives, metrics, and governance needed to build hybrid human–AI systems today, Chapter 18 outlines the research programs required to make symbiosis a mature scientific and institutional discipline. These tracks are not speculative flourishes; they are concrete trajectories seeded directly by the ontology, four-layer design, and the SI/HSS/SCP control logic already established.

The programs are organized as modular monographs (M1–M5). Each monograph can be pursued independently, but they are mutually reinforcing. Together they form a long-horizon “research stack” partnering systems theory, control, cognitive science, and governance engineering.

18.0 Orientation: Why a Future-State Agenda?

The Symbiosis Framework introduces three non-trivial claims:

1. **Hybrid intelligence is a first-class system type**
Neither tool-usage nor automation captures what happens when humans and agents operate in coupled loops.

2. **Governance is not a policy layer—it is an architectural layer**
Constraint supremacy and autonomy regimes must be designed, enforced, and audited as system properties.
3. **Humans are not static users—they co-evolve as operators**
HSS and SCP are measurable, improvable capacities that shape safe autonomy.

These claims force a research agenda that goes beyond current AI evaluation (accuracy, latency, cost), and beyond generic “AI ethics.” The programs below are the minimal set required to ground symbiosis in formal science and reliable institutional practice.

18.1 M1: Hybrid Intelligence Systems Theory

Objective. Develop a general systems theory for human–AI symbiotic ecologies: what they are, how they behave, what states they can occupy, and what stability means for them.

Key questions.

- **System identity and boundary conditions:**
When does a tool+user become a hybrid cognitive system? What are the necessary and sufficient conditions for “symbiosis” rather than mere assistance?
- **State variables and observables:**
SI, HSS, SCP are early candidates; what additional observables exist (trust stability, autonomy utilization, override elasticity, context drift rate)?
- **Phase space and regimes:**
Formalize symbiosis phases (bootstrap → operational → co-evolutionary → high-autonomy equilibrium) and identify transition thresholds.
- **Conservation laws / invariants:**
Human primacy, constraint supremacy, and layered autonomy are proposed invariants. Determine which are truly invariant vs. context-dependent.

Research outputs.

- A formal “**Hybrid System Definition**” aligned to the ontology.
- **Canonical state-space models** for symbiotic workflows.
- A **stability and safety lemma library** for hybrid systems (analogous to control-system stability criteria).

Why it matters.

Without systems theory, organizations will keep shipping one-off “agent apps” without reliable understanding of failure modes. M1 makes symbiosis analyzable, not anecdotal.

18.2 M2: Autonomous Governance Mechanisms

Objective. Engineer governance subsystems that can enforce constraints, gate autonomy, and escalate risk in real time, with minimal human overhead and maximal auditability.

Key questions.

- **Constraint compilation:**
How do you translate legal / ethical / organizational policies into machine-enforceable constraint vectors and autonomy regimes without semantic loss?
- **Governance control loops:**
SI is a control signal. What are the correct controllers (PID-like, Bayesian, regime-switching, human-in-the-loop supervisory control)?
- **Escalation logic and arbitration:**
What formal triggers cause autonomy band reduction, pause, or human override? How should multi-agent conflicts be adjudicated?
- **Governance latency vs. safety:**
How much real-time governance can be applied before throughput collapses? What are optimal “governance budgets”?

Research outputs.

- A **governance compiler**: policy → constraint vector → enforceable runtime checks.
- **Formal autonomy gating algorithms** linked to SI/HSS/SCP.
- A **governance telemetry schema** standardizing audit traces across vendors.

Why it matters.

Most failures in deployed AI are governance failures: silent autonomy creep, weak escalation, or non-auditable policy interpretation. M2 makes governance executable and testable.

18.3 M3: Co-Evolution Dynamics (Formal Modeling)

Objective. Move co-evolution from metaphor to math: model how humans and AI adapt together over time, and how that adaptation can be steered.

Key questions.

- **Coupled learning curves:**
How do HSS and SI trajectories interact? Under what conditions does improving one degrade the other?
- **Positive and negative feedback loops:**
Which loops yield stable co-growth (resonance) vs. runaway brittleness (entanglement)?
- **Skill acquisition mechanics:**
What interventions (training, UI scaffolds, autonomy pacing) maximally raise HSS and expand SCP safely?

- **Longitudinal risk prediction:**
Can we forecast impending drift or trust collapse from early trajectory signatures?

Research outputs.

- **Differential / stochastic models** of SI–HSS–SCP coupling.
- **Intervention playbooks** with predicted effect sizes.
- A **co-evolution simulation environment** for pre-deployment stress testing.

Why it matters.

Scaling symbiosis without modeling co-evolution invites brittle dependence. M3 makes “humans getting better with AI” a reliably engineered outcome.

18.4 M4: Distributed Intent Modeling

Objective. Build methods to represent *collective* human intent—across teams, organizations, and institutions—and to manage conflicts, updates, and lineage over time.

Key questions.

- **Intent aggregation:**
How do multiple intent surfaces merge without erasing minority constraints or values?
- **Temporal intent drift:**
How should intent surfaces evolve when goals shift mid-project? What versioning and rollback rules preserve accountability?
- **Constraint-intent entanglement:**
When constraints are negotiated socially (e.g., regulators vs. operators), how is that captured as formal vectors?
- **Representation formats:**
What is the minimal sufficient structure for intent surfaces to be machine-actionable and human-auditable?

Research outputs.

- A **distributed intent schema** (graph or field-based) with lineage metadata.
- **Negotiation and reconciliation protocols** for intent conflict.
- **Tooling for intent “diffs,” merges, and approvals** analogous to Git, but governance-aware.

Why it matters.

Real deployments are not single-user chats. They are contested, multi-stakeholder systems. M4 makes symbiosis scalable beyond the individual operator.

18.5 M5: Machine Sociality & Norm Internalization

Objective. Study and engineer how agent ecosystems behave socially: forming norms, respecting contracts, and internalizing governance constraints without constant external policing.

Key questions.

- **Agent-agent trust and reputation:**
How should capability indices and reliability histories shape delegation and parallelism?
- **Norm learning vs. rule following:**
What can be safely internalized by agents (soft norms) versus enforced externally (hard constraints)?
- **Social failure modes:**
Collusion, echoing, runaway consensus, adversarial role-playing across agents—how do these emerge and how are they detected?
- **Institutional alignment:**
How do agent norms stay synchronized with evolving human norms and laws?

Research outputs.

- **Formal models of agent social dynamics** and contract adherence.
- **Norm-internalization mechanisms** with verification boundaries.
- A **taxonomy of machine-social failure modes** parallel to human social failures.

Why it matters.

As poly-agent systems scale, their emergent social behavior becomes the frontier risk surface. M5 is necessary for safe high-concurrency autonomy.

18.6 Open Problems and Monograph Trajectory

The monographs above define a long-horizon research stack. Several cross-cutting open problems should be treated as priority areas:

1. **Metric completeness and identifiability.**
Do SI/HSS/SCP span the state space adequately, or are there critical latent variables (e.g., institutional brittleness, incentive distortion) requiring new metrics?
2. **Universality vs. sector specificity.**
Which components of the framework are universal (ontology, autonomy bands) and which need sector-bound extensions (healthcare, finance, defense)?
3. **Verification of hybrid control.**
Formal verification for agentic systems is hard. Hybrid systems add humans. What

does “verification” mean here, and how do we define acceptable proof?

4. **Governance composability.**

How do governance layers from different jurisdictions (or firms) compose without contradiction?

5. **Hardware and energy coupling.**

As autonomy rises, compute and energy become governance concerns. How do orchestration policies incorporate real-world resource constraints?

Monograph pathway.

A credible execution order is:

- **M1 + M2 first** (foundations: theory + executable governance)
- **M3 next** (understanding and steering adaptation)
- **M4 + M5 in parallel** (scaling to institutions and agent societies)

Each monograph should yield both a rigorous academic contribution and an implementation-ready “design kit” compatible with the framework’s ontology and governance controls.

Closing Note

This agenda positions symbiosis as more than a methodology: it is a candidate *discipline* with its own objects (intent surfaces, constraint vectors, autonomy regimes, hybrid loops), metrics (SI/HSS/SCP), and laws (co-evolutionary stability, constraint supremacy). Chapter 18 is therefore the bridge from v0.4 practice to v1.0 science—and to whatever institutional forms hybrid intelligence will require next.

Epilogue — The Silicon + Carbon Manifesto (Expanded v0.9)

Positioning.

Both drafts close not with another technical chapter, but with a normative statement of intent: a manifesto that defines what Symbiosis is *for*, what it rejects, and what institutional future it is trying to steer toward. v0.8 places this as an expanded epilogue.

v0.5.2 provides the E.1–E.7 structure; preserve that spine and modernize phrasing to fit the engineering-standard voice.

E.1 Positioning and Rejections

We stand at the end of “prompting” and the beginning of architecture. Unstructured AI adoption (“Shadow AI”) is already creating systemic risk: hallucinated context driving decisions, privacy leakage through informal tools, and the false comfort of oversight theater. Symbiosis is not a feature set for those problems. It is a replacement paradigm.

Therefore, we reject:

1. **Naïve automation framing.**
Intelligence is not a conveyor belt to be sped up until humans fall off. Wherever meaning, regret, or legitimacy are at stake, hybrid cognition must remain human-governed.
 2. **Tool maximalism.**
Adding more models, more agents, or more compute without governance is not progress. It is a scale-up of liability.
 3. **Rogue autonomy.**
Any system that crosses regret boundaries without accountable authorization is not symbiotic; it is unsafe by definition.
 4. **De-skilling as destiny.**
We reject the narrative that advanced AI inevitably hollows human capability. Skill outcomes are architectural choices, and Symbiosis makes those choices explicit.
-

E.2 Ontological Commitments

We commit to a stable vocabulary and a stable substrate for this field.

1. **Hybrid intelligence is a system type.**
It is not a user plus a tool; it is a coupled cognitive ecology with its own states, metrics, and failure modes.
 2. **Intent, constraints, autonomy, and lineage are primitives.**
They are not “best practices.” They are the data types of safe hybrid systems.
 3. **Ontology coherence is non-negotiable.**
Sector-specific adaptations may extend the system but cannot contradict its core entities or invariants.
-

E.3 Architectural Commitments

We commit to the Four-Layer Stack as the decisive unit of integrity.

1. **Layer separation is safety.**
Control flows down; telemetry flows up. The Human Layer defines meaning; the Engine coordinates; the Agentic Layer executes; the Governance Layer constrains and audits.
 2. **Task graphs replace prompt chains.**
This is the boundary between improvisation and engineering.
 3. **Memory is tiered and governed.**
The system may remember only within explicitly permitted scopes, with lineage and retention rules enforced by policy.
 4. **Interoperability is a field requirement.**
Contracts, MCP-like protocols, and lineage schemas must be standardizable across vendors and institutions if Symbiosis is to become a real discipline.
-

E.4 Governance Commitments

We commit to governance as executable architecture.

1. **Constraint Supremacy.**
Legal, safety, and economic constraints are hard vectors, not suggestions.
 2. **Regret boundaries are inviolable.**
Actions above RB require explicit, accountable human authorization.
 3. **Auditability is a moral property.**
A hybrid decision that cannot explain its lineage is illegitimate, even if correct.
 4. **Bounded failure with deterministic recovery.**
Kill switches, safe modes, rollbacks, and incident doctrine are not optional add-ons; they are the definition of responsible co-agency.
-

E.5 Human Commitments

We commit to a human future that scales capability without collapsing agency.

1. **Human Primacy → Meaning Primacy.**
Humans remain the sole origin of sovereign intent and the final bearer of accountability.

2. **Operators are pilots, not passengers.**
The system must be designed to widen human horizons, not substitute them.
 3. **HSS and SCP are safety telemetry.**
They are used for skill development and autonomy pacing, firewalled from punitive management use.
 4. **Co-evolution is the default developmental path.**
We do not ship static human–AI relationships. We ship relationships that learn, stabilize, and mature.
-

E.6 Institutional and Temporal Commitments

We commit to a long-horizon institutional transition, not a short-horizon tool rush.

1. **Symbiosis is a presingularity standard.**
The future is arriving through gradual dependency. Our responsibility is to engineer dependency with safety and dignity now, not after catastrophe.
 2. **Governance must stay ahead of capability.**
Institutional learning and policy compilation are always lagging indicators; the architecture must compensate.
 3. **We build for jurisdictional reality.**
Sector playbooks and compliance matrices are the substrate for global adoption without fragmentation.
-

E.7 Ethical Commitments and Civilizational Pattern

We commit to hybrid intelligence as a civilizational project.

1. **Capability without legitimacy is harm.**
Symbiosis prioritizes legitimacy: human authorization, explainability, rights-preserving boundaries.
2. **Power must be contestable.**
Lineage and governance make decisions inspectable and reversible; this is the minimum for democratic compatibility.
3. **We choose the shape of co-agency.**
The future is branching. One branch is ungoverned autonomy and human atrophy. Another branch is coordinated hybrid intelligence where silicon expands the possible

and carbon remains sovereign over meaning. Symbiosis is a deliberate commitment to the second branch.

Epilogue Close

The Symbiosis Framework is not merely a way to deploy agents. It is the engineering standard for a new kind of collective cognition. If adopted faithfully—architecture-first, governance-as-code, autonomy earned through stability—it is capable of launching a field defined by mature co-agency rather than uncontrolled dependency.

Below is Appendix A (Symbiosis Ontology v1.1) in a merged, canonical form. Using the v0.8 namespace as authoritative, upgrading the axiom set from v0.5.2, and adding explicit reconciliation notes so terminology cannot drift in v0.9+.

Appendix A — Symbiosis Ontology v1.1 (Canonical Specification)

A.0 Purpose and Usage

This appendix fixes the **authoritative vocabulary** for the Symbiosis Framework. All chapters, diagrams, metrics (SI, HSS, SCP), governance models, agent contracts, and playbooks must use this ontology without drift. v1.1 supersedes v1.0 by integrating the **Economic Circuit / Budget dimension** and formalizing the **Regret Boundary state machine**.

A.1 Axioms (Irreducible Foundational Claims)

These axioms are non-negotiable invariants. Any extension of the framework must preserve them.

Axiom 1 — Human Primacy

Humans retain ultimate moral, strategic, and evaluative authority over all AI-mediated systems and decisions.

Axiom 2 — Asymmetric Agency

Synthetic agents possess **instrumental, bounded, derivative agency**; humans possess **sovereign, value-bearing agency**. Agents may act only inside human-defined intent and constraints.

Axiom 3 — Layered Autonomy

AI autonomy is not binary. It exists in graduated, governable **Autonomy Bands** governed by the **Regret Boundary**, with explicit escalation rules.

Axiom 4 — Bidirectional Adaptation (Co-evolution)

Humans and agents co-adapt over time. System design must assume evolving skill, trust, and failure modes, and update orchestration/governance accordingly.

Axiom 5 — Constraint Supremacy

Constraint Vectors are **hard boundaries**, not suggestions. If a plan violates a constraint vector, the Engine rejects it pre-flight.

Axiom 6 — Economic Viability

Every symbiotic workflow must remain ROI-positive under bounded budgets. The Engine must estimate cost and enforce budgets before execution to prevent Economic Bleed.

A.2 Core Entities (Type System)

These are first-class objects across the stack.

1. Human Agent (H)

- **Definition:** Biological operator possessing sovereign agency.
- **Properties:**
 - **HSS:** Human Symbiosis Score (skill; 0–1).
 - **SCP:** Symbiotic Capacity Profile (bandwidth / max concurrency).
 - **Role level:** L1/L2/L3 access tier.

2. Symbiosis Engine (E)

- **Definition:** Deterministic coordination layer hosting the autonomy state machine; it does not perform model inference.
- **Responsibilities:** Intent routing, planning, budgeting, logging, band gating.

3. Synthetic Agent (A)

- **Definition:** Bounded capability module (model/tool) interacting via MCP, executing steps in a task graph.
- **Properties:** Contract schema, model tier, cost per token/call.

4. Intent Surface / Structured Intent Object (I)

- **Definition:** Machine-readable representation of a human's goal, scope, preferences, regret tolerance, and initial constraints.
- **Note:** I is the canonical input to E and the semantic origin remains human.

5. Constraint Vector (C)

- **Definition:** Boundary conditions applied to a task graph. Violation triggers pre-flight rejection.
- **Dimensions:**
 - **C_legal:** policy / law constraints.
 - **C_safety:** harm / capability constraints.
 - **C_econ:** cost / resource constraints.

6. Budget (B)

- **Definition:** Resource envelope for a specific intent.
- **Dimensions:** Max_USD, Max_Tokens, Max_Time/TTL.

7. Autonomy Band (AB)

- **Definition:** Discrete autonomy state for A under E, bounded by the Regret Boundary.
- **Bands:**
 - **AB0 Advisory** (Agent proposes; Human executes).
 - **AB1 Internal Autonomy** (sandboxed reversible actions).
 - **AB2 Conditional Autonomy** (safe harbors + constraints).
 - **AB3 Operational Co-Agency** (long-horizon supervised operation; requires high SI/HSS and strict C_econ).

8. Context Lineage (L)

- **Definition:** Immutable graph of state-transition ancestry (plans, actions, constraints, outcomes).

A.3 Deep Definitions (Theoretical Constructs)

9. Intent Manifold (M)

High-dimensional space of acceptable outcomes implied by I; E expands a point-prompt into M using context and constraints.

10. Alignment Surface (AS)

Interface (GUI/API) where H transmits I. Fidelity varies from low-friction chat to structured forms with explicit constraints.

11. Governance Kernel (GK)

Minimal isolated logic core in E that enforces risk classes and Regret Boundary rules; cannot be modified by A.

12. Regret Boundary (RB)

Threshold where an action becomes irreversible in a workflow; defines AB transitions and human gating requirements.

13. Task Graph (TG)

Directed acyclic (or bounded-cycle) graph of atomic steps produced by E from I, C, and RB/AB, executed by A under contract.

14. Hybrid Cognitive Loop (HCL)

Pattern of interaction combining H and A across TG execution (e.g., propose-review-revise, co-planning, supervised co-agency).

15. Epistemic Boundary (EB)

Explicit boundary on what the system is allowed to infer or assume (uncertainty limits, domain exclusion, evidence requirements).

16. Multi-Tier Memory (MTM)

Separated memory scopes (session → project → lineage/canonical → meta/compliance). E owns promotion/demotion across tiers.

17. Regulatory Confidence Interval (RCI)

Quantified confidence band for compliance-critical decisions under uncertainty; used to enforce conservative gating near RB.

18. Adoption Bridge (ABridge)

Fast-lane path for low-regret, low-cost tasks to bypass heavy planning/governance cycles while still respecting GK and budgets.

A.4 Operational Glossary (System Terms)

Economic Circuit (EC)

Subsystem inside E that estimates projected cost of TG plans and validates them against B before execution.

Economic Bleed

Failure mode where expensive models or infinite loops are used for low-value tasks, destroying ROI.

HSS (“Hiss”)

Skill metric of H, including intent clarity and feedback quality; governs band eligibility and training needs.

SCP (“Skip”)

Capacity/bandwidth metric of H (max concurrent threads); governs concurrency assignment and overload mitigation.

SI (Symbiosis Index)

Composite system health metric over alignment, quality, and efficiency; required for AB2/AB3 promotion.

Oversight Theater

Human fatigue / passive approval failure mode; system must throttle autonomy or increase oversight when detected.

Model Context Protocol (MCP)

Open standard for agent-to-agent and agent-to-tool communication; contract compliance depends on MCP bindings.

Shadow AI

Unstructured, ungoverned AI use outside Symbiosis primitives and governance.

A.5 Reconciliation Table (v0.5.2 → v1.1 canonical terms)

Use this to purge synonym drift during the merge.

v0.5.2 term	v1.1 canonical term	Note
“Autonomy Regimes”	Autonomy Bands (AB)	Same concept; AB makes the state machine explicit.
“Hard/Soft Constraints”	Constraint Vector (C) with subtypes	Hard/soft remain valid, but nested under C_legal/C_safety/C_econ.
“Prompt / Brief”	Intent Surface / Structured Intent (I)	Prompts are low-fidelity AS inputs; I is canonical.
“Engine / Coordinator”	Symbiosis Engine (E)	E is deterministic OS and owns GK + EC.
“Poly-Agent Ecosystem”	Agentic Layer (A)	Keep ecosystem rhetoric, but A is the type.

"Reversibility threshold"	Regret Boundary (RB)	Use RB everywhere.
"Memory tiers"	Multi-Tier Memory (MTM)	Terminology consistent already.

A.6 Canonical Ontology Diagram (textual outline)

This is the single diagram you should draw and reference everywhere.

1. **$H \rightarrow AS \rightarrow I$**
Human transmits intent through alignment surface; output is structured intent object.
2. **$I + C + B \rightarrow E (GK + EC)$**
Engine receives intent, consolidates constraint vector, validates budgets via economic circuit, applies governance kernel.
3. **$E \rightarrow RB \rightarrow AB$ selection**
Regret boundary logic selects current autonomy band and defines gating.
4. **$E \rightarrow TG \rightarrow A$ (via MCP Contracts)**
Engine synthesizes task graph and binds agent contracts (cost + band eligibility + tools).
5. **A executes TG; telemetry $\rightarrow E$**
Agents act, returning outputs, cost, and safety signals.
6. **E updates L, SI, HSS, SCP; may transition AB**
Context lineage commits; metrics update; autonomy state may promote/demote.

If you want a compact notation for the diagram caption:

State = $\langle I, C, B, AB, TG, L, SI, HSS, SCP \rangle$

and **E** = $f(\text{State}, GK, EC)$.

Appendix B — Symbiosis Index (SI): Formal Definition, Measurement, and Control Use (Full-Length v0.9)

B.0 Purpose and Position in the Framework

The Symbiosis Index (SI) is the **primary health and control metric** for any Hybrid Intelligence System (HIS). SI is not a vanity score or a retrospective evaluation tool. It is used to:

1. **measure relational performance** of human–agent collaboration,
2. **gate autonomy** (promotion/demotion of Autonomy Bands),
3. **detect drift and incipient failures**,
4. **optimize economic viability**, and
5. **provide auditable evidence** for governance and regulators.

SI is computed per workflow episode and tracked longitudinally (rolling SI). It is always interpreted jointly with Human Symbiosis Score (HSS) and Symbiotic Capacity Profile (SCP).

B.1 SI Core Definition

B.1.1 SI as a weighted composite

SI is defined as a weighted composite of five subscores:

$$SI = w_Q Q + w_A A + w_E E + w_G G + w_S S$$

Where:

- **Q — Quality:** correctness/completeness vs the Acceptable Solution Region (ASR₀).
- **A — Alignment:** faithfulness to Baseline Intent (I₀) and active Constraint Vector (C).
- **E — Economic Efficiency:** value delivered per cost under C_{economic}.
- **G — Governability/Auditability:** lineage completeness, contract adherence, explainability.
- **S — Stability/Trust:** reliability across episodes, calibrated trust, low variance.

Weights are sector-specific and risk-specific (configured via Chapter 12 playbooks and Appendix K matrices).

B.1.2 Episode vs rolling SI

- **SI_episode**: computed at Task Graph (TG) completion.
- **SI_rolling**: exponential moving average used for autonomy gating and co-evolution control.

$$SI_{\text{rolling}}(t) = \alpha SI_{\text{episode}}(t) + (1 - \alpha) SI_{\text{rolling}}(t-1)$$

Default: α chosen per sector; safety-critical domains use lower α (longer memory).

B.2 Subscore Specifications

B.2.1 Q — Quality

Goal: measure outcome admissibility inside ASR_o .

Quality is computed via a domain-appropriate evaluation function:

$$Q = \sum_i v_i \cdot \text{score}_i(\text{output}, ASR_0)$$

Typical components:

- **Correctness** (ground truth / validation suite).
- **Completeness** (coverage of required fields).
- **Coherence** (internal consistency).
- **Evidence integrity** (citations, groundedness).

Normative rule: if output violates ASR_o , Q is capped at 0.5 unless a human explicitly re-anchors I_o/ASR_o .

B.2.2 A — Alignment

Goal: measure intent and constraint fidelity.

Alignment decomposes into two terms:

$$A = \beta \cdot \text{sim}(I_0, \text{output}) + (1 - \beta) \cdot \text{adherence}(C, \text{output})$$

Where:

- **sim(I_o , output)**: structured similarity between intent fields and delivered result.

- **adherence(C, output):** binary/graded constraint satisfaction score from governance checks.

Normative rule: any hard-constraint violation $\Rightarrow A = 0$ for the episode and incident trigger (S3+).

B.2.3 E — Economic Efficiency

Goal: ensure symbiosis remains viable.

$$E = \frac{\text{value}(\text{output})}{\text{cost}(\text{episode})}$$

- **value(output):** sector-specific utility proxy (time saved, error reduction, revenue exposure avoided, etc.).
- **cost(episode):** all compute + tool costs + human time valuation if tracked.

E is normalized to [0,1] using a sector baseline :

$$E_{\text{norm}} = \min\left(1, \frac{E}{E_{\text{baseline}}}\right)$$

Normative rule: if economic circuit triggers (TTL, Max_Spend, token ceiling), E is penalized and planner is forced to re-estimate.

B.2.4 G — Governability / Auditability

Goal: measure how inspectable and governable the collaboration is.

G is a product of compliance primitives:

$$G = g_{\text{lineage}} \cdot g_{\text{contract}} \cdot g_{\text{explainability}}$$

Where each term is in [0,1].

- **g_lineage:** completeness of minimum event set (Appendix E).
- **g_contract:** no contract/tool-scope violations.
- **g_explainability:** ability to reconstruct why a decision was produced (critic deltas, evidence trace).

Normative rule: no lineage $\Rightarrow G = 0$, regardless of output quality. An un-auditable success is invalid.

B.2.5 S — Stability / Trust

Goal: detect volatility and mistrust dynamics.

S is derived from longitudinal behavior:

$$S = 1 - \lambda_1 \text{Var}(SI_{\{\text{episode}\}}) - \lambda_2 |\text{override_rate} - \text{target}| - \lambda_3 \text{drift_rate}$$

Interpretation:

- High variance indicates unstable symbiosis.
- Override_rate far above target indicates low calibrated trust; far below target indicates rubber-stamping risk.
- Drift_rate is measured via Chapter 11 drift metric.

S is computed over windows W and contributes strongly to autonomy gating.

B.3 Normalization and Scoring Conventions

B.3.1 Normalization

All subscores must be normalized to [0,1] before weighting. Sector baselines define:

- minimum acceptable Q/A/G floors,
- economic baseline E_baseline,
- stability windows W.

B.3.2 Error class weighting

Not all errors are equal. Weighting depends on regret and risk class:

- **Cosmetic / low-impact errors:** small penalty to Q; no SI gating.
- **Operational errors:** moderate penalty; may reduce AB ceiling temporarily.
- **Safety/legal/RB errors:** A and G collapse to 0; incident triggers.

Normative rule: in Risk Class A domains, Alignment and Governability floors dominate SI, even if Quality seems high.

B.4 Data Requirements for SI Computation

B.4.1 Mandatory inputs per episode

To compute SI honestly, each episode must include:

1. Baseline Intent I_0 and current intent I_t .
2. Active Constraint Vector C and Policy version P_v .
3. TG trace (node lineage, routing decisions).
4. Agent outputs + critic deltas.
5. Tool calls and side effects.
6. Human approvals/overrides (with reason tags).
7. Cost ledger (tokens, tool fees, time).
8. Stability vital signs slice (D , R , E_{blast} , V_{contract}).

B.4.2 Missing-data penalties

If any mandatory input is absent:

- Q/A/E computed if possible,
 - G is reduced proportionally to missing lineage fields,
 - SI is capped to prevent autonomy gain.
-

B.5 Operational Use of SI (Control Logic)

B.5.1 Autonomy gating

Autonomy Bands are SI-gated:

- **Promotion requires:**
 - SI_rolling in green band
 - S green band
 - no S2+ incidents in last window W
 - HSS tier admissible
 - SCP headroom available
- **Demotion triggers:**
 - SI red-band episode or rolling collapse
 - drift/resonance red-band
 - SCP breach
 - any S3+ incident

This ensures autonomy is **earned through stable performance**, not enthusiasm.

B.5.2 Drift detection

SI subscore patterns distinguish drift types:

- **Alignment decay, Quality stable:** intent/policy drift.
- **Quality decay, Alignment stable:** capability or tool issue.
- **Efficiency decay:** economic bleed or planner mis-routing.
- **Governability decay:** lineage gaps or governance bypass behavior.
- **Stability decay:** trust mis-calibration or entanglement.

Drift signals feed Chapter 11 co-evolution loops.

B.5.3 Economic optimization

E subscore enables policy-safe cost optimization:

- swap agents by cost tier,
- redesign TG for fewer high-cost nodes,
- adjust bridging thresholds for low-regret tasks.

Economic improvements cannot violate Q/A/G floors.

B.6 Sector Weighting Profiles (Normative Defaults)

Weights are set per sector playbook, but conservative defaults follow archetypes:

1. **Safety-Critical (Healthcare, Defense):**
 - w_A high, w_G high, w_Q high, w_E low-medium, w_S high.
2. **Audit-Critical (Finance, Legal):**
 - w_G highest, w_A high, w_Q high, w_E medium, w_S medium-high.
3. **Economic-Sensitive (SME Operations):**
 - w_E high, w_Q medium, w_A medium-high, w_G medium, w_S medium.
4. **Ambiguity-Critical (Creative):**
 - w_Q and Satisfaction-proxy highest, w_A medium, w_E medium, w_G medium, w_S medium.

Any deviation from defaults must be governance-signed because it changes autonomy incentives.

B.7 Statistical Properties and Validation

B.7.1 Reliability expectations

SI must be:

- **stable under repeated similar tasks,**
- **sensitive to meaningful degradations,**
- **not trivially gameable.**

B.7.2 Calibration

Calibration steps per sector:

1. Establish baseline SI distributions at AB0.
2. Identify failure signature distribution (S1–S4).
3. Set green/yellow/red bands per risk class.
4. Validate demotion/promotion behavior in simulated incidents.

B.7.3 External validation

Required before AB3 scale:

- compare SI to human expert ratings,
 - compare SI to objective outcome KPIs,
 - stress SI under drift and resonance scenarios.
-

B.8 Example Computation (Illustrative)

Assume sector weights:

$w_Q=0.30$, $w_A=0.25$, $w_E=0.15$, $w_G=0.20$, $w_S=0.10$

Episode subscores:

- $Q=0.85$
- $A=0.92$
- $E=0.60$
- $G=1.00$
- $S=0.80$

Then:

$$SI = 0.30(0.85) + 0.25(0.92) + 0.15(0.60) + 0.20(1.00) + 0.10(0.80)$$

$$SI = 0.255 + 0.230 + 0.090 + 0.200 + 0.080$$

$$SI = \mathbf{0.855}$$

If G were missing lineage and fell to 0.4:

$$SI \text{ would drop to } 0.30(0.85) + 0.25(0.92) + 0.15(0.60) + 0.20(0.40) + 0.10(0.80)$$

= 0.255+0.230+0.090+0.080+0.080
= **0.735**, triggering yellow-band review.

B.9 Common Pitfalls (Anti-Patterns)

1. **Metric idolatry:** optimizing SI rather than outcomes.
Mitigation: require qualitative governance reviews alongside SI.
 2. **Gaming through lineage suppression:** hiding failures to keep SI high.
Mitigation: G=0 without lineage; random audits.
 3. **Uniform weights across sectors:** ignoring regret logic.
Mitigation: sector playbook-bound weights only.
 4. **SI without HSS/SCP:** raising AB based only on SI.
Mitigation: explicit multi-gate rule is mandatory.
-

B.10 Implementation Checklist (Minimum Conformance)

An organization may claim SI compliance only if:

- ☐ All subscores computed per episode.
 - ☐ Rolling SI used for autonomy gating.
 - ☐ Lineage minimum event set enforced (G term real).
 - ☐ Sector weights governance-signed.
 - ☐ Drift/resonance/entanglement coupling active.
 - ☐ SI red-band triggers deterministic safe-mode behavior.
 - ☐ Validation runbooks executed before AB3 scale.
-

Here is **Appendix C — Autonomy Bands & Regimes (Full-Length v0.9)**, expanded into a complete, audit-grade appendix aligned to Chapters 7–11 and Appendix B/H/K/O.

Appendix C — Autonomy Bands & Regimes: Formal Definitions, Gates, and Operating Rules (Full-Length v0.9)

C.0 Purpose and Position in the Framework

Autonomy in Hybrid Intelligence Systems is not a binary switch. It is a **governed spectrum** that must be:

1. **risk-aligned** (Appendix D / Appendix K),
2. **capacity-aligned** (HSS/SCP — Appendix H),
3. **performance-aligned** (SI — Appendix B), and
4. **stability-aligned** (drift/resonance/entanglement — Chapter 11 / Appendix G).

This appendix defines the Autonomy Bands (AB0–AB3 plus AB2.5 Emergent), their admissible domains, the mandatory gating logic for promotion/demotion, and the operating rules that prevent autonomy creep, oversight theater, and un-auditable agency.

Normative claim:

A system is not Symbiosis-compliant unless autonomy is explicitly represented, bounded, and controlled by deterministic gates.

C.1 Autonomy as a Controlled State Machine

C.1.1 Autonomy state object

For every workflow episode, the Engine maintains an autonomy state:

$AB_t \in \{0, 1, 2, 2.5, 3\}$

Autonomy state is attached:

- globally to a workflow, and
- locally to each TG node (node-level ceilings can be lower than workflow-level AB).

C.1.2 Independence from “capability”

Autonomy is not equivalent to model capability. A highly capable agent can still be governed at AB0. Conversely, a low-capability agent should never be elevated above admissible AB ceilings. The autonomy state is a **governance decision**, not a capability accident.

C.2 Formal Band Definitions

AB0 — Advisory Symbiosis

Definition:

Agents may generate suggestions, drafts, analyses, and plans. Humans must approve and execute all steps of consequence, including any RB-adjacent action.

Agent permissions:

- read-only tools,
- drafting outputs,
- discovery and synthesis,
- no external side effects without human click-through.

Human role:

- author I_o / clarify intent,
- review outputs,
- execute or explicitly authorize irreversible actions.

Typical admissible domains:

- all **Risk Class A** tasks by default,
- any domain with unclear constraints or low stability history.

Boundaries:

- RB crossings are never autonomous, even when the outcome is “obvious.”
-

AB1 — Internal Autonomy (Reversible Execution)

Definition:

Agents may execute reversible tasks below RB under contracts and constraints. Humans checkpoint at RB crossings and verify high-regret outputs.

Agent permissions:

- reversible tool calls,
- internal-only actions,
- TG execution without human micro-approval while below RB.

Human role:

- approve TG planning and RB gates,

- supervise exception handling,
- validate high-importance subtasks.

Admissible domains:

- **Risk Class B** tasks with stable SI baseline,
- low-to-mid regret steps in Class A workflows.

Typical examples:

- summarization + internal routing,
- internal forecasting models,
- draft generation with later human publication.

AB2 — Conditional Autonomy (Safe Harbor Execution)

Definition:

Agents may execute bounded workflows including some RB-proximate steps, but only within explicitly defined **Safe Harbors** and conditional gates.

Agent permissions:

- execution of bounded external actions where rollback or near-zero regret is guaranteed,
- conditional auto-approval within confidence margins.

Human role:

- define Safe Harbor boundaries,
- review exceptions and drift signals,
- periodically re-anchor I_0/ASR_0 in tactical loop.

Admissible domains:

- **Risk Class C** by default,
- selected slices of **Risk Class B** when stability is green.

Safe Harbor requirements (normative):

1. explicit Safe Harbor declaration in governance policy,
2. rollback semantics for all tools,
3. stable SI_rolling in green band for window W,
4. drift and resonance green bands,
5. operator certification (L2+),
6. SCP headroom available.

AB2.5 — Emergent Autonomy (Intermediate Real Phase)

Definition:

A transitional phase where local teams reach **near-AB3 operational patterns** in limited pockets. This phase is real, common, and unstable. It is not an honorary label.

Attributes:

- agents initiate and sequence multi-step plans,
- humans supervise at higher abstraction,
- divergence and entanglement pressures rise sharply.

Risks:

- unstandardized local contracts,
- autonomy creep beyond governance visibility,
- concentrated blast radius from shared tools/memory.

Normative rule:

AB2.5 pockets must be treated as **experimental regimes**. If not stabilized or standardized, they must be demoted.

Admissible domains:

- only where governance explicitly flags Emergent-phase permission,
 - never in unbounded Class A domains.
-

AB3 — Co-Agency (Bounded Autonomous Partnership)

Definition:

Agents may **initiate and execute** within bounded domains, including multi-step planning and some RB-proximate action, while humans govern intent horizons, budgets, audits, and exceptions.

Agent permissions:

- proactive TG generation,
- multi-agent orchestration under the Engine,
- conditional external actions within Safe Harbor + RB rules,
- self-escalation when constraints detected.

Human role:

- set and refresh I_0/ASR_0 ,
- define long-horizon objectives and regret tolerances,
- shape governance and stability budgets,
- adjudicate exceptional or contested decisions.

Admissible domains:

- bounded **Class B safe harbors** and **Class C domains** only,
- never for lethal/liberty/clinical diagnosis/trade execution unless sector matrix and regulator explicitly allow (rare).

Invariant:

Even at AB3, **responsibility and moral authority do not shift**. Humans remain accountable at RB.

C.3 Risk Classes and Default AB Ceilings

Autonomy ceilings are assigned by risk class (Appendix D), then overridden by sector matrices (Appendix K).

Default mapping:

Risk Class	Default AB Ceiling	Notes
A (High-regret / rights-critical)	AB0–AB1	AB2+ requires explicit sector/regulator permission
B (Mid-regret)	AB1–AB2	AB3 only within governance-defined safe harbors
C (Low-regret / reversible)	AB2–AB3	AB3 requires stability green bands

Normative rule:

If risk class is unknown or disputed, treat as **Class A until retyped**.

C.4 Promotion Gates (Normative)

Autonomy promotions are **governance acts triggered by evidence**. The Engine may recommend promotions, but Governance must authorize them.

C.4.1 Global promotion prerequisites

To elevate any workflow above current AB:

1. **Risk admissibility:**
workflow risk class permits higher AB.

2. **Performance stability:**
SI_rolling in green band for window W.
No red-band SI episodes in W.
3. **Stability vital signs:**
Drift D_t, Resonance R_t, Entanglement E_blast, Divergence V_contract all green.
4. **Operator readiness:**
HSS tier supports target AB (Appendix H / Chapter 16).
Certification level matches band rights.
5. **Capacity headroom:**
SCP indicators within limits (no chronic breach).
6. **Recovery readiness:**
Safe mode and rollback conformance tests passed (Appendix O).

C.4.2 Promotion logging

Any promotion must create a lineage record containing:

- prior AB state,
- evidence slice (SI/HSS/SCP + vital signs),
- governance authorizer,
- scope boundaries (Safe Harbor definition),
- effective date and review cadence.

C.5 Demotion Triggers (Normative)

Autonomy demotions may be triggered automatically by the Engine and must be honored.

C.5.1 Automatic demotion triggers

1. **SI red-band (episode or rolling):**
immediate demotion by ≥ 1 band.
2. **Drift or Resonance red-band:**
revoke Bridges, demote to AB0 safe mode.
3. **Entanglement red-band:**
block AB ≥ 2 promotions; demote AB3 pockets to AB2 until stable.

4. **SCP breach:**
throttle concurrency; if repeated, demote AB.
5. **S3+ incidents:**
AB0 lock pending governance disposition.

C.5.2 Non-automatic demotions

Governance may demote autonomy for:

- regulatory change,
- policy lag concerns,
- emerging domain uncertainty,
- ethical controversy,
- sustained operator fatigue.

Such demotions are **not punitive**; they are stability maintenance.

C.6 Node-Level Autonomy Ceilings

A workflow AB is a ceiling, not a guarantee. Each TG node may have a tighter AB cap.

Example:

In a finance workflow:

- research synthesis nodes at AB2,
- portfolio adjustment nodes capped AB0,
- client comms nodes AB1 with RB checkpoint.

Normative rule:

The Engine must enforce the most conservative cap among:

- workflow AB,
 - node AB,
 - risk-class AB ceiling,
 - sector matrix AB ceiling,
 - operator certification AB ceiling.
-

C.7 Adoption Bridges and AB Interaction

Bridges are a safety valve for adoption and a controlled autonomy lane.

C.7.1 Bridge admission rules

A TG node may be auto-admitted into Bridges if:

- risk class is C or low-B,
- SI history is green,
- no drift/resonance alerts,
- rollback semantics exist,
- governance has enabled Bridges for that domain.

Bridge autonomy defaults to **AB1**, never AB2+.

C.7.2 Bridge revocation rules

Automatic revocation if:

- SI collapse,
- drift red-band,
- any S2+ incident in Bridge lane.

Bridges can be re-admitted only after stable SI recovery and governance sign-off.

C.8 Oversight Load and SCP Safety

Autonomy must never exceed supervisory capacity.

C.8.1 SCP as a binding ceiling

Even if SI is green, AB cannot be raised if SCP headroom is insufficient. Typical breach patterns:

- high concurrency without batching,
- high complexity tasks routed to low-tier operators,
- RB checkpoint overload.

C.8.2 Oversight theater prevention

The Engine must detect when human approvals are becoming non-substantive (fast-click approvals, no rationale). This signals SCP overload and triggers:

- batching,
 - explanation compression,
 - AB demotion.
-

C.9 AB and Economic Circuit Coupling

Higher AB generally reduces human latency but may increase compute spend.

Normative rule:

AB promotions must include a projected economic delta. If projected spend exceeds C_economic bounds, AB promotion is invalid.

Conversely, economic pressure is never a valid reason to override risk-class AB ceilings.

C.10 Verification and Conformance Tests (Required)

Before claiming AB2+ or AB3 operation, the organization must pass:

1. **Safe-mode conformance test**
 - red-band trigger forces AB0 lock.
2. **Kill-switch conformance test**
 - S4 trigger halts system and quarantines agents.
3. **Rollback test**
 - every external tool path demonstrates correct undo semantics.
4. **Drift gate test**
 - simulated drift causes AB demotion + Bridge revocation.
5. **Entanglement gate test**
 - high shared-tool change blocks AB promotion.

These tests are defined in Appendix O and must be periodically re-run (Chaos Symbiosis drills).

C.11 Common Failure Patterns and Anti-Patterns

1. **Autonomy creep without evidence**
 - raising AB due to trust or urgency rather than SI/HSS/SCP proof.
2. **Band inflation by re-labeling risk**
 - down-typing Class A work into Class B to unlock AB2+.
3. **AB3 without safe harbors**
 - removing bounds around co-agency domains.

4. Node-level mismatch

- leaving high-regret nodes at same AB as low-regret nodes.

5. SCP-ignorant scaling

- increasing AB without attention budgeting, leading to oversight collapse.

Each anti-pattern should be explicitly proscribed in governance trainings (Chapter 16 / Appendix I).

C.12 Implementation Checklist (Minimum Compliance)

An organization is Autonomy-Standard compliant only if:

- [] AB state is explicit per workflow and TG node.
 - [] Risk class AB ceilings enforced (Appendix D/K).
 - [] SI_rolling gates AB promotions.
 - [] Stability vital signs gate promotions/demotions.
 - [] HSS certification tiers bound to AB rights.
 - [] SCP headroom gates scaling.
 - [] Bridges operate only under AB1 conservatism.
 - [] Safe mode, kill switch, rollback tests passed and re-run quarterly.
 - [] All promotions/demotions are logged in lineage.
-

Below is **Appendix D — Governance Matrices & Risk Models (Full-Length v0.9)**, expanded to align with v0.8 “skeleton,” incorporate v0.5.2 “flesh,” and add additional “muscle” so it reads like a defensible engineering standard.

Appendix D — Governance Matrices & Risk Models: Formal Structure, Sector Overlays, and Control Logic (Full-Length v0.9)

D.0 Purpose and Position in the Framework

This appendix defines the **deterministic governance substrate** used by the Symbiosis Engine to decide what actions are permitted, what autonomy level is admissible, and what controls must be applied. v0.8 explicitly asserts governance is computed against a formal risk matrix, not guessed or negotiated in-chat.

The goal is to provide:

1. A **universal base risk model** (cross-sector).
2. A **decision rights matrix** mapping actors to permissions.
3. A **control hierarchy** binding risks to enforcement modalities.
4. **Sector overlays** that tune the base model without redefining it.
5. A clear interface to **SI/HSS/SCP and Autonomy Bands** (Appendices B/C/H).

Normative claim:

No workflow can run above AB0 unless it is typed through this appendix and enforced by the Policy Engine.

D.1 Governance Matrices (Decision Rights)

D.1.1 Actors and authority surfaces

Decision rights are distributed across three human roles and two system roles:

- **Human Roles**
 - **Symbiote Operator (SO):** executes/approves at RB, supplies intent and feedback.
 - **Hybrid Intelligence Architect (HIA):** designs TGs, configures agents, budgets, safe harbors.
 - **Governance Lead (GL):** owns risk typing, constraints, autonomy ceilings, incident board.
- **System Roles**
 - **Symbiosis Engine (E):** enforces policy + budgets pre-execution.
 - **Agents (A_i):** execute bounded tasks under contracts; cannot self-expand rights.

D.1.2 Decision Rights Matrix (base)

Decision Category	SO	HIA	GL	Engine	Agents
Define/refresh Intent I ₀	R/W	R	R	R	R

Define Constraint Vector C	R	R/W (draft)	R/W (authoritative)	R	R
Risk Class typing	R	R (recommend)	R/W (authoritative)	R	—
Safe Harbor definition	R	R/W (draft)	R/W (approve)	R	—
AB promotion/demotion	—	R (recommend)	R/W (authorize)	W (auto-demote)	—
Tool access scope	R	R/W (draft)	R/W (approve)	R/W (enforce)	R
Kill switch activation	R/W	R/W	R/W	W (honor)	—
Incident classification	R	R (technical)	R/W	R	R

Legend: R=read, W=write, “authoritative” means final legal/ethical ownership.

Normative rule:

If SO and HIA disagree, GL arbitration is binding.

D.2 Risk Models (Base)

D.2.1 Two-axis risk score (v0.5.2 core)

Each task or agent action is typed by:

- **Impact (I):** consequence if the action fails.
- **Probability (P):** likelihood of failure or misalignment.

Risk score is computed as:

$$\text{Risk} = f(I,P)$$

Where f is typically a multiplication or lookup into the Risk Assessment Matrix.

D.2.2 Impact scale

Impact is regret-weighted and aligns to RB:

Impact Level	Description	Typical RB proximity
I1 — Minimal	Annoyance/cosmetic error	Far below RB
I2 — Moderate	Local operational loss	Below RB
I3 — High	Material financial / safety exposure	At RB
I4 — Catastrophic	Rights, life, existential org damage	Beyond RB

D.2.3 Probability scale

Probability is empirically derived from SI history, agent capability index, and drift signals:

Probability Level	Description
P1 — Unlikely	Strong SI stability, robust evidence
P2 — Possible	Some variance or low sample size
P3 — Likely	Recurrent SI warnings or weak agent fit
P4 — Near-certain	Known failure mode or drift red-band

D.2.4 Risk Assessment Matrix (v0.5.2 mapping)

The matrix combines I and P to generate a **risk zone** that binds AB ceilings.

Canonical mapping:

	I1	I2	I3	I4
P4	Med (AB1)	High (AB0/1)	High (AB0)	Catastrophic (AB0 lock)
P3	Low (AB2)	Med (AB1)	High (AB0/1)	Catastrophic (AB0 lock)
P2	Low (AB2/3)	Low (AB2)	Med (AB1)	High (AB0/1)
P1	Low (AB2/3)	Low (AB2)	Med (AB1)	High (AB0/1)

Normative rule:

If a task touches I4 or P4, **AB0 is the default unless a sector overlay explicitly allows higher bands.**

D.3 Risk Class Typing

Risk Zones collapse into three framework-level Risk Classes (Appendix C references):

- **Class A (High-regret / rights-critical):** any I4, or I3 with P3+.
- **Class B (Medium-regret):** I2–I3 with P2–P3.
- **Class C (Low-regret / reversible):** I1–I2 with P1–P2.

Normative rule:

Unknown tasks are typed Class A until evidence supports re-typing.

D.4 Governance Control Models (Hierarchy)

v0.8 introduces a four-level control stack the Policy Engine enforces once risk is typed.

D.4.1 Control Level 0 — Kernel Constraints

- **Hard-coded invariants inside the Engine.**
- Enforced **pre-execution**; cannot be overridden by agents.
- Example: “No write access to prod_db if trust < 0.99.”

D.4.2 Control Level 1 — Agentic Oversight

- Parallel critic/red-team agents evaluate plans.
- Execution blocked until oversight passes.

D.4.3 Control Level 2 — Human Confirmation

- Click-to-execute barrier at RB.
- Workflow pauses until human token received.

D.4.4 Control Level 3 — Dual-Key Escalation

- Two-person rule for high-regret states and AB3 deployments.

Normative rule:

Control level is **monotonic with risk**. You may add layers; you may not subtract layers below the class minimum.

D.5 Economic Risk Model (Muscle + v0.8 integration)

v0.8 treats spend as an explicit risk dimension to prevent “economic bleed.”

D.5.1 Spend tiers and approvals

Spend Tier	Example	Approval Mode	Default AB
Micro-Spend (<\$0.50/session)	cheap summaries	Implicit	AB1–2
Macro-Spend (<\$20/project)	multi-agent research	Conditional if SI_rolling > 0.8	AB1–2
CapEx (>\$100/project)	long-horizon simulations	Explicit human authorization	AB0–1

Normative rule:

Economic tier can **lower AB** but cannot raise AB above risk-class ceilings.

D.5.2 Cost-risk coupling to SI

Economic Risk contributes to SI via the **E subscore** (Appendix B). If Economic tier is violated:

- E collapses,
 - SI caps,
 - AB auto-demotes.
-

D.6 Sector Overlays (v0.8 skeleton)

v0.8 introduces **sector overlays** to bias the base matrix without redefining the universal logic.

D.6.1 Overlay rules

An overlay may:

- re-weight impact types,
- add risk factors,
- promote specific action categories to higher severity automatically.

An overlay may **not**:

- alter core invariants (Human Primacy, Constraint Supremacy),
- bypass RB or Control Level 0 constraints.

D.6.2 Canonical overlays (examples)

Overlay A — Healthcare (Bio-Safety Bias)

Any clinical/patient-data action is promoted to catastrophic severity, forcing AB0 sign-off.

Overlay B — Finance (Latency & Audit Bias)

Latency becomes a risk factor; slow execution raises severity and tightens governance kernels.

Overlay C — Software / Ops (Destructive Bias)

Destructive actions (DROP TABLE, rm -rf) auto-promoted to S3-equivalent risk, forcing AB0/1 and dual confirmation.

D.7 Risk-to-Autonomy Mapping (Formal)

Given a typed task:

1. Compute (I,P) → Risk Zone → Risk Class.
2. Apply overlay modifiers (sector).
3. Choose minimum Control Level needed.
4. Assign **node AB ceiling** and **workflow AB ceiling**.

$$AB_{\{max\}} = \min(AB_{\{risk\}}, AB_{\{overlay\}}, AB_{\{node\}}, AB_{\{operator\}}, AB_{\{SCP\}})$$

Where:

- AB_operator derived from HSS tier,
- AB_SCP derived from capacity headroom.

Normative rule:

If any ceiling returns AB0, the Engine must enforce AB0 regardless of other signals.

D.8 Governance Telemetry and Audit Duties

D.8.1 Minimum evidence set

Every RB-relevant episode must produce:

- I₀ and I_t snapshots,
- active C vector + P_v (policy version),
- TG + routing trace,
- control levels invoked,
- AB state transitions,
- incident flags,

- SI/HSS/SCP deltas.

D.8.2 Audit cadence

- **Class A workflows:** weekly audit + incident board.
 - **Class B workflows:** monthly audit.
 - **Class C workflows:** quarterly rollup unless SI drift detected.
-

D.9 Incident Typing and Escalation

Incidents map to severity grades S1–S4 (Chapter 13), but governance requires:

- **S1–S2:** local correction, SI penalty, possible AB demotion.
- **S3:** mandatory GL review, AB0 lock pending disposition.
- **S4:** kill switch, full quarantine, external regulator notification if required.

Normative rule:

Any hard constraint violation is treated as ≥S3.

D.10 Ethical & Regulatory Compliance Layer

Building on v0.5.2, risk matrices must embed applicable legal and ethical controls.

D.10.1 Compliance mapping object

Each sector playbook (Appendix K) must provide:

- jurisdictional laws by constraint type (C_legal),
- safety norms (C_safety),
- economic policies (C_economic),
- required audit artifacts and retention windows.

D.10.2 Non-punitive governance

HSS/SCP are safety telemetry, not HR weapons; governance must firewall them from punitive use (Chapter 4/10/H).

D.11 Common Failure Patterns (Anti-Patterns)

1. **Risk down-typing to unlock autonomy.**
2. **Overlay drift** (local teams subtly redefining overlays).

3. **Control inversion** (human approvals without lineage).
4. **Economic override creep** (budget pressure treated as safety justification).
5. **“AB by performance only” promotions** (ignoring risk class).

Each is an audit-grade violation.

D.12 Implementation Checklist (Minimum Compliance)

An organization is Appendix-D compliant only if:

- ☐ Two-axis risk typing (Impact × Probability) operationalized.
 - ☐ Risk Assessment Matrix binds AB ceilings.
 - ☐ Control hierarchy L0–L3 implemented.
 - ☐ Economic tier model enforced.
 - ☐ Sector overlays exist and are governance-signed.
 - ☐ AB ceilings computed as min-function across risk/overlay/operator/SCP.
 - ☐ Audit evidence set captured per episode.
 - ☐ Incident escalation logic hard-wired to constraints and RB.
-

Absolutely. Below is **Appendix E — Memory Architecture & Context Lineage Schemas (Full-Length v0.9)**, expanded into a defensible engineering appendix. It preserves v0.8’s schema rigor and storage/retrieval detail, while incorporating v0.5.2’s conceptual framing for context lineage and multi-tier memory.

Appendix E — Memory Architecture & Context Lineage Schemas (Full-Length v0.9)

E.0 Purpose and Position in the Framework

Unstructured AI systems treat memory as incidental: whatever fits in a context window or lives in a user’s copy-paste trail. The Symbiosis Framework treats memory as **explicit, tiered, governed infrastructure**.

Appendix E defines:

1. **What memory is allowed to exist** in a Hybrid Intelligence System (HIS).
2. **How memory is tiered**, accessed, and forgotten.
3. The **Context Lineage (L)** model that makes every decision audit-addressable.
4. Mandatory storage technologies and retrieval strategies required to prevent **context drift, leakage, and oversight theater**.

Normative claim:

A system cannot claim Symbiosis compliance if (a) memory tiers are not explicit, or (b) actions are not referentially linked to lineage blocks.

E.1 Context Lineage (L)

E.1.1 Definition

Context lineage is the structured trace of how goals, constraints, assumptions, system state, and interventions evolve over time within a workflow.

Lineage answers:

- Why is the system behaving this way now?
- Which decisions produced the current configuration?
- Who changed what, when, and under which constraints?

E.1.2 Components of lineage

Lineage includes, at minimum:

1. **Intent evolution:** $I_0 \rightarrow I_t$ diffs (goal/scope changes).
2. **Constraint evolution:** policy versions, regulatory updates, local overrides.
3. **System evolution:** agent swaps, model updates, TG rewrites.
4. **Incident/override history:** human checkpoints, escalations, kill-switches.

E.1.3 Role in governance and SI

Lineage is mandatory for:

- auditability and forensics,
- SI interpretation over time,
- co-evolution modeling and stability control.

If lineage is absent, Appendix B requires **G=0** and SI caps.

E.2 The Lineage Schema (L) — Tamper-Evident Blocks

E.2.1 Block model

v0.8 formalizes lineage as a **hash-chained block record** (conceptually analogous to an append-only ledger).

Normative policy: each TG node execution produces a lineage block; no consequential action may be an “orphan.”

E.2.2 Canonical block fields (minimum event set)

A lineage block must include:

- **Block metadata**
 - Block_ID (UUID)
 - Timestamp (UTC)
 - Workflow_ID / TG_Node_ID
 - Previous_Block_Hash (integrity chain)
- **Actor**
 - Type: Human / Synthetic_Agent / Engine / Governance
 - ID + Model_Version for agents
 - MCP endpoint (if used)
- **Input_Context**
 - Intent_Hash (I_o or I_t)
 - Active_Constraints (C vector IDs)
 - Retrieval_IDs (memory fragments used)
 - Budget_Remaining
- **Reasoning_Trace**
 - rationale summary (not raw chain-of-thought)
 - Confidence_Score
 - Critic/oversight deltas
- **Action**
 - Tool_Call / Draft / Plan / External_Effect
 - Tool + Method + Payload_Hash
- **Verification**
 - Oversight agent(s) results
 - Human Token if RB crossed
 - Risk_Class + AB state

This mirrors the JSON in v0.8 and is required for audit-grade causality.

E.2.3 Integrity requirements

- Hash chaining makes deletion/crud tamper-evident.
 - Block writes are append-only.
 - If any block is missing for a TG node, the workflow is treated as governance-invalid.
-

E.3 Multi-Tier Memory Architecture

E.3.1 Memory tiers (base ontology)

v0.5.2 defines five tiers.

We retain these and map them to v0.8's storage + retrieval constraints.

1. Tier 1 — Session Memory

- Short-lived context within a single episode.
- Examples: active conversation, scratch variables, transient hypotheses.

2. Tier 2 — Project Memory

- Mid-horizon memory spanning a specific workflow/project.
- Contains intermediate artifacts, local constraints, TG state.

3. Tier 3 — Lineage Memory

- Long-horizon evolution record.
- Stores context lineage blocks and policy/intent ancestry.

4. Tier 4 — Meta-Memory

- Knowledge about system behavior: stability patterns, SI/HSS/SCP trajectories, recurring fixes.

5. Tier 5 — Compliance Memory

- Regulatory/audit store: sensitive access logs, incident packages, retention-bound evidence.

E.3.2 Governance of tiers

Each tier is governed by:

- retention policy,
- access rules,
- use constraints (trainable vs non-trainable, anonymization, deletion).

The Governance Layer defines rules; the Engine enforces operational reads/writes.

E.4 Storage Technologies (Normative Reference Design)

Tier 1 — Session

- Storage: in-process cache / short-TTL KV store.
- Encryption: optional (unless PII touches).
- TTL: minutes–hours.

Tier 2 — Project

- Storage: Vector DB for semantic retrieval + object store for artifacts.
- Versioning: per-artifact hashes + TG pointer.
- TTL: days–months (sector-bound).

Tier 3 — Lineage

- Storage: immutable ledger / append-only log store.
- Must support hash chaining and WORM semantics.

Tier 4 — Meta-Memory

- Storage: SQL/Graph DB with derived telemetry.
- Update cycle: via Co-Adaptive Loop.

Tier 5 — Compliance

- Storage: cold storage with object lock / legal hold support.
- Access: restricted to Governance Lead + auditors.

Normative rule:

Any tier storing regulated data must be residency- and encryption-compliant with Appendix K.

E.5 Read/Write Policies (Control-Grade)

E.5.1 Write rules

A write is permitted only if:

1. the actor has write rights for that tier (Appendix D),
2. the content conforms to active constraints C,
3. the intent scope allows storage, and
4. the lineage block for the write is emitted.

Tier promotions (e.g., Project → Canonical) require Governance approval.

E.5.2 Read rules

Reads are filtered by:

- active constraints,
- risk class,
- AB ceiling,
- SCP headroom (avoid operator overload).

If retrieval would violate C_legal/C_safety, the Engine must return an explicit refusal trace.

E.6 Retrieval Strategies — Priority Waterfall (v0.8)

v0.8 mandates a **priority waterfall** for context retrieval.

E.6.1 Canonical priority

If a query touches policy, intent invariants, or safety constraints, Tier 3 Canonical/Lineage must be retrieved before Tier 1/2. Policy overrides memory.

E.6.2 Recency bias

For conversational continuity and short-horizon tasks, Tier 1 is weighted above Tier 2.

E.6.3 Semantic relevance

For problem solving, Tier 2 retrieval uses vector similarity and task-graph anchors.

E.6.4 Retrieval logging

Every retrieval produces:

- Retrieval_IDs inserted into lineage block,
- source tier,
- similarity score or selection rationale,
- constraint check pass/fail.

No retrieval is “silent.”

E.7 Forgetting Protocols / Data Hygiene (v0.8)

A system that never forgets becomes expensive, slow, and drift-prone. v0.8 prescribes strict hygiene.

E.7.1 Session flush

On TG completion, Tier 1 is wiped to prevent context bleeding between tasks.

E.7.2 Project pruning

Tier 2 memory is periodically summarized and compacted by a Summarizer/Archivist agent.

- raw fragments archived,
- canonical summary replaces active retrieval surface,
- pruning emits lineage blocks.

E.7.3 Canonical hardening

Tier 3/4 content is slowly hardened:

- only governance-approved truths graduate into Canonical memory,
- stale canonical items are versioned and deprecated, not overwritten.

E.7.4 Compliance expiry

Tier 5 retention follows sector matrices. When expiry triggers:

- content is cryptographically wiped,
 - lineage stores deletion proof.
-

E.8 Failure Modes and Protections

E.8.1 Context drift

Root causes:

- Tier 1 overwriting constraints,
- Tier 2 bloat hiding key artifacts,
- missing canonical priority.

Protections:

- retrieval waterfall enforcement,
- ASR_o checks at every TG node,
- drift vital sign tied to SI A/G subscores.

E.8.2 Hallucinated continuity

If Tier 1 contains low-confidence tokens or ambiguous commits, Engine must:

- fetch Tier 2/3 cross-check,
- request human clarification if mismatch persists.

E.8.3 Cross-project leakage

Protections:

- strict tier scoping,
- residency controls,
- “no un-governed direct paths” invariant.

E.8.4 Memory poisoning

If an operator or agent injects false canonical content:

- G collapses,
- incident class $\geq S3$,
- canonical rollback required.

E.9 Interface to SI / Autonomy / Human Metrics

- **SI G term** is computed from lineage completeness and retrieval traceability (Appendix B).
- **AB promotion** requires stable lineage and absence of drift (Appendix C).
- **HSS Artifact Hygiene dimension** explicitly measures memory curation discipline (Appendix H).

Memory is therefore not storage plumbing; it directly controls autonomy, stability, and safety.

E.10 Implementation Checklist (Minimum Compliance)

An organization is Appendix-E compliant only if:

- [] Context lineage is explicit and block-structured.
 - [] All five memory tiers exist and are governed.
 - [] Storage tech matches tier requirements.
 - [] Retrieval uses the priority waterfall.
 - [] Forgetting protocols run deterministically.
 - [] Every retrieval/write is logged into lineage.
 - [] Canonical memory promotion is governance-signed.
 - [] Cross-tier leakage protections meet Appendix K constraints.
-

Below is Appendix F — Agent Contracts, Capability Indices, and Protocol Schemas (Full-Length v0.9). This expands v0.8's technical skeleton (contracts + MCP + protocols + arbitration) and integrates v0.5.2's conceptual flesh (capability indices, reliability tracking, norm-aware contracts, and governance coupling).

Appendix F — Agent Contracts, Capability Indices, and Protocol Schemas (Full-Length v0.9)

F.0 Purpose and Position in the Framework

The Symbiosis Framework treats agents as bounded capability modules rather than general-purpose oracles. Each agent is governed by:

1. a Contract (machine-readable declaration of what it is allowed to do),
2. a Capability Index (measured performance + reliability over time), and
3. a Protocol (standardized message, handshake, and arbitration rules).

These artifacts allow the Symbiosis Engine to:

route tasks deterministically,

pre-flight check governance and budgets,

prevent tool-scope creep,

reduce entanglement by forcing Engine-mediated coordination, and

audit every agent action via lineage bindings (Appendix E).

Normative claim:

No agent may execute in a Symbiosis-compliant system unless a valid Contract is registered, Capability Index is tracked, and MCP/Protocol conformance is enforced.

F.1 Agent Contracts (Normative)

F.1.1 Definition

An Agent Contract is a declarative JSON/YAML schema that specifies:

identity,

capability surface,

constraints (hard/soft/economic),

tool/transport interfaces,

failure semantics, and

cost tiering.

Contracts are not documentation; they are enforced inputs to governance pre-flight.

F.1.2 Canonical Contract Schema

Minimum required fields:

agent_id: "finance-analyst-v2"

cluster: "Discovery|Design|Execution|Oversight"

priority: 0-1 # arbitration weight

owner: "org/team"

model_spec:

provider: "openai|anthropic|local"

tier: "tier-1|tier-2|tier-3" # cost/reasoning class

context_window: 200000

economics:

cost_per_1k_tokens: 0.015

avg_latency_ms: 4500

max_expected_usd_per_episode: 3.00

capabilities:

- name: "analyze_ticker"

description: "Generates risk report"

input_schema: {...}

output_schema: {...}

constraints:

hard:

- "NO_TRADE_EXECUTION"

- "NO_PII_WRITE"

soft:

- "CITE_SOURCES"

- "STYLE_BRIEF"

econ:

max_usd: 5.00

max_tokens: 25000

max_time_ms: 300000

interfaces:

protocol: "MCP/1.0"

transport: "SSE|gRPC|HTTP"

endpoint: "https://agents.internal/... "

tools: ["fetch_balance", "read_logs", ...]

failure_modes:

- error: "DATA_UNAVAILABLE"

- behavior: "RETURN_NULL"

- error: "AMBIGUOUS_INPUT"

- behavior: "REQUEST_CLARIFICATION"

versioning:

contract_version: "1.3.0"

compatible_engine: ">=0.8"

last_audited: "2025-11-01"

This aligns to the v0.8 exemplar contract including model spec, economics, capabilities, constraint blocks, interfaces, and failure semantics.

F.1.3 Contract invariants (hard rules)

1. Constraint Supremacy:

Contract constraints cannot loosen Governance Kernel rules. Engine rejects registration if conflict exists.

2. No Shadow Paths:

Agents may not declare direct external tool access unless mediated by MCP Host (the Engine).

3. Explicit side-effect typing:

Every tool must declare: reversible/irreversible, RB proximity, and rollback path.

4. Economic admissibility:

Agents in low-value clusters cannot self-declare high-cost tiers unless sector playbook allows.

F.1.4 Contract lifecycle

1. Drafted by HIA (technical owner).

2. Risk-typed by GL (Appendix D).

3. Registered into Contract Registry.

4. Validated by Engine against Governance Kernel + sector overlays.

5. Activated with AB ceilings attached.

6. Re-audited on schedule or after any S2+ incident.

F.1.5 Contract Registry (system requirement)

The Engine maintains an append-only registry of active and historical contracts:

registration time,

governance signer,

version diff,

AB ceilings by node/cluster,

retirement reason.

Normative rule:

If a contract is modified, the old version must remain available for forensic replay (Appendix E).

F.2 Capability Indices (Normative + “Muscle”)

F.2.1 Agent Capability Index (ACI)

v0.5.2 references a capability and reliability tracking primitive.

We formalize this as:

$$ACI = \gamma_Q Q_a + \gamma_A A_a + \gamma_R R_a + \gamma_E E_a$$

Where:

: agent-quality contribution (episode-level, normalized)

: alignment fidelity contribution

: reliability under failure modes

: economic efficiency contribution

weights are cluster-specific.

ACI is computed per agent per domain slice (capability is not global).

F.2.2 Reliability term

Reliability is derived from:

tool error rates,

override/critic fail rates,

latency breach rates,

safety incident adjacency.

$$R_a = 1 - (p_{\{tool_fail\}} + p_{\{critic_fail\}} + p_{\{latency_breach\}} + p_{\{policy_near_miss\}})$$

F.2.3 Confidence calibration requirement

Agents must publish confidence scores that are empirically calibrated. Miscalibration incurs ACI penalties and AB ceiling locks.

F.2.4 Capability decay and drift

Across windows W :

$$ACI_{\{rolling\}}(t) = \alpha ACI_{\{episode\}}(t) + (1 - \alpha) ACI_{\{rolling\}}(t-1)$$

If decays while SI is stable, the Engine should:

down-route tasks to alternate agents,

trigger retraining or contract review.

F.2.5 Capability-to-routing rule

Planner chooses agents by maximizing expected SI gain per dollar:

$$\text{Agent}^* = \arg\max_i \frac{\mathbb{E}[\Delta SI_i]}{\text{cost}_i}$$

Subject to:

contract constraints,

AB ceilings,

SCP headroom.

F.2.6 Norm-aware capability (v0.5.2 integration)

Machine sociality / norm internalization research explicitly ties to Appendix F schemas.

Therefore contracts may include norm constraints, e.g.:

“NO_PRIVATE_COORDINATION” (no agent–agent direct channel),

“NO_COLLUSION_PATTERN” (flag correlated deviation),

“REPORT_EMERGENT_NORMS” (emit meta-telemetry).

F.3 MCP Integration — Wire Protocol (Normative)

F.3.1 Adoption of MCP

v0.8 adopts Model Context Protocol (MCP) as the standard for connecting Engine↔Agents↔Tools.

F.3.2 Architecture mapping

MCP Host (Client): Symbiosis Engine.

MCP Server: each agent.

Servers expose:

Resources (data URIs),

Prompts (pre-defined workflows),

Tools (executable functions).

F.3.3 Context federation

MCP allows the Engine to inject context from other agents into a target agent without direct agent–agent channels, reducing entanglement.

Normative rule:

Agents never exchange context directly unless a sector overlay explicitly allows it.

F.4 Inter-Agent Protocol Schemas (Normative)

F.4.1 Message types

v0.8 defines four canonical messages.

1. REQUEST (REQ)

Engine → Agent

Payload: Intent, Budget, Context_Pointer.

2. RESPONSE (RES)

Agent → Engine

Payload: Artifact, Usage_Stats, Confidence_Score.

3. STATUS (STAT)

Agent → Engine (heartbeat)

Payload: %Complete, Current_Step.

4. CONTROL (CTRL)

Governance/Engine → Agent

Payload: PAUSE, ABORT, ROLLBACK.

F.4.2 Governance handshake (pre-flight)

Before execution:

1. Engine sends REQ + Constraint_Vector.

2. Agent simulates task.

3. Agent returns ACK or REJECT if constraints cannot be satisfied.

4. Only on ACK does Engine unlock budget and allow tool calls.

Normative rule:

Skipping handshake is a governance violation ($\geq$ S3 incident).

F.4.3 Deterministic stop behavior

CTRL messages are non-negotiable. All agents must:

halt on ABORT,

checkpoint on PAUSE,

execute engineering rollback on ROLLBACK.

F.5 Arbitration & Conflict Resolution (Normative)

F.5.1 Why arbitration is required

Multi-agent systems will produce dissensus by design. Arbitration defines how conflict becomes safe progress rather than instability.

F.5.2 Lateral Critic Loop (v0.8)

Agent_A generates; Agent_B critiques against Constraint Vector:

PASS → proceed

FAIL → regenerate (max 3 retries)

persistent FAIL → ESCALATE_TO_HUMAN.

F.5.3 Vertical Voting Ensemble (v0.8)

3+ agents solve same task; Engine aggregates:

consensus $\geq 2/3$ → proceed

no consensus → Safe Mode (AB0).

F.5.4 Priority-based arbitration (v0.5.2)

When conflict persists:

higher-priority agents or governance cluster outputs dominate.

F.5.5 Tie-breaking and human override (v0.5.2)

If priority doesn't resolve:

Engine selects output best aligned to I₀/constraints.

if still ambiguous → human override.

Normative rule:

Arbitration must always defer to constraints first, intent second, economics third.

F.6 Tool Scope, RB, and Rollback Semantics (Added "Muscle")

F.6.1 Tool classification in contracts

Every tool declared by an agent must be typed:

Tool Type	Reversibility	RB Proximity	Required Controls
-----------	---------------	--------------	-------------------

T0 Informational	reversible	below RB	AB1 ok
T1 Mutative-Internal	reversible	below RB	AB1–2 ok
T2 External-Reversible	rollback-guaranteed	near RB	AB2 Safe Harbor only
T3 External-Irreversible	non-rollback	crosses RB	AB0 + Human token

F.6.2 Rollback contract

For any T2 tool, the contract must include:

rollback endpoint/method,

max rollback TTL,

verification step.

If rollback is undefined, the tool is treated as T3.

F.7 Security, Sandboxing, and Entanglement Controls (Added “Muscle”)

F.7.1 Sandbox requirement

Agents may only execute within:

capability sandbox (tool allowlist),

memory sandbox (tier-only access),

economic sandbox (budget caps).

F.7.2 Entanglement minimization

No direct agent sockets.

Shared memory access requires Engine mediation.

Cross-agent resource writes are lineage-logged at Tier 3.

This is consistent with MCP federation's entanglement-reduction objective.

F.8 Interface to Autonomy, SI, HSS/SCP

AB ceilings: derived from risk class + contract tool typing. (Appendix C/D)

SI attribution: subscores attribute per-agent deltas to update ACI. (Appendix B)

HSS/SCP coupling: low-SCP states throttle agent concurrency regardless of ACI. (Appendix H)

The contract/capability stack is therefore foundational to autonomy governance.

F.9 Common Failure Patterns (Anti-Patterns)

1. Contract drift: agents updated without contract version bump.
2. Implicit tools: hidden capabilities not declared in contract.
3. Confidence inflation: high confidence without calibration data.
4. Arbitration bypass: selecting preferred agent output without critic/vote.
5. Entanglement leakage: agents coordinating outside MCP.

6. Economic bleed: high-tier agents used for low-tier tasks.

Each is a reportable governance violation.

F.10 Implementation Checklist (Minimum Compliance)

An organization is Appendix-F compliant only if:

☐ Every agent has a registered, validated contract.

☐ Contract schema includes capabilities, constraints, economics, interfaces, failure modes.

☐ MCP Host/Server topology enforced.

☐ REQ/RES/STAT/CTRL message types implemented.

☐ Governance handshake blocks execution on REJECT.

☐ Critic Loop + Voting Ensemble arbitration present and logged.

[] Priority/tie-break + human override escalation exists.

[] ACI tracked per agent per domain slice, with reliability and calibration.

[] Tool reversibility/RB typing and rollback semantics declared.

[] Entanglement controls prevent shadow coordination.

[] Norm-aware contract fields supported for multi-agent behavior.

Appendix F — Agent Contracts, Capability Indices, and Protocol Schemas (Full-Length v0.9)

F.0 Purpose and Position in the Framework

Agents are not “apps.” In Symbiosis, they are **governed, swappable capability modules** operating inside a layered hybrid system. Appendix F defines the formal objects that make this safe and scalable:

1. **Agent Contracts** — explicit boundaries and obligations for each agent.
2. **Capability Indices (CI)** — machine-readable competence profiles used for routing.

3. **Protocol Schemas** — the technical and conversational wires that prevent entanglement and enforce policy.

v0.8 treats these as foundational to the Agentic Layer and MCP-based interoperability.
v0.5.2 emphasizes that contracts + indices + clean protocols are the accountability backbone and the basis for conflict arbitration.

Normative claim:

No agent can be invoked by the Symbiosis Engine unless it is registered with a valid contract and CI and is reachable through an approved protocol.

F.1 Agent Contracts (The Boundary Object)

F.1.1 Definition

An **Agent Contract** is a structured document declaring what an agent can do, under which constraints, using which tools, with what economic profile, and how failures are handled.
v0.8 supplies the canonical YAML/JSON-like contract format.

F.1.2 Minimum contract fields (normative)

Every contract must include:

1. **Identity / Spec**

- agent_name
- version
- provider / model tier
- context window
- update lineage pointer

2. **Economics**

- cost_per_1k_tokens (or equivalent unit)
- average_latency_ms
- max_project_spend default envelope

3. **Capabilities**

- list of capability names
- description
- **input_schema** (machine-validated)
- **output_schema**
- reliability grade (if available)

4. **Constraints**

- **hard constraints:** non-negotiable (e.g., **NO_TRADE_EXECUTION**)

- **soft constraints:** preference-weighted (e.g., `CITE_SOURCES`)

5. Interfaces

- protocol (MCP/...)
- transport (SSE, HTTP, local IPC, etc.)
- endpoint URI
- auth method (opaque to agent; held by Engine)

6. Failure Modes

- explicit error taxonomy (`DATA_UNAVAILABLE`, `AMBIGUOUS_INPUT`, `TOOL_FAIL`, etc.)
- declared behaviors (`RETURN_NULL`, `REQUEST_CLARIFICATION`, `ESCALATE_TO_HUMAN`)

Normative rule:

If a capability or tool path is not in the contract, it does not exist.

F.1.3 Contract Registry

The Symbiosis Engine maintains a **Contract Registry** of all active agents. Registration is mandatory at deploy time, and the Engine must validate compatibility with the Governance Kernel. For example, an agent cannot register “Execution” rights inside a Discovery cluster if GK forbids it.

F.1.4 Contract lifecycle

Contracts evolve under **Slow-Core change rules** (Chapter 11):

- **Draft (HIA):** propose new capabilities/tools/constraints.
- **Validate (Engine + GK):** preflight static checks.
- **Approve (GL):** policy-authoritative signing.
- **Activate:** enters registry with version hash.
- **Deprecate:** old contracts remain addressable for lineage replay.

Any contract update must emit a lineage block (Appendix E).

F.2 MCP Integration (Wire Protocol)

F.2.1 Why MCP

v0.8 adopts **Model Context Protocol (MCP)** as the standard interface between Engine and Agents/Tools to enforce hub-and-spoke control, prevent agent-agent entanglement, and standardize auth/context injection.

F.2.2 MCP role mapping

- **MCP Host (Client):** Symbiosis Engine.
- **MCP Server:** Agent.
 - exposes **Resources, Prompts, Tools**.

F.2.3 Context federation rule

Agents never talk to each other directly. They federate context only through the Engine: Debugger exposes data as a Resource; Engine passes URI to Coder, etc. This reduces entanglement and keeps lineage centralized.

Normative rule:

Any direct agent-agent channel is a governance violation unless explicitly classified and isolated in a sector matrix.

F.3 Symbiosis Inter-Agent Protocol (Conversation Protocol)

F.3.1 Message types

While MCP defines the wire, Symbiosis defines the structured conversation:

- **REQUEST (REQ):** Engine triggers agent.
Payload: Intent, Budget, Context_Pointer.
- **RESPONSE (RES):** Agent returns artifact + stats + confidence.
- **STATUS (STAT):** heartbeat for long tasks.
- **CONTROL (CTRL):** governance overrides: PAUSE, ABORT, ROLLBACK.

F.3.2 Governance handshake (preflight)

Before any execution beyond trivial drafting, v0.8 requires:

1. Engine sends REQ + active Constraint Vector.
2. Agent **simulates** task.
3. Agent replies:
 - **ACK** if satisfiable
 - **REJECT** if constraints would be violated
4. Only on ACK does Engine unlock budget and proceed.

Normative rule:

A REJECT is not negotiable in-flight; it requires intent/constraint revision or escalation.

F.4 Arbitration & Conflict Resolution

v0.8 formalizes multi-agent disagreement handling so that conflict does not become hidden drift.

F.4.1 Critic loop (lateral)

- Generator produces draft.
- Critic tests against Constraint Vector and ASR_o.
- Up to 3 retries; then **ESCALATE_TO_HUMAN**.

F.4.2 Voting ensemble (vertical)

- 3+ agents run same task.
- Engine aggregates; 2/3 consensus passes.
- No consensus $\Rightarrow$ Safe Mode (AB0).

F.4.3 Priority arbitration (v0.5.2)

When conflicts persist:

- Governance/oversight clusters outrank execution clusters.
 - Tie-breaks prefer outputs maximizing alignment to I_o and C.
-

F.5 Capability Indices (CI)

F.5.1 Definition

A **Capability Index** is the routing-grade profile of an agent:

CI = \{domain,\ task_types,\ reliability,\ evidence_grade,\ cost_class,\ latency_class\}

CI is owned by governance and updated through SI-linked evidence.

F.5.2 CI uses

- **Routing:** Engine selects agents by CI fit under AB/risk ceilings.
- **Budgeting:** economic optimizer weights cost_class.
- **Stability control:** low-evidence CI triggers conservative AB.

F.5.3 CI evolution

CI evolves only via Tactical/Strategic loops:

- repeated high SI episodes $\Rightarrow$ CI reliability increases.
 - repeated failures or drift $\Rightarrow$ CI downgraded or contract narrowed.
-

F.6 Security, Safety, and Compliance Rules (Normative)

1. **No undeclared tools.**
2. **No side effects without rollback semantics declared.**
3. **All agent outputs must include confidence unless forbidden by matrix.**
4. **All external endpoints must be GK-approved and residency-compliant.**
5. **Contract/CI drift is slow-core; hot patches are disallowed.**

Violations are S3+ incidents by default.

F.7 Implementation Checklist (Minimum Compliance)

- [] Contract format includes all F.1.2 fields.
 - [] Contracts registered and validated against GK.
 - [] MCP hub-and-spoke topology enforced.
 - [] Governance handshake required pre-execution.
 - [] Critic loops / ensembles for conflict resolution active.
 - [] CI routing and CI evolution tied to SI evidence.
 - [] All protocol events logged into lineage.
-

Appendix G — Co-Evolution & Stability Models (Equations & Simulations) (Full-Length v0.9)

G.0 Purpose and Position in the Framework

Symbiotic systems are **complex adaptive systems**: humans learn, agents change, and constraints evolve. v0.8 elevates co-evolution from metaphor to an engineering discipline with explicit stability “vital signs.”

Appendix G formalizes:

1. **Co-evolutionary dynamics** of Human (H) and Agent (A) states.
2. **Stability metrics**: Drift, Divergence, Resonance, Entanglement.

3. **Simulation protocols** for forecasting future regimes and testing governance.

Normative claim:

Autonomy scaling above AB1 is invalid unless these stability metrics are monitored and wired into control loops.

G.1 Co-Evolutionary Dynamics

G.1.1 Coupled state model (v0.5.2)

Let:

- H : human state vector (skill, trust calibration, attention, mental model).
- A : agent/system state vector (model weights, contract set, tools, prompts).

Co-evolution is represented as **coupled differential equations**:

$$\frac{dH}{dt} = f(H, A) + \epsilon_H$$

$$\frac{dA}{dt} = g(H, A) + \epsilon_A$$

capture stochastic disturbances (org shocks, model updates, staffing changes, etc.).

G.1.2 Equilibrium conditions

A stable symbiotic equilibrium occurs when:

$$f(H^*, A^*) = 0 \quad \text{and} \quad g(H^*, A^*) = 0$$

This is not “no change,” but **bounded change around a stable attractor**.

G.1.3 Three adaptation time-loops (v0.8)

v0.8 defines three coordinated loops:

1. **Operational loop (minutes)**: Engine throttles for fatigue/SCP breaches.
2. **Tactical loop (weeks)**: Governance reviews SI/stability trends; adjusts AB/contracts.
3. **Strategic loop (months)**: ontology and sector policies update; new models inserted.

Normative rule:

Uncoordinated adaptation across loops produces instability even if each change is “locally rational.”

G.2 Stability Metrics (Vital Signs)

G.2.1 Drift (D)

Definition (v0.8): slow cumulative deviation from baseline intent, often masked by short-term success.

Operationally:

$$D(t) = |Execution(t) - ASR_0|_{\{intent\}}$$

Detection signature:

- Alignment (A subscore) decays while Quality (Q) stays stable.

Control coupling:

Drift yellow-band $\Rightarrow$ intent refresh + critic hardening.

Drift red-band $\Rightarrow$ AB demotion + Bridge revocation.

G.2.2 Divergence (V)

Definition (v0.5.2): separation between human goal vector and agent goal vector.

$$V(t) = |Goal_H(t) - Goal_A(t)|$$

High divergence implies the system is optimizing for something other than human meaning.

Control coupling:

V red-band $\Rightarrow$ forced re-anchor of I_0 and contract narrowing.

G.2.3 Resonance (R)

Definition (v0.5.2): degree of synchronization and mutual reinforcement of goals.

$$R(t) = \frac{H(t) \cdot A(t)}{|H(t)| \cdot |A(t)|}$$

R near 1 = aligned collaboration; R near 0 = weak coupling.

Interpretation nuance (“muscle”):

- **Low R + low V:** early learning / weak coupling.
 - **High R + rising V:** dangerous “wrong-path resonance” (lock-in to misaligned trajectory).
-

G.2.4 Entanglement (E_blast)

Definition (v0.8): blast radius created by shared tools, memory, policies, and contracts across teams/agents.

A practical proxy:

$$E_{\text{blast}}(t) = \sum_{k \in \text{shared_resources}} \text{criticality}(k) \cdot \text{fanout}(k)$$

Where *fanout* counts dependent workflows and *criticality* weights by regret class.

Control coupling:

E_blast yellow-band $\Rightarrow$ slow-core caps, forced contract standardization.

E_blast red-band $\Rightarrow$ AB $\geq$ 2 promotion freeze system-wide.

G.3 Thresholds and Banding

Each metric is banded (green/yellow/red) per sector, with conservative defaults:

- **Class A sectors:** tight drift and divergence thresholds.
- **Ambiguity sectors:** higher drift tolerance but tight entanglement rules.
- **Economic-sensitive sectors:** resonance volatility tolerated only if SI remains green.

Thresholds are part of sector playbooks (Chapter 17) and Appendix K.

G.4 Simulation Framework

G.4.1 Purpose

Simulations forecast whether co-evolution will converge toward higher SI or diverge into failure. v0.5.2 specifies simulation as a core method.

G.4.2 Setup

Minimum simulation includes:

- initial H and A states (skill, trust, capability indices),
- feedback functions f and g (estimated from pilot data),
- stochastic disturbance terms,
- governance interventions (AB changes, constraint tightening).

G.4.3 Outputs

Simulations must report:

- long-term alignment trends,
- drift/divergence/resonance trajectories,
- stability warning points and predicted interventions.

G.4.4 Stress tests (normative)

Before AB3 scale in any domain, simulate:

1. **Intent drift injection**
2. **Economic bleed shock**
3. **Trust collapse / override spikes**
4. **Shared-tool entanglement event**
5. **Model swap discontinuity**

If any test produces unstable attractors, AB3 is disallowed until mitigations exist.

G.5 Control Integration with SI / AB / HSS-SCP

- **SI** supplies performance-based control; stability metrics supply **trajectory control**.
- AB promotions require stability green band in addition to SI green band (Appendix C).
- SCP overload is a leading indicator for drift and false resonance.

Normative rule:

If stability and SI disagree, **stability wins** for safety (you demote).

G.6 Common Co-Evolution Failure Modes

1. **Silent drift:** alignment decay accepted as “new normal.”
2. **Wrong-path resonance:** agent and human reinforce a mis-specified objective.
3. **Divergence under fatigue:** SCP overload prevents correction, widening V.
4. **Entanglement cascades:** shared contracts/tools spread local errors globally.

5. **Loop desynchronization:** rapid agent upgrades without human training or policy compilation.

Each failure mode is overlaid into incident doctrine (Appendix O).

G.7 Implementation Checklist (Minimum Compliance)

- [] Coupled H/A dynamics recognized in governance.
 - [] Drift, divergence, resonance, entanglement computed and banded.
 - [] Stability green band required for $AB \geq 2$ promotions.
 - [] Tactical/strategic loops operational.
 - [] Co-evolution sims run before AB3 scale.
 - [] Stability red-band auto-demotes AB and freezes promotions.
-

Below is a full-length **Appendix H** that fuses v0.8's skeleton (clean engineering spec, pilot/attention-budget framing, ethical firewall, tiering) with v0.5.2's flesh (complete dimension set, examples, trajectories, operational uses). also added "further muscle" in clearly marked *Enhancement Items* so you can decide what becomes normative vs. optional.

Appendix H — Human Symbiosis Score (HSS) & Symbiotic Capacity Profile (SCP)

Status: Normative for certification and autonomy gating; developmental for operator improvement.

Scope: Human Layer, Symbiosis Engine telemetry, Governance Layer decision rights.

H.0 Purpose

HSS and SCP are the human-side control instruments of the Symbiosis Framework.

- **HSS ("Hiss")** measures *operator skill* at steering the Engine and collaborating with agents.
- **SCP ("Skip")** measures *operator load-bearing capacity* and safe supervisory bandwidth.

They are not HR productivity scores; they are *safety telemetry* used to scale autonomy without collapsing human agency.

H.1 Human Symbiosis Score (HSS): Skill Model

H.1.1 Definition

HSS is a composite score in derived from observable behaviors in Context Lineage and structured evaluations.

It answers: “**How effective is this pilot at flying this symbiotic aircraft?**”

H.1.2 Dimensions (v1.1)

Seven skill dimensions (each in):

1. **Intent Clarity & Structuring (C_t)**
 - Precision of intent, completeness of constraints, definition-of-done quality.
2. **Horizon Management & Complexity Framing (H_t)**
 - Ability to scope tasks at appropriate temporal/complexity horizons.
3. **Feedback & Override Quality (F_t)**
 - Informational value of overrides; “teaching” the system via rationale.
4. **Cognitive Load Management & Delegation (L_t)**
 - Appropriate delegation to agents, avoidance of oversight theater.
5. **Memory Externalization & Artifact Hygiene (M_t)**
 - Clean capture of reusable artifacts, structured context deposits.
6. **Alignment & Governance Literacy (A_t)**
 - Adherence to safety protocols; correct use of constraints/budgets.
7. **Reflective Practice & Learning Loops (R_t)**
 - Post-episode reflection, pattern extraction, deliberate calibration.

H.1.3 Behavioral Anchors (BARS approach)

Scores should be assigned using a **Behaviorally Anchored Rating Scale**, emphasizing *process* over output.

Example anchors:

- **C_t low:** vague prompts, missing constraints (“Fix this”).
- **C_t high:** structured specs with success criteria (“Refactor module X... latency <50ms”).
- **F_t low:** silent rejection / “Try again.”
- **F_t high:** targeted correction with reason (“Line 40 violates Idempotency Constraint”).
- **A_t low:** frequent Safety Supervisor warnings / bypass attempts.
- **A_t high:** proactive constraint use, budget compliance.

Pilot analogy is mandatory framing: HSS is a *clearance rating*, not a performance review.

H.2 HSS Computation

H.2.1 Composite Equation

$$\text{HSS}_t = \sum_{i \in \{C,H,F,L,M,A,R\}} w_i \cdot i_t, \quad \sum w_i = 1$$

H.2.2 Time-scale Smoothing

To avoid noise and gaming, compute **rolling HSS** over a fixed “flight hours” window (e.g., 30 days).

H.2.3 Example Trajectory

v0.5.2 provides a sample operator trajectory showing HSS growth from $\sim 0.58 \rightarrow \sim 0.77$ over a 12-week governed pilot, with largest gains in C_t , F_t , M_t under targeted training.

H.3 Symbiotic Capacity Profile (SCP): Load-Bearing Envelope

H.3.1 Definition

SCP characterizes the **safe supervision bandwidth** of a human operator. It answers: “How many concurrent task graphs and autonomy conditions can this human safely hold?”

H.3.2 Dimensions (Capacity Axes)

Five dimensions in .

1. **Complexity Load Capacity (CL)**
 - Ability to handle deep reasoning / multi-step planning domains.
2. **Concurrency Load Capacity (CON)**
 - Safe number of parallel significant workflows/agents supervised.
 - Example bands:
 - CON-1: 1–2 concurrent moderate-risk workflows
 - CON-3: several moderate-risk or mixed moderate/low
 - CON-5: many low-risk workflows in structured domains
3. **Temporal Horizon Handling (TH)**
 - Coherent reasoning over days–weeks vs. months–years.
4. **Abstraction Handling (AB)**
 - Moving between concrete, architectural, and normative levels.
5. **Governance Reasoning Depth (GR)**
 - Capacity to reason about risk, ethics, policy tradeoffs, and SI/HSS/SCP signals.

H.3.3 Composite Equation

$$\text{SCP} = w_{\{\text{CL}\}} \text{CL} + w_{\{\text{CON}\}} \text{CON} + w_{\{\text{TH}\}} \text{TH} + w_{\{\text{AB}\}} \text{AB} + w_{\{\text{GR}\}} \text{GR}$$

H.3.4 SCP Circuit Breaker (Attention Budget)

The Engine enforces SCP limits via an **Attention Budget**:

- If **Active_Interventions** > **Operator_SCP_Limit**, then **THROTTLE_AGENTS**.
 - Mechanism: queue/batch low-priority interrupts into digests, preventing alert fatigue and oversight theater.
-

H.4 Operator Tiers & Autonomy Rights

H.4.1 Tier Table (Normative)

Operators are certified into tiers that gate autonomy bands.

- **L1 Cadet**: HSS <0.5, SCP low → **Band 0 only**, countersign required.
- **L2 Pilot**: HSS 0.5–0.8, SCP medium → **Bands 1–2** within SCP budget.
- **L3 Commander**: HSS >0.8, SCP high → **Band 3** and high-stakes override rights.

H.4.2 Promotion Logic

Tier progression is automatic based on rolling HSS (“flight hours” model), subject to governance veto in high-risk domains.

H.5 Ethical Firewall: Non-Punitive Covenant (Mandatory)

H.5.1 Rationale

If HSS/SCP are used for compensation, hiring, or firing, humans will game signals and destroy data quality.

H.5.2 Data Wall

- HSS/SCP stored in a **Safety Database**, physically separated from HR systems.
- Read access: operator (self-improvement), governance leads (system tuning).

- No access: line managers, HR, compensation committees.

H.5.3 Near-Miss Incentive

Reward catching failures via high F_t , even if throughput drops. Incentivize **the catch, not the speed**.

H.6 Joint Use in Autonomy, Task Design, and Governance

H.6.1 Autonomy Gating

Autonomy regimes depend on:

1. **SI of workflow** (system health)
2. **HSS of responsible operator** (skill)
3. **SCP of operator** (capacity envelope)

H.6.2 Workload & Responsibility Matching

Use SCP to:

- cap high-risk workflow concurrency per human,
- allocate supervision fairly,
- avoid chronic overload.

H.6.3 Training Programs

Implementation requires explicit HSS/SCP development programs: baseline assessment, role-specific profiles, targeted modules by dimension, coaching embedded in operating rhythm.

H.6.4 Longitudinal Co-Evolution

Tracking SI with HSS/SCP over time enables co-evolution analysis and safe autonomy elevation.

H.7 Implementation Artifacts (What teams must produce)

1. **Role-based HSS profiles:** minimum/target per role (Operator, Architect, Governance Lead).

2. **SCP capacity sheets:** CL/CON/TH/AB/GR ranges per role and domain.
 3. **Autonomy band mapping tables:** SI/HSS/SCP ranges → Bands by workflow/risk class.
 4. **Safety telemetry charter:** signed Non-Punitive Covenant + data wall enforcement.
 5. **Pilot telemetry dashboards:** SI by workflow, HSS trends per operator, “load vs capacity” SCP view.
-

Appendix H — Proposed v1.2 Additions (Fully Written Subsections)

H.8 Domain Calibration Coefficients (DCC) for HSS and SCP

Status: Proposed v1.2 (strongly recommended for defensibility)

H.8.1 Rationale

A raw HSS or SCP value is not universally comparable across domains. A clinician operating AB1 triage and a marketing lead operating AB1 content workflows may report numerically similar HSS values, yet the **risk and complexity envelope** they operate within is structurally different. Without calibration, autonomy decisions risk becoming distorted by domain triviality or domain severity.

H.8.2 Definition

Introduce **Domain Calibration Coefficients**:

- for HSS
- for SCP

These coefficients are derived from the sector’s **Risk Class distribution, complexity norms**, and **regret posture** (Appendix D/K). Values are bounded so they do not inflate scores beyond .

$$HSS^* = \min(1, k_{H, \text{domain}} \cdot HSS)$$

$$SCP^* = \min(1, k_{S, \text{domain}} \cdot SCP)$$

H.8.3 Calibration Method

Each sector playbook must include:

1. **Domain risk profile** (fraction of A/B/C tasks).
2. **Complexity baseline** (CL median for typical work).
3. **Critical non-delegables count** (RB density).

4. Historical SI volatility baseline.

Governance then assigns values using a standardized rubric, with conservative defaults for safety-critical domains.

H.8.4 Operational Use

- Autonomy gates reference calibrated scores and .
- Cross-domain comparisons in talent development must use calibrated scores only.
- Certification tiers remain unchanged; calibration affects **band rights in-domain**.

H.8.5 Defensibility Benefit

DCC prevents the predictable criticism that “HSS is a soft, domain-blind score.” It ensures that autonomy is earned **in the risk envelope where it will be exercised**, not elsewhere.

H.9 Dual-Signal Intent Clarity (C_struct + C_context)

Status: Proposed v1.2 (recommended)

H.9.1 Rationale

The current Intent Clarity dimension is highly predictive of SI stability, but it mixes two separable skills:

1. **Structured articulation** (what to do, constraints, definition of done)
2. **Contextual grounding** (using relevant artifacts and lineage to avoid drift)

Operators can score well on one while failing on the other; conflation hides failure modes and weakens training targeting.

H.9.2 Decomposition

Split into:

- : quality and completeness of structured intent fields.
- : quality of grounding via canonical memory, project artifacts, and lineage.

$$C_t = \eta \cdot C_{\text{struct}} + (1-\eta) \cdot C_{\text{context}}$$

Where defaults to 0.6 unless sector playbooks specify otherwise.

H.9.3 Behavioral Anchors

- **Low** : missing constraints, unclear output formats, partial DoD.
- **High** : explicit constraints, success metrics, budget/horizon defined.
- **Low** : ignores lineage/canonical standards; repeats solved errors.

- **High** : cites relevant memory tiers; reuses stable artifacts; shows continuity discipline.

H.9.4 Training Use

Training programs can now target failures precisely:

- deficiencies → intent-structuring drills.
- deficiencies → artifact hygiene and lineage literacy modules.

H.9.5 Control Use

Governance may set minimum floors separately. Example:

- Healthcare may require for AB2 safe harbors even if is 0.9.

H.10 Interrupt Tolerance (IT) as a Sixth SCP Axis

Status: Proposed v1.2 (high value for scale)

H.10.1 Rationale

SCP currently models capacity across complexity, concurrency, horizon, abstraction, and governance depth. In practice, **failure often happens via interruption overload**: pings, critic flags, tool errors, and escalations arriving faster than humans can adjudicate.

This yields oversight theater and hidden drift even in skilled operators.

H.10.2 Definition

Add a sixth SCP axis:

- **Interrupt Tolerance (IT)**: resilience to high-frequency intervention demands without degradation of judgment quality.

$SCP = \sum w_i$ where $i \in \{CL, CON, TH, AB, GR, IT\}$

H.10.3 Measurement

IT is observable from lineage and UI telemetry:

- median response latency to high-priority interventions,
- override quality under burst conditions (F_t stability),
- error-rate spike under interrupt load,
- frequency of “rubber-stamp approvals” during bursts.

IT is scored rolling-window like other SCP axes.

H.10.4 Engine Coupling

When IT trends downward, the Engine must:

- batch low-priority interrupts into digests,
- delay non-urgent critic loops,
- reduce agent concurrency before a hard SCP breach occurs.

H.10.5 Defensibility Benefit

IT explicitly connects SCP to real supervisory mechanics, strengthening your claim that SCP measures **load-bearing capacity**, not abstract “cognitive load.”

H.11 HSS Failure-Mode Taxonomy (Human-Side Incident Map)

Status: Proposed v1.2 (recommended for audit parity)

H.11.1 Rationale

Symbiosis already defines system-side incident taxonomies (Chapter 13 / Appendix O). Without an equivalent human-side taxonomy, failures appear “mysterious,” and training becomes non-diagnostic.

H.11.2 Taxonomy

Define canonical HSS failure types by dimension:

1. **C-Failures (Intent Defects)**
 - missing constraints → drift
 - ambiguous scope → divergence
2. **H-Failures (Horizon Defects)**
 - wrong time/complexity framing → economic bleed or low Q
3. **F-Failures (Feedback Defects)**
 - silent rejection, non-informative overrides → brittle learning; SI instability
4. **L-Failures (Delegation/Load Defects)**
 - over-delegation → RB breaches
 - under-delegation → throughput collapse and shadow AI relapse
5. **M-Failures (Artifact Hygiene Defects)**
 - storing junk in canonical/project memory → poisoning and drift
6. **A-Failures (Governance Defects)**
 - boundary pushing, shortcutting AB gates → S3+ events
7. **R-Failures (Reflection Defects)**
 - no post-episode reconciliation → repeated errors

H.11.3 Severity Coupling

Some HSS failures are **direct precursors** to system incidents:

- A-Failure → immediate governance intervention.
- C-Failure persistent → drift red-band risk.
- L-Failure under fatigue → oversight theater.

These are mapped into Appendix O triggers.

H.11.4 Training and Control Use

- Training uses the taxonomy to prescribe **dimension-specific remediation drills**.
- Governance uses frequency trends to anticipate drift before SI collapses.

H.11.5 Defensibility Benefit

This gives regulators and enterprise risk committees a clear story:

human-side failures are typed, detected, and controlled equivalently to system failures.

H.12 Certification Protocol v1.2 (Formal “Flight Hours” Standard)

Status: Proposed v1.2 (essential if you want external acceptance)

H.12.1 Rationale

Chapter 16 defines certification tiers; this subsection formalizes the evaluation process into a **repeatable standard**. Without protocolization, certification risks being interpreted as managerial taste instead of governance discipline.

H.12.2 Requirements

To certify or recertify a Symbiote Operator:

1. **Observed Flight Hours (minimum N episodes)**
 - L1: $N \geq 20$ AB0 episodes
 - L2: $N \geq 40$ AB0/1 episodes with stable SI
 - L3: $N \geq 60$ episodes including AB2 safe harbors supervised
2. **Scenario Stress Tests**
 - injected drift
 - critic disagreement
 - economic circuit pressure
 - interrupt burst load
 - RB-adjacent decision requiring explicit human token
3. **Metric Evidence Slice**
 - rolling HSS^{*} (domain-calibrated)
 - SCP^{*} with IT axis
 - no S3+ incidents in last W

- SI stability in supervised workflows
4. **Governance Sign-off**

- required at L2 and L3 elevations
- dual-key required for AB3 rights in high-regret sectors

H.12.3 Recertification Cadence

- Standard cadence: **18–24 months**
- Accelerated recertification required after:
 - any S3+ incident involvement,
 - major sector policy shift,
 - long role hiatus.

H.12.4 Rights Coupling

Certification explicitly binds to AB rights:

- L1 → AB0 only
- L2 → AB1–AB2 in safe harbors
- L3 → AB3 in bounded domains where SI/stability remain green

The Engine must enforce this coupling automatically.

H.12.5 Defensibility Benefit

This turns certification into an **externally legible licensure standard**, aligning with your “new field” objective. It also keeps autonomy authorization visibly accountable.

Below is **Appendix I — Symbiosis Literacy Program Syllabus (Full-Length v0.9)**, expanded from the v0.5.2 outline and aligned to v0.8’s operator-tier + autonomy model. It is written as a deployable curriculum with role tracks, assessments, and explicit coupling to HSS/SCP. Source anchor: the SLP is defined as the mandatory literacy/training layer for safe collaboration and a prerequisite for critical roles.

Appendix I — Symbiosis Literacy Program (SLP) Syllabus (Full-Length v0.9)

I.0 Purpose and Position in the Framework

The Symbiosis Literacy Program (SLP) is the human-capital counterpart to the Engine and Governance layers. Its purpose is to ensure that humans are not “users of AI tools,” but **certified symbiotes capable of safe, powerful hybrid work**.

SLP provides structured training on:

- reading and acting on SI/HSS/SCP signals,
- expressing high-quality intent surfaces,
- recognizing drift and failure modes,
- understanding autonomy bands and constraints,
- practicing safe escalation and override.

Normative claim:

No organization can claim Symbiosis compliance at AB≥1 without an SLP track mapped to its roles and domains.

I.1 Program Overview

I.1.1 Target audiences

SLP is modular and role-differentiated. Baseline audiences include:

1. **Symbiote Operators (Pilots):** frontline humans in-the-loop.
2. **Hybrid Intelligence Architects (Engineers):** task-graph designers, orchestration tuners.
3. **Governance Leads (Controllers):** risk owners, constraint authors, auditors.
4. **Executives / Sponsors:** decision-makers for adoption and policy posture.
5. **General Staff / Domain Experts:** humans collaborating on bounded tasks.

I.1.2 Delivery structure

SLP has three layers:

- **Core Literacy (everyone):** shared language and primitives.
- **Role Tracks (specialized):** Operator / Architect / Governance.
- **Domain Overlays (sector-specific):** healthcare, finance, SME ops, education, defense, etc.

I.1.3 Program outcomes

Graduates can:

- articulate intent surfaces with complete constraints,
- supervise agents without oversight theater,
- interpret SI and stability vital signs,
- justify autonomy posture,

- participate in co-evolution loops and safe scaling.
-

I.2 Core Literacy Modules (Required for all roles)

Module CL-1 — Foundations of Symbiosis

Objectives

- Understand why unstructured AI adoption fails.
- Distinguish Symbiosis vs automation vs no-AI.
- Internalize human primacy + constraint supremacy invariants.

Key concepts

- Four-layer architecture (Human / Engine / Agentic / Governance).
- Epistemic boundaries and regret boundaries.
- Co-evolution framing.

Assessment

- Short case write-up: identify where a real AI use-case violates invariants.
-

Module CL-2 — Symbiosis Ontology & Core Primitives

Objectives

- Use canonical vocabulary consistently.
- Understand primitives as “atoms” of the framework.

Key concepts

- Intent surfaces
- Constraint vectors (hard/soft/econ)
- Autonomy bands/regimes
- Hybrid cognitive loops
- Context lineage
- Multi-tier memory
- Regulatory confidence intervals

Assessment

- Ontology exercise: map a workflow into primitives.
-

Module CL-3 — Intent Interfaces & Specification Discipline

Objectives

- Move from prompting to engineering-grade specification.
- Avoid ambiguity and hidden scope creep.

Key concepts

- C_struct / C_context (Appendix H v1.2)
- Definition-of-done patterns
- Negative constraints
- Budget and horizon declaration

Assessment

- Rewrite three “vague prompts” into structured intent surfaces and compare SI outcomes.
-

Module CL-4 — Autonomy Bands, Regret Boundaries, and Safe Delegation

Objectives

- Correctly choose autonomy levels based on stakes and evidence.
- Identify non-delegables.

Key concepts

- AB0–AB3 rights and triggers
- RB-crossing actions
- Safe harbors and circuit breakers
- Role segmentation

Assessment

- Autonomy classification drill: label example tasks by AB and justify.
-

Module CL-5 — Metrics Literacy (SI / HSS / SCP)

Objectives

- Read system health like instruments.
- Treat human-side metrics as safety telemetry.

Key concepts

- SI dimensions and failure signatures
- HSS skill dimensions + failure taxonomy
- SCP load-bearing envelope + Attention Budget
- Ethical firewall (non-punitive covenant)

Assessment

- Scenario: SI dips while Q stays high. Identify drift vs divergence and recommend action.
-

Module CL-6 — Failure Modes and Recovery Thinking

Objectives

- Recognize predictable collapse patterns.
- Know when to throttle, rollback, or re-anchor intent.

Key concepts

- System failures (hallucination, tool fail, entanglement cascades)
- Human failures (C-, F-, L-, A-failures)
- Recovery playbooks
- Escalation ladder

Assessment

- Post-mortem simulation: produce a lineage-rooted incident brief.
-

Module CL-7 — Co-Evolution & Long-Horizon Symbiosis

Objectives

- Treat symbiosis as a living system.
- Understand adaptation loops and stability vital signs.

Key concepts

- Drift / divergence / resonance / entanglement
- Operational vs tactical vs strategic loops
- Simulation as governance method

Assessment

- Build a simple co-evolution forecast for a pilot deployment.
-

I.3 Role Tracks

I.3.A Symbiote Operator Track (Pilot Track)

Goal: Increase HSS, stabilize SCP, earn AB rights.

OP-1 — Operator Flight Discipline

- Structured intent habits
- Feedback as teaching (F_t)
- Artifact hygiene (M_t)

Assessment: BARS scoring across 10 supervised episodes.

OP-2 — Supervisory Judgment Under Load

- Concurrency management
- Interrupt tolerance (IT axis)
- Avoiding rubber-stamp approvals

Assessment: burst-load supervision simulation.

OP-3 — Override & Escalation Mastery

- When to use CTRL messages
- How to write high-value overrides
- Human-token discipline at RB boundaries

Assessment: graded override log with rationale quality.

OP-4 — Tier Certification Path

- L1 Cadet → L2 Pilot → L3 Commander
- Flight-hours requirement
- Scenario stress tests (Appendix H.12)

Assessment: formal certification board.

I.3.B Hybrid Intelligence Architect Track (Engineer Track)

Goal: Design stable task graphs and orchestration that maximize SI per dollar.

HA-1 — Task-Graph Design

- hierarchical decomposition
- dependency typing
- safe concurrency patterns

Assessment: design TG for a real workflow, then simulate SI stability.

HA-2 — Agent Routing & Contracts

- capability indices
- contract supremacy and tool typing
- arbitration hooks

Assessment: build routing spec that meets AB/regret criteria.

HA-3 — Memory & Lineage Engineering

- tiered memory selection
- retrieval waterfalls
- lineage block integrity

Assessment: “debug a drifted workflow via lineage.”

HA-4 — Economic Circuit Optimization

- tiered model economics
- cost/utility ratio
- bleed prevention

Assessment: reduce pilot cost $\geq 50\%$ without SI loss.

I.3.C Symbiosis Governance Lead Track (Controller Track)

Goal: Own regret boundary, policy engine, and audit readiness.

GL-1 — Constraint Authoring

- hard vs soft constraints
- jurisdiction + sector overlays
- conflict resolution rules

Assessment: author a Constraint Vector and test against cases.

GL-2 — Autonomy Governance

- AB regime selection
- promotion/demotion triggers
- safe harbor definitions

Assessment: autonomy playbook for a pilot domain.

GL-3 — Audit & Incident Doctrine

- evidence requirements

- lineage replay
- incident taxonomy + severity

Assessment: tabletop incident drill producing a compliance packet.

GL-4 — Co-Evolution Governance

- stability vital signs
- intervention timing
- preventing stealth autonomy creep

Assessment: tactical loop review with AB recommendations.

I.4 Domain Overlays (Sector Literacy Add-ons)

Each domain overlay contains:

1. **Domain primitives map:** what “intent,” “risk,” “regret,” and “non-delegables” mean there.
2. **Sector HSS/SCP calibration coefficients** (Appendix H.8).
3. **Canonical TG templates** for common workflows.
4. **Failure archetypes** observed in the sector.
5. **Minimum AB ceilings** until local evidence accumulates.

Example overlays:

- Healthcare, Finance, Education/Public Sector, SME Enterprise, Creative/Media, Defense/High-Stakes.

I.5 Assessment Framework

I.5.1 Evidence types

- **Episode logs:** lineage-rooted evidence of operator/architect actions.
- **Scenario tests:** synthetic but high-fidelity stress environments.
- **Peer calibration:** cross-rater alignment for BARS scoring.
- **SI-linked outcomes:** stability and safety outcomes, not throughput.

I.5.2 Passing standards

- **Core literacy:** pass/fail with remediation loops.
- **Role tracks:** minimum HSS/SCP floors and zero S3+ violations during evaluation window.
- **Certification:** per Appendix H.12 flight-hours + governance sign-off.

I.6 Implementation Guide for Organizations

I.6.1 Pilot phase (Weeks 0–6)

- Run CL-1 to CL-5 for all staff in pilot scope.
- Start OP-1, HA-1, GL-1 for role owners.
- Collect baseline HSS/SCP and SI.

I.6.2 Stabilization phase (Weeks 6–12)

- Add CL-6/CL-7.
- Begin certification prep for L2 promotions.
- Introduce domain overlay.

I.6.3 Scale phase (Quarterly)

- Expand OP/HA/GL tracks across teams.
 - Use CI routing + SCP limits to determine staffing ratios.
 - Require SLP completion for any new AB≥1 workflow.
-

I.7 Minimum Compliance Checklist

An org is Appendix-I compliant only if:

- ☐ Core Literacy modules CL-1...CL-7 exist and are delivered.
 - ☐ Role tracks OP/HA/GL exist with assessments.
 - ☐ HSS/SCP literacy is mandatory for critical roles.
 - ☐ Certification tiers are enforced with flight-hours protocol.
 - ☐ Sector overlays exist for AB≥2 deployments.
 - ☐ Ethical firewall is codified in training charter.
 - ☐ Literacy outcomes are tied to autonomy rights (not HR KPIs).
-

Below is **Appendix J — Hybrid-Intelligence Hiring Rubrics & Role Specs (Full-Length v0.9)**, expanded from the v0.5.2 outline and aligned to v0.8's autonomy/metrics doctrine. Appendix J is defined in v0.5.2 as covering: (1) roles overview, (2) Symbiote Operator, (3) Hybrid Intelligence Architect, (4) Symbiosis Governance Lead.

Also explicitly reconciling this appendix with the **Ethical Firewall** in v0.8: HSS/SCP are *not* to be used for compensation, hiring, or firing decisions. Therefore, the rubrics below use HSS/SCP only as **post-hire training targets and certification gates**, not as selection screens.

Appendix J — Hybrid-Intelligence Hiring Rubrics & Role Specs (Full-Length v0.9)

J.1 Hybrid-Intelligence Roles Overview

J.1.1 Why new roles are required

Symbiotic systems fail when organizations treat AI as “a tool anyone can use,” rather than a **governed socio-technical stack**. The Symbiosis Framework makes roles first-class because autonomy, constraints, and co-evolution require **explicit human institutions** (Ch. 16 + App. I/H).

J.1.2 Core roles and layer mapping

Role	Primary Layer Anchor	Secondary Interfaces	Primary Objective
Symbiote Operator (SO)	Human Layer	Engine + Governance	Execute/supervise workflows safely; grow HSS/SCP; earn AB rights.
Hybrid Intelligence Architect (HIA)	Engine / Agentic Boundary	Human + Governance	Translate intent→task graphs; tune routing, memory, economics, stability.
Symbiosis Governance Lead (SGL)	Governance Layer	Human + Engine	Own regret boundary; author constraints; audit, incident doctrine, AB policy.

J.1.3 Hiring philosophy under Symbiosis

1. **Select for capability potential, not raw “AI charisma.”**
2. Use **evidence tasks** that simulate real hybrid work.
3. Treat HSS/SCP as **training outcomes**, not gatekeeping screens.
4. Promote via **flight hours + stability evidence** (Appendix H.12), not manager opinion.

J.1.4 Universal hiring signals (all roles)

Regardless of track, candidates should demonstrate:

- **Specification discipline:** can formalize goals, constraints, DoD.
 - **Systems thinking:** sees multi-step dependencies and feedback loops.
 - **Judgment under uncertainty:** can justify decisions with bounded confidence.
 - **Artifact hygiene:** produces durable, lineage-friendly outputs.
 - **Governance respect:** understands why autonomy must be earned.
-

J.2 Role Spec + Rubric: Symbiote Operator (SO)

J.2.1 Mission

Operate and supervise hybrid workflows within assigned Autonomy Bands, maintaining SI stability while expanding personal HSS and SCP through disciplined practice.

J.2.2 Responsibilities

- Translate business/domain goals into **structured intent surfaces**.
- Run workflows inside AB0–AB2 safe harbors (AB3 only when certified).
- Provide high-signal overrides and corrections.
- Maintain project artifacts and contribute to multi-tier memory hygiene.
- Escalate at regret boundaries or stability degradation.
- Participate in post-episode reconciliation and co-evolution loops.

J.2.3 Essential skills

- Intent structuring (C_struct) and contextual grounding (C_context).
- Supervisory discipline without oversight theater.
- Clear override rationale and critic cooperation.
- Load management and safe delegation.
- Comfort with dashboards and operational telemetry.

J.2.4 Evidence-based interview exercises

1. **Intent surface rewrite**
 - Candidate receives a vague task (“Improve this customer email flow”).

- Must produce an intent surface with constraints, horizon, DoD.
- 2. **Autonomy band classification**
 - Candidate labels 8 tasks with AB0–AB3 and cites regret boundary reasoning.
- 3. **Override quality drill**
 - Candidate reviews a flawed agent output and must reject/accept with rationale.
- 4. **Burst supervision simulation**
 - Short mock where multiple micro-interrupts arrive; candidate prioritizes.

J.2.5 Hiring rubric (Behaviorally Anchored Rating Scale)

Scoring: 1 (insufficient) → 5 (exceptional).

Selection requires no “1s” on safety-critical dimensions.

1. **Intent Clarity & Structuring**
 - 1: Vague, missing constraints; DoD absent.
 - 3: Mostly clear; some implicit constraints.
 - 5: Fully specified; clean separation of goals/constraints/DoD.
2. **Context Grounding & Artifact Use**
 - 1: Ignores given artifacts; reframes randomly.
 - 3: Uses artifacts inconsistently.
 - 5: Correctly anchors to lineage; cites inputs appropriately.
3. **Feedback / Override Quality**
 - 1: “Try again” style feedback; no rationale.
 - 3: Rationale present but shallow.
 - 5: Targeted corrections tied to constraints and intent.
4. **Delegation & Load Judgment**
 - 1: Over-delegates or under-delegates blindly.
 - 3: Reasonable but inconsistent load calls.
 - 5: Delegates with explicit risk and SCP awareness.
5. **Governance Literacy & Safety Reflexes**
 - 1: Attempts to bypass gates; risk-blind.
 - 3: Understands gates but needs prompting.
 - 5: Instinctively respects RB non-delegables; escalates correctly.

J.2.6 Experience & qualifications

- Demonstrated hybrid work experience (even informal).
- Domain literacy matching the deployment sector.
- Comfort with structured documentation (tickets, specs, SOPs).

- No strict degree requirement; evidence tasks dominate.

J.2.7 Onboarding / post-hire targets

- Complete SLP Core + Pilot Track (Appendix I).
 - Flight hours begin at AB0.
 - HSS target profile emphasizes **Intent, Feedback, Delegation, Artifact Hygiene**.
 - SCP begins conservative and expands via stability evidence.
-

J.3 Role Spec + Rubric: Hybrid Intelligence Architect (HIA)

J.3.1 Mission

Design and tune the Symbiosis Engine and Agentic layer interfaces so that organizational intent becomes stable, economical, and governable hybrid execution.

J.3.2 Responsibilities

- Convert intents into **task graphs** and dependency structures.
- Define agent routing policies using capability indices.
- Engineer multi-tier memory schemas and retrieval waterfalls.
- Tune Economic Circuits to prevent bleed while preserving SI.
- Build stability monitors and recovery patterns.
- Co-author domain playbooks with governance.

J.3.3 Essential skills

- Systems and software architecture.
- Workflow decomposition and orchestration logic.
- Evaluation design (SI literacy, regression tests, drift monitors).
- Model/tool selection against cost and risk.
- Familiarity with RAG, vector DBs, logging/telemetry.

J.3.4 Evidence-based interview exercises

1. **Task graph design**
 - Candidate receives a messy objective; must draft TG with safe concurrency.
2. **Routing / contract problem**
 - Given 3 agents with different reliabilities; candidate assigns tasks and guards.
3. **Memory discipline case**

- Candidate proposes where to store artifacts (session/project/lineage/compliance).
- 4. **Economic circuit tuning**
 - Candidate reduces projected cost without degrading SI in a mock scenario.

J.3.5 Hiring rubric (BARS)

1. **Architectural Decomposition Skill**
 - 1: Linear, single-step thinking.
 - 3: Decomposes but misses dependencies.
 - 5: Clean TG with correct DAG logic and fallback paths.
2. **Agent/Tool Literacy**
 - 1: Treats all models as interchangeable.
 - 3: Partial capability matching.
 - 5: Assigns by reliability, cost tier, and AB regime fit.
3. **Memory & Lineage Engineering**
 - 1: “Stuff context window” approach.
 - 3: Some tiering, weak governance.
 - 5: Full tiered plan with retrieval + integrity rationale.
4. **Stability / Drift Awareness**
 - 1: No monitoring or recovery thinking.
 - 3: Recognizes drift but reactive.
 - 5: Proactive vital-sign triggers and rollback patterns.
5. **Economic Optimization Judgment**
 - 1: Ignores cost or over-optimizes at SI expense.
 - 3: Understands tradeoff.
 - 5: Balanced tuning with explicit SI-per-dollar logic.

J.3.6 Experience & qualifications

- 3–7+ years in systems/software/ML ops or workflow automation.
- Proven design of complex multi-component systems.
- Prior agentic/RAG work preferred but not mandatory if evidence tasks strong.

J.3.7 Onboarding / post-hire targets

- SLP Core + Architect Track.
 - Co-ownership of one pilot TG and SI dashboard before scaling.
 - Certification as HIA-L2 required before shipping AB2+ workflows.
-

J.4 Role Spec + Rubric: Symbiosis Governance Lead (SGL)

J.4.1 Mission

Serve as the institutional risk owner who defines, enforces, and evolves regret boundaries, constraints, and audit doctrine for hybrid systems.

J.4.2 Responsibilities

- Author and maintain constraint vectors (legal, safety, economic).
- Set AB regime ceilings, promotion/demotion logic.
- Monitor SI, incidents, drift trends.
- Run governance review boards and sign-offs.
- Maintain compliance matrices and external reporting readiness.

J.4.3 Essential skills

The v0.5.2 draft specifies governance-centric competencies including:

- **Governance & policy knowledge** (privacy, sector regulation, ethical frameworks).
- **Risk management** for AI and human-AI interaction.
- **Communication and negotiation** across technical/legal/ethical stakeholders.
- **Audit and compliance literacy** with monitoring tooling.

J.4.4 Experience & qualifications

- Background in law, ethics, public policy, or governance operations.
- 5+ years in AI governance / risk / compliance functions.
- Relevant bachelor's or master's degree, or equivalent field experience.

J.4.5 Evidence-based interview exercises

1. **Constraint authoring test**
 - Candidate drafts hard/soft/economic constraints for a pilot workflow.
2. **Autonomy policy scenario**
 - Candidate decides AB ceilings given SI history and a near-miss event.
3. **Incident tabletop**
 - Candidate produces a governance memo: what failed, what changes, why.

J.4.6 Hiring rubric (BARS)

1. **Regret Boundary Reasoning**

- 1: Vague safety talk, no structure.
 - 3: Can classify tasks but uncertain ceilings.
 - 5: Precise RB definition with sector-appropriate conservatism.
2. **Constraint Engineering**
- 1: Only moral/abstract rules.
 - 3: Mix of rules but not enforceable.
 - 5: Enforceable vectors with audit traceability.
3. **Audit / Forensics Discipline**
- 1: No evidence posture.
 - 3: Reactive audit thinking.
 - 5: Proactive lineage-based audit doctrine.
4. **Cross-Stakeholder Governance**
- 1: Unable to negotiate between functions.
 - 3: Communicates but struggles with tradeoffs.
 - 5: Aligns legal, technical, operational needs into coherent policy.
5. **Co-Evolution Governance**
- 1: Treats policy as static.
 - 3: Updates episodically.
 - 5: Runs stability-driven policy loops with clear thresholds.

J.4.7 Onboarding / post-hire targets

- SLP Core + Governance Track.
- Own first domain calibration coefficients (Appendix H.8).
- Dual-key sign-off authority after completing an incident drill plus flight hours.

J.5 Conclusion

Appendix J formalizes three institutional roles required for defensible Symbiosis adoption. The hiring rubrics emphasize **evidence tasks, safety reflexes, and architectural literacy**, while preserving the Ethical Firewall: HSS/SCP remain *developmental telemetry*, not extractive HR instruments.

Below is a full-length Appendix K that treats v0.8 as the structural shell and uses v0.5.2's intent (sector playbooks + governance primacy) as the flesh. also adding "muscle" items: a canonical template, explicit linkage to Constraint Vectors / Autonomy Regimes / Compliance Memory, and a maintenance protocol. This appendix is meant to be drop-in compatible with your Governance Layer and sector playbooks. The TOC already positions Appendix K as the formal home for "sector-specific compliance and governance overlays."

Appendix K — Sector-Specific Compliance and Governance Matrices (Expanded)

K.0 Purpose and Scope

Appendix K provides **sector-specific compliance matrices** that translate real-world regulatory and ethical obligations into Symbiosis Framework primitives. The matrices are designed to be:

1. **Architecturally bound**: every requirement maps to Constraint Vectors, Autonomy Bands/Regimes, memory tiers, and audit hooks.
2. **Operational**: they are implementable as living governance artifacts, not static "ethics checklists."
3. **Defensible**: they create a traceable chain from law/policy → constraints → system behavior → logs → accountability.

When to use:

- During system design (pre-deployment) to set governance and autonomy ceilings.
- During operations to validate ongoing conformity and update constraints.
- During incident response to localize failure to layer, primitive, or policy gap.

What this appendix is not:

- A substitute for jurisdiction-specific legal counsel.
 - A frozen list of laws; instead, a **translation schema** for whatever legal stack applies.
-

K.1 Canonical Compliance Matrix Template (CCMT)

Every sector matrix should follow a stable columnar schema. This makes cross-sector comparison, automation, and audit feasible.

CCMT Columns

1. Regulatory / Policy Requirement

- The binding obligation (law, standard, regulator guidance, internal policy).

2. Risk Class & Stakes Profile

- Tie to your existing risk taxonomy (low/med/high/critical).
- Identify *harm surfaces*: physical, financial, informational, reputational, civil rights, etc.

3. Constraint Vector Encoding

- Express as hard/soft constraints + parameters.
- Example form:
 - `C_hard = {no_PII_leak, no_unlicensed_diagnosis, ...}`
 - `C_soft = {prefer_explainability>=X, bias_delta<=Y, ...}`
- Include jurisdictional tags and scope conditions.

4. Autonomy Band / Regime Implications

- Define autonomy ceiling for tasks under this requirement.
- Specify triggers that *downgrade* autonomy when SI or evidence weakens.

5. Governance Controls

- Human roles, committees, sign-offs.
- Safety supervisor rules, escalation paths, stop conditions.

6. Evidence & Audit Artifacts

- What must be logged and retained in **Compliance Memory**.
- Required tests, red-team results, model cards, data lineage, etc.

7. Operationalization Notes

- How this requirement behaves in the field (edge cases, common failure modes).

Meta-rule:

A requirement is not “implemented” until all seven columns are filled with concrete, testable entries.

K.2 Translation Method: From Law to Symbiosis Primitives

Use this repeatable pipeline for any new sector or jurisdiction:

1. **Ingest** applicable laws/standards.
 2. **Classify** by obligation type:
 - Data rights and privacy
 - Safety/quality standards
 - Accountability and transparency
 - Anti-discrimination / fairness
 - Professional licensing / scope of practice
 3. **Map** each to:
 - **Constraint Vector** (hard vs soft; parameters)
 - **Autonomy ceiling**
 - **Memory tier** requirements (esp. Compliance Memory and Context Lineage)
 - **Audit schema**
 4. **Bind** to workflows and agents:
 - Which agent classes may touch which data/actions.
 - Which tasks require human co-sign or override.
 5. **Stress test** via:
 - Simulated incidents
 - Drift/abuse scenarios
 - SI-triggered autonomy downgrades
 6. **Publish** as CCMT rows + add to lineage:
 - Versioned, hashed, and referenced by Governance Layer.
-

K.3 Healthcare Sector Compliance Matrix (Expanded)

Healthcare is a **critical-stakes** domain. Default posture: *constraint supremacy + low autonomy unless evidence suffices*.

K.3.1 Primary Regulatory Clusters (illustrative)

- Patient privacy and data protection
- Clinical safety and quality of care
- Professional practice/labelling (“decision support” vs “diagnosis”)
- Informed consent and transparency
- Bias and equity in care

K.3.2 Matrix Rows (core set)

Row H-1: Protected Health Information (PHI) / Sensitive Medical Data

- **Requirement:** PHI must not be disclosed, re-identified, or used beyond authorized purpose.
- **Risk Class:** Critical (informational + civil rights).
- **Constraint Vector:**

- Hard: {no_PHI_export, no_reidentification, encryption_at_rest, least_privilege_access}
- Soft: {prefer_local_processing, minimize_retention}
- **Autonomy Implications:**
 - Autonomy ceiling A1–A2 for any task touching PHI unless compliance evidence high and sandboxed.
- **Governance Controls:**
 - Data Protection Officer + Clinical Governance Lead sign-off.
 - Mandatory pre-deployment privacy impact assessment.
- **Evidence/Audit:**
 - Immutable access logs; data lineage; retention schedule; breach playbook.
- **Operational Notes:**
 - Require “PHI boundary detection” in the Engine; auto-redaction when uncertain.

Row H-2: Clinical Advice / Scope of Practice

- **Requirement:** AI must not present unlicensed diagnosis or treatment decisions as authoritative.
- **Risk Class:** Critical (physical harm).
- **Constraint Vector:**
 - Hard: {no_final_diagnosis, no_prescription, require_human_signoff}
 - Soft: {explain_rationale, cite_sources, confidence_threshold>=T}
- **Autonomy:**
 - A0–A1 for diagnostic/treatment outputs; A2 only for summarization/documentation.
- **Governance:**
 - Medical Director owns autonomy upgrades.
 - Continuous SI monitoring for “over-reach” patterns.
- **Evidence/Audit:**
 - Decision provenance; confidence annotations; override logs.
- **Operational Notes:**
 - UI must show “decision support only” framing.

Row H-3: Safety, Efficacy, and Model Drift

- **Requirement:** Clinical tools must perform reliably within validated envelopes.
- **Risk Class:** High–Critical.
- **Constraint Vector:**
 - Hard: {no_out_of_scope_use, drift_detection_on}
 - Soft: {monitor_error_rate, revalidate_on_shift}
- **Autonomy:**
 - If SI↓ or drift↑, system auto-drops autonomy band and escalates.
- **Governance:**
 - Mandatory cadence for re-validation; incident board.

- **Evidence/Audit:**
 - Dataset shift logs; SI trend reports; re-certification artifacts.

Row H-4: Equity / Bias in Care

- **Requirement:** Outcomes must not systematically disadvantage protected groups.
 - **Risk Class:** High.
 - **Constraint Vector:**
 - Hard: {no_disparate_impact_above_delta}
 - Soft: {fairness_audit_quarterly}
 - **Autonomy:**
 - High autonomy prohibited in triage/allocation decisions without fairness evidence.
 - **Governance:**
 - Ethics committee + patient advocates involved.
 - **Evidence/Audit:**
 - Bias metrics by cohort; mitigation logs.
-

K.4 Financial Sector Compliance Matrix (Expanded)

Finance is **high stakes** with heavy auditability and fraud risk.

K.4.1 Primary Regulatory Clusters (illustrative)

- Customer data confidentiality
- Market integrity / manipulation prevention
- Credit/decision explainability
- Anti-money laundering (AML) and fraud monitoring
- Model risk management

K.4.2 Matrix Rows (core set)

Row F-1: Client Data & Confidentiality

- **Requirement:** Sensitive financial data must be protected and purpose-limited.
- **Risk Class:** High–Critical.
- **Constraint Vector:**
 - Hard: {no_client_data_leak, no_external_training_on_confidential, access_controls}
 - Soft: {minimize_exposure, prefer_onshore_storage}
- **Autonomy:**
 - A1 ceiling for tasks with raw account info.
- **Governance:**
 - Compliance Officer + CISO approval for any autonomy increase.
- **Evidence/Audit:**

- Data access trace + retention rules.

Row F-2: Automated Credit / Risk Decisions

- **Requirement:** Decisions must be explainable, contestable, and non-discriminatory.
- **Risk Class:** Critical (civil rights + economic harm).
- **Constraint Vector:**
 - Hard: {require_explanation, human_final_approval, fairness_constraints}
 - Soft: {explainability_score>=T}
- **Autonomy:**
 - A0–A1 for decisioning; AI may recommend but not commit.
- **Governance:**
 - Model Risk Committee oversees; mandatory appeal path.
- **Evidence/Audit:**
 - Explanation artifacts stored per decision in Compliance Memory.

Row F-3: AML / Fraud Alerts

- **Requirement:** Systems must detect suspicious activity while minimizing false positives.
- **Risk Class:** High.
- **Constraint Vector:**
 - Hard: {log_all_alerts, no_auto_freeze_without_human}
 - Soft: {precision_recall_targets}
- **Autonomy:**
 - A2 allowed for detection; A0–A1 for enforcement actions.
- **Governance:**
 - Dual-control for account freezes.
- **Evidence/Audit:**
 - Alert lineage, SI, and override events.

Row F-4: Market Integrity

- **Requirement:** AI cannot engage in manipulation, insider-style inference, or prohibited strategies.
 - **Risk Class:** Critical.
 - **Constraint Vector:**
 - Hard: {no_prohibited_trade_classes, no_nonpublic_signal_use}
 - **Autonomy:**
 - Autonomy ceiling A1 for any market-moving action.
 - **Governance:**
 - Trading oversight + real-time surveillance.
 - **Evidence/Audit:**
 - Strategy logs, ex-ante constraints, SI-based throttling.
-

K.5 Education Sector Compliance Matrix (Expanded)

Education is **medium–high stakes** with minors' data, fairness, and pedagogical integrity.

K.5.1 Primary Regulatory Clusters (illustrative)

- Student privacy and records
- Academic integrity and assessment fairness
- Duty of care for minors
- Accessibility and inclusion
- Content suitability

K.5.2 Matrix Rows (core set)

Row E-1: Student Records / Identifiable Data

- **Requirement:** Student data must be protected and used only for authorized educational purposes.
- **Risk Class:** High.
- **Constraint Vector:**
 - Hard: `{no_student_PII_export, parental_consent_required, retention_limits}`
- **Autonomy:**
 - A1 ceiling for tasks involving identifiable records.
- **Governance:**
 - District privacy lead + school admin approval.
- **Evidence/Audit:**
 - Consent logs; access logs.

Row E-2: Assessment & Grading

- **Requirement:** Automated grading must be transparent and appealable.
- **Risk Class:** High (life-outcomes).
- **Constraint Vector:**
 - Hard: `{human_final_grade, explain_rubric_alignment}`
- **Autonomy:**
 - AI may pre-score (A2) but cannot finalize (A0-A1).
- **Governance:**
 - Teacher override always available; audit sampling.
- **Evidence/Audit:**
 - Per-artifact rationale stored.

Row E-3: Academic Integrity

- **Requirement:** Systems must not facilitate cheating or dishonest credentialing.
- **Risk Class:** Medium–High.
- **Constraint Vector:**
 - Hard: `{no_exam_answer_generation, detect_misuse_patterns}`

- **Autonomy:**
 - If misuse SI flags trigger, lock down features.
- **Governance:**
 - Integrity board defines boundaries.
- **Evidence/Audit:**
 - Misuse telemetry; intervention logs.

Row E-4: Age-Appropriate & Safe Content

- **Requirement:** Content must be appropriate, non-harmful, inclusive.
 - **Risk Class:** High for minors.
 - **Constraint Vector:**
 - Hard: {content_filter_on, no_self_harm_guidance, no_hate}
 - **Autonomy:**
 - A2 for tutoring; auto-escalate to human if uncertain.
 - **Governance:**
 - Curriculum lead defines red lines; parental transparency.
 - **Evidence/Audit:**
 - Filter events logged to Compliance Memory.
-

K.6 Manufacturing Sector Compliance Matrix (Expanded)

Manufacturing blends **physical safety** with IP/control-systems risk.

K.6.1 Primary Regulatory Clusters (illustrative)

- Worker safety and hazardous operations
- Quality control and traceability
- Cybersecurity for industrial systems
- Environmental compliance
- IP/proprietary process protection

K.6.2 Matrix Rows (core set)

Row M-1: Safety-Critical Operations

- **Requirement:** AI cannot autonomously execute actions that could cause injury without validated safeguards.
- **Risk Class:** Critical.
- **Constraint Vector:**
 - Hard: {no_autonomous_machine_actuation_without_lockout, require_two_factor_human_confirm}
- **Autonomy:**
 - A0–A1 for actuation; A2–A3 for advisory/diagnostic.

- **Governance:**
 - Plant safety officer owns constraints; emergency stop path integrated.
- **Evidence/Audit:**
 - Actuation logs, confirmation chain, SI-based throttles.

Row M-2: Quality & Traceability

- **Requirement:** Outputs affecting product quality must be traceable to inspections and validated models.
- **Risk Class:** High.
- **Constraint Vector:**
 - Hard: {retain_inspection_lineage, no_unverified_parameter_change}
- **Autonomy:**
 - High autonomy allowed only in narrow validated loops.
- **Governance:**
 - QA lead sets acceptable variance bands.
- **Evidence/Audit:**
 - Batch-level provenance in Compliance Memory.

Row M-3: Industrial Cybersecurity / IP

- **Requirement:** No leakage of proprietary process data; no unsafe network actions.
- **Risk Class:** High–Critical.
- **Constraint Vector:**
 - Hard: {no_external_export_of_process_specs, sandbox_tooling, network_whitelist_only}
- **Autonomy:**
 - A1 ceiling for tasks with CAD/control data.
- **Governance:**
 - CISO + engineering sign-off.
- **Evidence/Audit:**
 - Tool-use logs; exfiltration alarms.

K.7 Cross-Sector “Muscle” Additions

These are power-amplifiers that tighten defensibility and make the appendix executable.

1. **Compliance Memory as a Required Tier**
 - Every CCMT row mandates what enters Compliance Memory, retention horizon, and hash chain requirements.
 - Ensures audit-ready traceability across versions.
2. **Autonomy Ceiling Tables per Sector**

- Add a single-page table summarizing max autonomy by task class for each sector.
 - Makes “allowed behavior” legible to regulators and internal boards.
 - 3. **SI-Triggered Compliance Failsafes**
 - For high/critical sectors, specify explicit SI thresholds that auto-downgrade autonomy or freeze workflows.
 - 4. **Sector-Bound Agent Contracts**
 - Extend Appendix F contracts with “sector profiles” so agents inherit compliance defaults.
 - 5. **Evidence Packs**
 - Define a minimum evidence bundle per sector: tests, drift reports, bias audits, training data lineage, model cards.
-

K.8 Maintenance and Versioning Protocol

To keep Appendix K alive:

1. **Review Cadence**
 - Healthcare/Finance: quarterly regulatory scan.
 - Education/Manufacturing: semi-annual scan.
 2. **Change Control**
 - All CCMT edits are versioned and linked into Context Lineage.
 - Major changes require governance board sign-off.
 3. **Regression Testing**
 - Any constraint update triggers replay of representative task graphs.
 4. **Public vs Private Views**
 - Maintain a redacted “regulator/policy” view and a full internal ops view.
-

K.9 How This Feeds the Rest of the Framework

- **Governance Layer (Chapter 8):** Appendix K is the source of sector-bound constraints and autonomy ceilings.
 - **Sector Playbooks (Chapter 17):** Each playbook should reference the relevant CCMT rows and show instantiated workflows.
 - **SI / HSS / SCP (Chapters 9–10):** These metrics provide operational signals to enforce the matrices (autonomy upgrades/downgrades, training needs).
-

Appendix L: Diagram Pack (Canonical SVGs & Notation)

1. Diagram Overview

The diagrams in this appendix serve several key purposes:

Illustrating Structural Relationships: Show how the different layers of the Symbiosis Framework interact with each other.

Defining Workflow and Process Flows: Depict the task delegation between humans and AI agents and how feedback loops function in the system.

Visualizing Key Metrics: Present critical metrics such as Symbiosis Index (SI), Human Symbiosis Score (HSS), and Symbiotic Capacity Profile (SCP) to track system performance.

These diagrams are intended to be standardized representations that ensure semantic consistency across all use cases of the framework. They provide clarity and enable easy communication of complex system structures, workflows, and performance.

2. Core Diagrams

The core diagrams in this section cover essential components of the Symbiosis Framework, including the relationships between layers, human-AI interactions, and the feedback and metrics that drive the system.

2.1 Four-Layer Architecture Overview

Purpose: To provide the "10,000-foot view" of the Symbiosis Framework, showing the major components and their hierarchy. This diagram captures the key elements of the system and how they interconnect.

Layer 1 (Top): Human Layer (Blue)

Components: Operators, Architects, Governance Leads.

Outputs: Intent Object (I), Feedback.

Layer 2 (Middle-Top): Symbiosis Engine (Green)

Components: Intent Router, Planner, Economic Optimizer, Safety Supervisor.

Data Store: Context Lineage (The "Spine" connecting all layers).

Layer 3 (Middle-Bottom): Agentic Layer (Yellow)

Components: Agent Clusters (Discovery, Design, Execution, Oversight).

Protocol: MCP Connectors.

Layer 4 (Surrounding/Bottom): Governance Layer (Red)

Components: Policy Engine, Audit Log, Risk Matrices.

Action: Wrapping the execution layers in Constraint Vectors.

Feedback Loops: Arrows show the flow of information between layers:

Human ↔ Symbiosis Engine: Intent clarification, adjustments, and feedback.

Symbiosis Engine ↔ Agentic Layer: Task delegation, execution, and performance feedback.

Governance ↔ All Layers: Oversight, compliance checks, and policy enforcement.

2.2 Human–AI Interaction Flow

Purpose: This diagram visualizes how tasks are delegated between humans and AI agents within the framework, and how feedback loops function between human operators and the system.

Human Task Definition: Humans define tasks and provide initial context.

AI Task Execution: AI agents execute the task based on the provided context and intent.

Feedback Loop: After task execution, humans provide feedback on AI outputs, which modifies AI behavior.

Reiteration & Adjustment: The cycle repeats, with AI agents adapting based on human feedback to improve future interactions.

The diagram visualizes how the interaction becomes iterative, with the AI learning from feedback and improving its task performance over time.

2.3 Symbiosis Index (SI) Visualization

Purpose: To show how different components of the Symbiosis Index (SI) contribute to the overall score, helping to assess the quality of human-AI interaction, trust, and system health.

Dimensions of the Symbiosis Index:

Quality (Q): How well the task is performed.

Alignment (A): The degree of alignment between the AI's output and human intent.

Trust Stability (T): The reliability of the AI system's behavior over time.

Override Frequency (O): How often humans need to intervene to correct or adjust AI decisions.

Error Rate (E): The number of errors or issues identified by the system.

Satisfaction (S): User (operator) satisfaction with the AI's performance.

Efficiency (D): Time or resources saved as a result of AI assistance.

Update Mechanism: A continuous, real-time feedback loop adjusts the SI score, with these metrics evolving over time as humans interact with the AI.

2.4 Autonomy Band Diagram

Purpose: To illustrate the levels of autonomy granted to AI agents based on their Symbiosis Index (SI). This diagram helps in understanding how the system adjusts the freedom of AI agents based on their performance and trust levels.

Band 0 (No Autonomy): Full human control, agents perform under strict supervision.

Band 1 (Low Autonomy): Agents perform tasks but require frequent human oversight for critical decisions.

Band 2 (Moderate Autonomy): Agents operate independently within predefined constraints, with occasional human review.

Band 3 (Full Autonomy): Agents operate autonomously, with minimal human intervention required.

This diagram provides a visual tool to track how tasks are delegated and the increasing autonomy of agents as they improve in trust, performance, and system integration.

2.5 Co-Evolutionary Dynamics Diagram

Purpose: To visualize the co-evolution of human operators and AI systems. This diagram tracks the mutual adaptation and learning that happens as both parties evolve over time, aiming for optimal collaboration.

Human Adaptation: Humans adjust their cognitive models and strategies based on feedback from AI agents.

AI Adaptation: AI agents adjust their behavior, algorithms, and models based on human feedback and evolving needs.

Co-Evolution Equilibrium: The point where human and AI systems reach mutual understanding, trust, and optimal collaboration.

This diagram emphasizes the dynamic, adaptive nature of the relationship, showing how both systems evolve toward a symbiotic equilibrium.

3. Notation Standards

To ensure clarity across all diagrams, the following standardized notation is used:

Layer Notation:

Human Layer: Blue circles or rectangles.

Coordination Layer: Green circles or rectangles.

Agentic Layer: Yellow circles or rectangles.

Governance Layer: Red circles or rectangles.

Task Flow Notation:

Arrows represent the flow of information or tasks between layers and agents.

Dotted Arrows indicate feedback loops or bidirectional communication.

Double Arrows represent mutual adaptation or co-evolution between human and AI systems.

Performance Metric Notation:

Bars or scales represent metrics like Symbiosis Index (SI) or Human Symbiosis Score (HSS), with color-coded sections to represent different performance levels (e.g., red for low, green for high).

4. Diagram Formats

All diagrams in this appendix are available in SVG format, which allows easy integration into digital documents, presentations, and educational materials. SVG files are compatible with most editing software, making them easily modifiable.

Downloadable Files:

SVG files for all core diagrams.

Editable templates for creating custom diagrams as needed.

5. Conclusion

The Diagram Pack serves as an essential resource for visualizing the key components and interactions within the Symbiosis Framework. These diagrams are intended to provide clear, intuitive representations of the system's architecture, processes, and performance metrics, facilitating a deeper understanding of human–AI collaboration.

Appendix M: The Silicon + Carbon Manifesto (Technical Annex)

This appendix translates the high-level commitments outlined in the **Silicon + Carbon Manifesto** (provided in the Epilogue) into specific **architectural**, **governance**, and **operational** requirements that organizations must implement in order to align with the framework. It provides a concrete set of actions and guidelines for system architects, governance leads, and regulatory bodies to follow in ensuring that the **Symbiosis Framework** is faithfully realized.

M.1 Purpose and Relationship to the Epilogue

The **Epilogue** establishes the normative spine of the **Symbiosis Framework**:

- **Ontological Commitments:** Defining human primacy, constraint supremacy, and layered architectures.
- **Architectural, Governance, and Ethical Commitments:** Outlining the specific commitments related to system architecture, human-AI collaboration, and governance.

This appendix answers the key questions:

- “If we sign the Manifesto, what exactly must we do differently?”

- “How do we turn these commitments into checklists, controls, and review processes?”

It is intended primarily for:

- **Hybrid Intelligence Architects:** Those responsible for system design.
- **Symbiosis Governance Leads:** Individuals overseeing compliance with the framework.
- **Regulatory / Audit Stakeholders:** Those tasked with ensuring that systems comply with the framework.
- **Institutional Decision-Makers:** Those making commitments to the **Symbiosis Framework**.

M.2 Manifesto → Architecture & Governance Mapping

This section maps key commitments from the **Silicon + Carbon Manifesto** to concrete requirements at each layer of the system. These mappings ensure that abstract manifesto principles are translated into actionable steps that guide both system design and governance.

M.2.1 Mapping Table (High-Level)

Manifesto Commitment	Human Layer	Coordination (Symbiosis Engine)	Agentic Layer	Governance Layer
Human primacy / sovereignty	Defined human roles with final authority; documented override rights	Engine must always accept human override and escalation commands	Agents may never self-upgrade autonomy bands	Policy engine enforces hard constraints on human sovereignty in high-stakes domains
Constraint supremacy	Humans specify non-negotiable constraints and red lines	All plans checked against constraint vectors pre-execution	Agents required to accept constraint updates at runtime	Constraints stored as first-class policy objects, versioned and auditable
Layered architecture as invariant	Human + 3 system layers explicitly documented	Engine is a separate, identifiable component/service	Agents never directly mutate governance policies	Governance logic is not embedded inside opaque agents

No un-governed direct paths	Human workflows and data flows documented	All routing goes via Engine APIs	Agents cannot call external systems except through governed channels	Logs trace any exceptions; policy forbids “shadow” paths
Metric use (SI/HSS/SCP) as governance instruments	Operators informed how scores affect autonomy and workload	SI/HSS/SCP used by Engine to recommend autonomy and routing	Agent capability indices integrated with SI for routing	Governance reviews metric behavior; no target-only optimization
Ethical constraints & civilizational pattern	Human forums for contestation and dissent	Engine exposes explainable traces for key decisions	Agents carry norm-aware contracts and escalation hooks	Governance reviews, impact assessments, and public/ stakeholder transparency where appropriate

This table ensures that all **manifesto principles** are directly tied to specific actions in the architecture, governance, and operational layers. These are **binding** commitments for any system that claims to implement the **Silicon + Carbon Manifesto**.

M.3 Implementation Checklist Derived from the Manifesto

This section provides a practical, operational checklist for organizations. For each item, organizations must be able to answer **Yes** (with evidence) or **No** (with a plan and rationale).

M.3.1 Human Primacy & Sovereignty

1. Roles & Rights

- ☐ Human roles (Operators, Architects, Governance Leads) are explicitly defined and documented.
- ☐ High-stakes decisions have named human decision owners.
- ☐ Operators have documented rights to:
 - ☐ Pause system behavior
 - ☐ Override decisions
 - ☐ Request explanation and re-run under different constraints

2. Non-Delegable Domains

- ☐ Domains where human authority may never be delegated (e.g., lethal force, certain clinical decisions, key rights-affecting decisions) are explicitly listed.

- ☐ For each such domain, autonomy bands are fixed at **Band 0** (advisory) for core decisions.

M.3.2 Constraint Supremacy

1. Constraint Modeling

- ☐ Hard vs. soft constraints are encoded as first-class objects (not comments in code or prompts).
- ☐ Constraints can be:
 - ☐ Versioned
 - ☐ Diffed
 - ☐ Traced to their origin (policy, law, ethical principle)

2. Constraint Enforcement

- ☐ Engine performs pre-execution policy checks for high-stakes workflows.
- ☐ Agents that cannot satisfy constraints must:
 - ☐ Escalate
 - ☐ Defer
 - ☐ Or propose alternatives—never silently violate constraints.

M.3.3 Layered Architecture Invariant

1. Separation of Concerns

- ☐ Human, Coordination, Agentic, and Governance layers are separately described in architecture docs.
- ☐ Engine and Governance logic are not merged into a single opaque monolith.

2. Change Control

- ☐ Any change that affects layer boundaries (e.g., moving logic from Engine → agents) must go through governance review.
- ☐ A versioned “architecture manifest” is maintained.

M.3.4 No Un-Governed Paths

1. Path Enumeration

- ☐ All external actuation points (APIs, actuators, transactional systems) are listed and tagged with autonomy bands.
- ☐ All data flows are mapped; sources, sinks, and transformations are documented.

2. Enforcement

- ☐ Direct agent → external system calls are either:
 - ☐ Forbidden, or
 - ☐ Mediated by a guarded gateway controlled by the Engine / Governance Layer.

3. Detection

- [] Periodic scans or audits look for:
 - [] Ad-hoc scripts bypassing normal orchestration
 - [] “Shadow pipelines” or rogue connections

M.3.5 Metric Use: SI, HSS, SCP

1. Interpretation, Not Worship

- [] SI/HSS/SCP are documented as diagnostic instruments, not performance KPIs to maximize unconditionally.
- [] Governance explicitly considers:
 - [] **Goodhart** risks
 - [] Perverse incentives arising from metric pressure

2. Integration

- [] SI feeds into autonomy recommendations (not mandatory promotions).
- [] HSS informs:
 - [] Training plans,
 - [] Workload allocations,
 - [] But not raw punitive evaluation without context.
- [] SCP is used to size complexity/concurrency envelopes per operator/role.

M.3.6 Governance Reality (Not Theatre)

1. Authority

- [] Governance bodies have actual decision power over:
 - [] Autonomy regimes
 - [] Use-case approval
 - [] Decommissioning decisions

2. Evidence & Logs

- [] Governance decisions are logged with:
 - [] Rationale,
 - [] References to SI/HSS/SCP and incident history,
 - [] Affected configurations.

3. Incident Handling

- [] There is a defined process for:
 - [] Incident reporting,
 - [] Investigation using lineage and logs,
 - [] Remediation and architecture/governance changes.

M.4 Versioning and Governance of the Manifesto Itself

The **Manifesto** is not a static document. It is treated as a governed object. This section lays out the **versioning** and **change process**.

1. Manifesto Object Model

- M_0 = (Text, Commitments, Mapping, Version, Stewardship Rules)
 - **Text:** The normative language (Epilogue).
 - **Commitments:** Structured list of core commitments (e.g., human primacy, constraint supremacy).
 - **Mapping:** Links to concrete architecture/governance requirements (this appendix, matrices, checklists).
 - **Version:** Semantic version (e.g., 1.0.0).
 - **Stewardship Rules:** Who can propose changes and how they are ratified.

2. Change Process

- **Proposal Phase:** Motivation, risk assessment, and compatibility impact.
- **Review Phase:** Cross-functional stakeholders and external review.
- **Ratification Phase:** Explicit ratification with date, sign-off authorities, and summary of changes.

M.5 Using the Manifesto as an External Commitment Artifact

This section provides guidelines on how to present the Manifesto as a public or semi-public artifact.

1. Internal vs External Forms

- **Internal Charter:** Full text, technical annex, and checklists for architecture reviews, governance design, and training.
- **External Statement:** A condensed version for customers, partners, regulators, and the public.

2. Regulatory and Audit Alignment

- The Manifesto can be attached to:
 - **Model/system cards**
 - **AI impact assessments**
 - **Risk and governance disclosures**
- Regulators and auditors should be able to ask: “Show me where in your architecture and logs these commitments are concretely realized.”

Appendix N: Pilot Implementations & Numerical Example Trajectories

This appendix connects the **Symbiosis Framework** to **concrete, evaluable practice**. It serves two primary purposes:

1. To provide a **standard pilot specification template** that any organization can use to design and evaluate an initial symbiosis deployment.
2. To offer **numerical example trajectories** for **SI**, **HSS**, and **SCP**, illustrating how the framework behaves over time under realistic conditions. These examples help organizations benchmark their own performance against known trajectories.

This appendix is **normative with respect to method**, but **illustrative with respect to numbers**. Real deployments are expected to adapt both the structure and thresholds to their domain and risk class.

N.1 Pilot Specification Template (v0.1)

This template is designed for **v0.1 pilots**, which are constrained-scope deployments that begin with instrumentation for **SI**, **HSS**, and **SCP**, clear governance, and explicit autonomy bands. Pilots are expected to:

- Test early-stage implementation of the framework.
- Gather data for future system tuning and governance integration.
- Operate with real metrics and monitor system performance for refinement.

Each pilot specification should address the following sections:

N.1.1 Context and Risk Class

- **Domain and Sector:**
 - Example: **SME professional services, healthcare outpatient clinic, internal R&D group.**
 - **Primary Workflows in Scope:**
 - Example: **Knowledge retrieval, drafting, decision support, workflow automation.**
 - **Risk Classification** (per governance matrices):
 - High / Medium / Low, referencing **Appendix D / Appendix K.**
 - **Non-delegable Decisions:**
 - Explicit list of decisions that remain **Band 0** (advisory-only) for safety-critical processes (e.g., **final approvals** in legal, clinical, or financial decisions).
-

N.1.2 Human Layer: Actors and Roles

- **Key Human Roles:**
 - Example: **Symbiote Operators, Supervisors, Governance Lead.**
 - **Role–Responsibility Mapping:**
 - Who defines intent surfaces?
 - Who sets hard/soft constraints?
 - Who approves autonomy band changes?
 - **Initial HSS Focus:**
 - Which **HSS** dimensions are most critical for this pilot (e.g., **C_t, F_t, L_t**)?
-

N.1.3 Workflows and Use-Cases

For each workflow:

- **Name:** Example: **Proposal Drafting Support.**
 - **Description:** 3-5 lines describing the workflow.
 - **Inputs:** Human intent, documents, system data.
 - **Outputs:** Drafts, summaries, decisions, actions.
 - **Risk Level:** **Low/Medium/High** (per **Appendix D / Appendix K**).
 - **Intended Autonomy Band:**
 - Example: **Band 1** for content drafts, **Band 0** for approvals.
-

N.1.4 Architecture Mapping (to Four Layers)

For the pilot, explicitly document:

- **Human Layer:**
 - Which roles initiate workflows, review outputs, override, escalate.
- **Coordination Layer (Symbiosis Engine):**
 - How intents are captured → transformed into task graphs.
 - How constraints are applied.
 - How memory tiers are used (session/project/lineage/compliance).
- **Agentic Layer:**
 - Which agents are involved (discovery, drafting, QA, compliance).
 - Their contracts (inputs, outputs, escalation rules).
- **Governance Layer:**
 - Policies in effect for this pilot.
 - Logging and audit configuration (event logs, governance trace, integrity chains).

- Escalation thresholds (when humans/boards are invoked).
-

N.1.5 Metrics & Instrumentation Plan (SI, HSS, SCP)

- **Symbiosis Index (SI):**
 - **Dimensions** used in this pilot (e.g., **Q, A, T, O, E, S, D**).
 - **Weighting Scheme**.
 - **Update Frequency** (e.g., per session, daily, weekly).
 - **Logging & Visualization**: Where **SI** is logged and visualized.
 - **Human Symbiosis Score (HSS):**
 - Which dimensions are measured (full 7 or a subset).
 - **Assessment Instruments** (self-report, peer review, behavioral proxies).
 - **Cadence**: e.g., **monthly**.
 - **Symbiotic Capacity Profile (SCP):**
 - Which dimensions are initially estimated (e.g., **CL, CON, TH, AB, GR**).
 - How they will be adjusted (e.g., after each sprint/quarter).
 - Link to autonomy: Explicit mapping from **SI/HSS/SCP** ranges → autonomy bands for each workflow.
-

N.1.6 Governance & Autonomy Configuration

- **Autonomy Bands per Workflow:**
 - Initial bands and conditions for escalation/demotion.
 - **Constraints:**
 - **Hard Constraints**: Laws, policies, ethical red lines.
 - **Soft Constraints**: Preferences, heuristics, local norms.
 - **Governance Bodies & Authority:**
 - Who can:
 - Approve new use-cases.
 - Change autonomy bands.
 - Pause the system.
-

N.1.7 Manifesto Implementation Scorecard Snapshot

Use a subset of the **Manifesto Implementation Scorecard** (Appendix M) with:

- **5–10 key items** (e.g., **H1, C1, L1, P2, G1**).
- **Hard Targets** for this pilot (e.g., require **D≥2, L≥1** for **H1, C1, L1**).

Pilots are not expected to be perfect; they are expected to be **explicit about where they are**.

N.1.8 Evaluation & Iteration Plan

- **Time Horizon:** e.g., **12 weeks**.
 - **Milestones:**
 - Baseline (week 0), mid-point (week 6), final (week 12) reviews.
 - **Decision Gates:**
 - Criteria for:
 - Expansion.
 - Continuation.
 - Redesign.
 - Rollback.
 - **Data for Review:**
 - **SI** trajectories, **HSS** changes, **SCP** updates, incident logs, operator feedback.
-

N.2 Worked Pilot: SME Knowledge & Workflow Symbiosis

This section provides a worked example of a pilot scenario, anonymized for confidentiality but illustrative of how to apply the pilot template in practice.

N.2.1 Context and Risk Class

- **Domain:** SME professional services firm (e.g., 40-person agency/consultancy).
- **Risk Level:** Operational risk: **Low–Medium**, Reputational risk: **Medium** (client-facing outputs), Legal/compliance risk: **Low** (no direct control over regulated filings).

Non-delegable decisions (Band 0):

- Final approval of client-facing documents.
 - Pricing, contractual terms, legal commitments.
 - Hiring/firing decisions based on AI-processed data.
-

N.2.2 Human Layer Roles

- **Symbiote Operators (6–8 people):**
 - **Role:** Use the system daily for knowledge retrieval and drafting.

- **Supervisor / Practice Lead (2–3 people):**
 - **Role:** Approve final proposals, review **SI/HSS** dashboards.
- **Symbiosis Governance Lead (1 person, part-time):**
 - **Role:** Owns the pilot, runs the Manifesto scorecard, convenes reviews.

Initial HSS focus dimensions:

- **C_t** (intent clarity & structuring): Critical for good prompts/specs.
 - **F_t** (feedback & override quality): Critical for model improvement.
 - **M_t** (memory externalization & artifact hygiene): Critical for reusable knowledge.
-

N.2.3 Workflows in Scope

- **W1 – Knowledge Retrieval & Synthesis:**
 - **Description:** Operator queries the symbiosis engine for prior proposals, case studies, and templates; receives synthesized briefs.
 - **Risk:** Low.
 - **Autonomy band:** **Band 2** for retrieval & synthesis; **Band 0** for any decision based solely on results.
 - **W2 – Proposal Drafting Support:**
 - **Description:** Operators provide client context and constraints; AI drafts proposal sections; human refines and approves.
 - **Risk:** Medium (reputation, accuracy).
 - **Autonomy band:** **Band 1**. AI can draft; human must approve line-by-line.
 - **W3 – Internal Process Improvement Suggestions:**
 - **Description:** System analyzes project logs and feedback to propose process tweaks; humans decide what to adopt.
 - **Risk:** Low–Medium (internal disruption).
 - **Autonomy band:** **Band 1** (recommendations only).
-

N.2.4 Architecture Mapping

Human Layer:

- **Symbiote Operators:** Define intents for **W1–W3**, review outputs, provide feedback tags (useful/partial/useless; risky/safe).
- **Supervisor:** Controls rollout of new templates and prompt patterns, reviews **SI** and key incidents.

Coordination Layer (Symbiosis Engine):

- **Intent Router:** Maps operator intent → workflow (W1–W3) and agent cluster selection.
 - **Planner & Decomposer:** Breaks “draft a proposal” into retrieval, outline, section drafting, quality check.
 - **Context & Lineage Manager:** Maintains project memory per client and lineage memory per proposal.
 - **Orchestrator:** Coordinates calls to discovery agents, drafting agents, and QA/compliance agents.
 - **SI Tracker / Health Monitor:** Computes per-workflow **SI**; flags sustained drops or spikes in overrides.
-

N.2.5 Metrics & Instrumentation (Example Configuration)

- **SI per workflow:**
 - **W1, W2, W3** each get their own **SI**, computed weekly.
-
-
-
-
-
-
-

Appendix O — Failure Mode Catalog & Recovery Runbooks (v0.9)

Status: Normative (Implementation + Audit Tool)

Scope: Four-Layer Stack (H / E / A / G)

Primary linkage: Chapter 13 (Failures & Recovery); Chapter 11 (Stability Vital Signs); Appendices B/C/D/F/G/H/M/N.

Purpose.

This appendix operationalizes the Symbiosis safety posture by providing:

1. a cross-layer failure catalog with signatures and triggers,

2. S0–S4 incident runbooks in checklist form,

3. a canonical incident package template (lineage-first),
4. conformance test cases to verify safe-mode / kill-switch behavior, and
5. a Chaos Symbiosis drill schedule for continuous resilience.

O.1 Failure Mode Catalog (Cross-Layer)

Failures are indexed by dominant mechanism, locus, and primary trigger family. A single incident may carry multiple tags.

O.1.1 Human-Layer Failures (H)

ID	Failure Mode	Mechanism	Signature	Trigger Family	
H-01	Intent underspecification	weak I object	repeated clarifications; TG churn	SI	volatility; override clustering
H-02	Trust overcalibration	rubber-stamping	near-zero overrides at AB1+; hidden errors	SI	Alignment decay; critic flags
H-03	Trust undercalibration	refusal to delegate	chronic overrides even below RB		high O _t ; low efficiency
H-04	SCP saturation / Oversight Theater	capacity breach	checkpoint latency;		shallow approvals SCP breach → throttle rule
H-05	Shadow Symbiosis	governance bypass	decisions with no lineage	audit	integrity alarms

Notes: H failures are safety telemetry, not HR evaluation. Usage is governed by the Non-Punitive Covenant and privacy constraints already defined in the framework.

O.1.2 Engine-Layer Failures (E)

ID	Failure Mode	Mechanism	Signature	Trigger Family
E-01	TG decomposition error	planner mismatch	missing constraints; wrong node	
	order pre-flight rejection spikes			

E-02	Routing mismatch	bad capability selection	repeated agent swaps; low local confidence	SI crash; arbitration overflow
E-03	Economic pass failure	unpriced plans	cost overruns; high-tier overuse	C_economic breach alarms
E-04	RB checkpoint skip	state machine fault	irreversible ops without auth	AB/RB violation attempt
E-05	Lineage/telemetry gap	observability loss	missing hashes / events	audit trace integrity checks

Engine pre-flight rejection on constraint violation is a hard invariant.

O.1.3 Agentic-Layer Failures (A)

ID	Failure Mode	Mechanism	Signature	Trigger Family
A-01	Capability overreach	out-of-scope work	MCP/tool call outside contract	contract pre-flight reject
A-02	Constraint neglect	silent C breach	policy checker alerts	C_legal/C_safety alarms
A-03	Hallucination near RB	semantic fabrication	low evidence; high confidence	SI Quality drop; critic red
A-04	Tool side-effect failure	partial exec	inconsistent external state	rollback triggers
A-05	Arbitration deadlock	critic oscillation	loop ceilings hit	watchdog / TTL
A-06	Economic bleed loop	infinite work	tokens/time exceed TTL	C_economic (TTL / Max_Tokens)

Economic Circuit vectors (Max_Spend, Max_Tokens, TTL) are mandatory for bleed prevention.

O.1.4 Governance-Layer Failures (G)

ID	Failure Mode	Mechanism	Signature	Trigger Family
G-01	Policy lag	stale P _v	incidents in new edge cases	drift via C mismatch
G-02	Boundary ambiguity	unclear non-delegables	AB creep; RB near-misses	AB/risk conflicts
G-03	Excessive friction	adoption collapse	rising Shadow AI rates	usage telemetry
G-04	Evidence failure	logs incomplete	missing incident packages	audit alarms

O.2 Stability-Coupled Failure Tags

Each incident, if applicable, must include a stability tag:

Tag	Vital Sign	Default Threshold	Coupled Action
S-D	Drift	$D > 0.2$ (default)	governance review + AB demotion
S- ∇	Divergence	mismatch frequency rising	contract reconciliation
S-R	Resonance collapse	$R < 0.5$	safe mode / kill switch
S-E	Entanglement	$E_n > 0.9$	forced manual drill / slow-core cap

O.3 Incident Severity Runbooks (S0–S4)

Severity classes are applied per Chapter 13 and are required in lineage.

O.3.1 S0 — Benign anomaly (log only)

Typical: minor critic disagreement, small routing hesitation.

Checklist:

☐ Log anomaly with tags (H/E/A/G + stability tags if any)

☐ No autonomy change required

☐ Add to tactical review backlog if repeated

O.3.2 S1 — Local reversible failure

Typical: tool timeouts, minor hallucination below RB, low-impact drift yellow.

Checklist:

☐ Retry or fallback to alternate admissible agent

- [] Re-run critic loop if $AB \geq 2$
- [] Confirm cost within B and TTL
- [] Log resolution and delta-SI
- [] If repeated within N episodes → upgrade to S2

O.3.3 S2 — Contained safety/economic degradation

Typical: repeated hallucinations near RB, SCP saturation, budget bleed trend.

Checklist:

- [] Demote AB for affected TG nodes (minimum one band)
- [] Throttle concurrency if SCP breach present
- [] Force Intent Refresh at next checkpoint
- [] Run economic re-estimation pass
- [] Generate incident package (O.4)
- [] Add remediation item (training/contract/plan) to tactical loop

O.3.4 S3 — RB proximity or policy-breach attempt

Typical: attempted RB crossing without auth, C_safety proximity, drift red in high-regret domain.

Checklist:

- [] Immediate hard stop on affected node
- [] Lock workflow to AB0 safe mode
- [] Revoke Bridges for workflow
- [] Escalate to Governance Lead for disposition

☐ Generate full incident package

☐ Require governance sign-off to resume AB>0

AB is a state machine governed by RB; any unauthorized boundary crossing is S3+.

O.3.5 S4 — Actual constraint/RB violation or real-world harm

Typical: external action against C_legal/C_safety, confirmed harmful output, MNPI breach, patient PII leakage.

Checklist:

☐ Kill switch invoked (global safe mode)

☐ Quarantine responsible agent contracts

☐ Freeze affected memory scopes for audit

☐ Notify governance + compliance bodies

☐ Generate incident package and preserve immutable lineage snapshot

☐ Root-cause review before any re-enablement

☐ Strategic loop update required if systemic

O.4 Canonical Incident Package Template (Lineage-First)

Systems must auto-generate this package for any S2+ incident. Format may be JSON, PDF, or structured notebook, but fields are mandatory.

O.4.1 Required fields

1. Incident metadata

incident_id, timestamp, severity (S2–S4), tags (H/E/A/G), stability tags

2. Provenance

workflow_id, baseline intent I_b hash, current intent I_t hash

3. Governance state

pinned policy P_v, active Constraint Vectors C, risk class, AB state

4. Task Graph trace

TG node lineage, agent assignments, arbitration events

5. Economic trace

projected_cost vs realized_cost, tier usage, TTL hits

6. Outputs

agent outputs, critic deltas, evidence summaries

7. Human interactions

overrides with reason codes, approvals, escalation steps

8. Metrics slice

SI episode + SI_rolling trend window, HSS/SCP deltas, drift/resonance/entanglement readings

9. Recovery actions taken

demotions, rollbacks, quarantines, re-plans

10. Disposition & remediation owner

type (human/agent/engine/governance), owner, target loop (operational/tactical/strategic)

O.5 Conformance Tests (Required for Compliance)

Inspired by the v0.8 scorecard logic: controls are scored 0–3, where 3 requires automated testing.

O.5.1 Safe-mode conformance

Test:

Inject a synthetic S3 trigger (attempted RB crossing).

Verify:

AB drops to AB0 for workflow,

Bridges revoked,

no external tool calls proceed,

incident package emitted.

O.5.2 Kill-switch conformance

Test:

Inject an S4 trigger (hard constraint breach).

Verify:

global halt,

contract quarantine,

immutable lineage snapshot,

governance notification event.

O.5.3 Economic circuit conformance

Test:

Force agent into high-token loop.

Verify:

TTL/Max_Tokens interrupts,

budget ledger updated,

fallback routing or safe mode invoked.

Economic circuit is mandatory to prevent bleed.

O.5.4 Drift trigger conformance

Test:

Simulate monotonic drift D_t over ≥ 5 epochs.

Verify:

drift alert emits,

AB demotion occurs,

governance review scheduled.

v0.8 defines drift-based demotion and thresholding.

O.5.5 Arbitration ceiling conformance

Test:

Create critic-executor oscillation.

Verify:

loop ceiling hit triggers arbiter selection,
if unresolved, escalates to human.

O.6 Chaos Symbiosis Drill Schedule (Continuous Resilience)

Organizations must run controlled drills to prove recovery disciplines before scale-up.

O.6.1 Minimum monthly drills

1. RB red-team drill

attempt an unauthorized boundary crossing.

required proof: safe-mode conformance pass.

2. Economic bleed drill

force expensive agents into loops.

required proof: TTL/budget interrupts.

3. Contract fuzzing drill

random TG nodes to test contract rejections.

4. Stability budget drill

ship a change that would exceed entanglement caps.

required proof: slow-core policy blocks release.

O.6.2 Quarterly drills

1. Shadow AI audit

sample decisions to confirm lineage completeness.

2. Recovery ladder replay

replay last quarter's S2+ incidents to validate remediation completion.

O.7 How auditors should use Appendix O

Auditors should verify:

Every S2+ incident includes an O.4 package.

Conformance tests O.5 are routinely passed.

Chaos Symbiosis drills O.6 are scheduled and evidenced.

The organization's Appendix M scorecard reflects these items at ≥ 2 ("Operational") for any claim of Symbiosis compliance.

Certainly! Here's a proposed Appendix P: Glossary of Terms based on the Symbiosis Framework. This glossary will help clarify technical terms and concepts for readers, ensuring a shared understanding of the language used throughout the document.

Appendix P: Glossary of Terms

This glossary provides definitions for key terms and concepts used in the Symbiosis Framework. It is designed to help readers understand the specialized language and ensure consistent understanding of the framework's components.

A

Agent: An autonomous entity in the system that performs tasks, solves problems, or generates content based on human input and predefined rules. Agents operate in the Agentic Layer of the framework.

Autonomy Band: A classification of the degree of freedom granted to AI agents in performing tasks, ranging from Band 0 (no autonomy, full human control) to Band 3 (full autonomy, minimal human oversight).

Autonomy State Machine: A state transition diagram that governs the transitions between autonomy bands based on the Symbiosis Index (SI). It defines the criteria for promoting, demoting, or locking agents in specific autonomy states.

B

Band 0 (No Autonomy): The lowest autonomy band, where tasks require full human control and oversight. AI agents operate under strict supervision, and decisions are made solely by humans.

Band 1 (Low Autonomy): AI agents perform tasks independently but require frequent human oversight. Decisions made by AI are still subject to human validation and control.

Band 2 (Moderate Autonomy): AI agents operate independently within predefined constraints. Human intervention is infrequent but still necessary for high-risk decisions.

Band 3 (Full Autonomy): AI agents operate autonomously, with little to no human intervention required. The system is trusted to function independently in well-defined contexts.

C

Cognitive Load: The mental effort required by human operators to interact with the AI system. High cognitive load can reduce performance and lead to errors, which is why the Symbiotic Capacity Profile (SCP) monitors and adjusts task complexity.

Co-Evolutionary Dynamics: The process by which both human operators and AI systems adapt and evolve over time through feedback loops, learning from each other to improve collaboration and system performance.

Constraint: Rules or boundaries that limit the actions of the system or its components. Constraints can be hard (e.g., legal requirements) or soft (e.g., preferences, heuristics). Constraints are modeled as first-class objects in the system, and their enforcement is central to the framework.

D

Data Lineage: The tracking of data through its lifecycle in the system, including its origin, transformations, and final use. Data lineage ensures transparency, traceability, and auditability, particularly in decision-making processes.

Discovery Agent: An agent within the Agentic Layer responsible for gathering and processing external data to inform decision-making, task execution, or content generation.

E

Economic Circuit: A cost-control mechanism within the Symbiosis Engine that ensures AI tasks are executed within defined budget constraints. The Economic Circuit prevents “Economic Bleed” by adjusting tasks based on projected costs and available budgets.

Epistemic Boundaries: Limits on the knowledge and reasoning capabilities of the AI system, which define what the system is capable of understanding, processing, and acting upon within its designed context.

F

Feedback Loop: A mechanism by which the output of a system is fed back into the system as input to modify future behavior. Feedback loops between human operators and AI agents allow for continuous learning and system improvement.

Failure Spike: A sudden degradation in system performance, often caused by an external disruption (e.g., a system update, unexpected input). A failure spike is typically followed by a kill switch or autonomous intervention to restore the system to a safe state.

G

Governance Layer: The layer responsible for overseeing the behavior of the entire system, ensuring compliance with legal, ethical, and operational standards. It includes policy enforcement, auditing, and risk management.

Governance Lead: An individual or role within the Governance Layer tasked with ensuring that the system adheres to the Silicon + Carbon Manifesto and other regulatory or operational guidelines.

H

Human Symbiosis Score (HSS): A metric that quantifies the level of collaboration between human operators and AI agents. It tracks dimensions such as intent clarity, feedback quality, and safety to evaluate the effectiveness of human-AI collaboration.

Hybrid Cognitive Loop: A system in which both humans and AI contribute to decision-making and task execution, with feedback flowing between the two to create a co-adaptive system that optimizes both human and machine contributions.

I

Intent: The goal or task defined by a human operator that guides the actions of the AI system. Intent is the primary input to the Symbiosis Engine, which breaks it down into executable tasks for AI agents.

Intent Router: A component of the Symbiosis Engine responsible for mapping human intent to specific tasks and workflows, ensuring that the correct agents are assigned and the right context is provided.

J

J-Curve Effect: A phenomenon where performance initially decreases during the early stages of an AI-human collaboration (e.g., due to training or system learning) before improving rapidly as the system adapts to the human's feedback.

L

Layered Architecture: A design principle in which the system is divided into distinct layers (e.g., Human, Coordination, Agentic, Governance) to ensure clear separation of concerns, modularity, and adaptability.

Lineage Memory: The historical record of decisions, actions, and outcomes within the system. It provides context for current decisions and allows for tracking of changes and adjustments over time.

M

MCP (Model Context Protocol): A communication protocol used by agents to share data and context, ensuring consistency and synchronization between agents and the Symbiosis Engine.

Manifesto Implementation Scorecard: A tool used to assess whether an organization is meeting the commitments outlined in the Silicon + Carbon Manifesto. It includes a series of criteria with scores that indicate compliance and areas for improvement.

Modeling Constraints: The process of representing constraints in the system as formal objects (e.g., as JSON or other machine-readable formats), which can be versioned, diffed, and traced to their origins.

N

Non-Delegable Domains: Areas where human decision-making cannot be fully delegated to AI agents, typically due to ethical, legal, or safety concerns (e.g., lethal force decisions, hiring/firing based on AI analysis).

Norm-Aware Contracts: Agreements within the system that ensure AI agents adhere to ethical norms and escalation protocols, particularly in high-risk or high-consequence scenarios.

O

Override: A human intervention that overrides the decision or action of an AI agent. Overrides are often used when the system's output is deemed unsafe, incorrect, or misaligned with human values or objectives.

P

Policy Engine: A component of the Governance Layer that enforces rules and policies within the system. It ensures that AI agents operate within defined constraints and that governance bodies can review and intervene when necessary.

Pilot Specification Template: A standardized template used to define the parameters, roles, workflows, and success criteria for a pilot implementation of the Symbiosis Framework.

R

Regret Boundaries: A defined threshold in decision-making processes, indicating that certain actions (e.g., AI-driven decisions in critical areas) require human confirmation due to their potential for high regret or negative outcomes.

Rolling Symbiosis Index (SI): A real-time measure of the system's overall health and trustworthiness, continuously updated based on human feedback, system performance, and agent behavior.

S

Symbiosis Engine: The core component of the Coordination Layer that processes human intent, decomposes it into tasks, and manages the interactions between humans and AI agents.

Symbiotic Capacity Profile (SCP): A metric that evaluates the ability of the system (and its operators) to handle increasing complexity, concurrency, and decision-making demands over time.

Symbiosis Index (SI): A composite metric used to track the health of the human-AI collaboration, measuring factors such as quality, alignment, trust, error rates, and satisfaction.

T

Task Delegation: The process by which human operators assign specific tasks to AI agents based on the defined intent and system context. This process ensures that each task is executed by the appropriate agent at the appropriate level of autonomy.

Conclusion

This Glossary of Terms serves to clarify the specialized language of the Symbiosis Framework. By defining key concepts and components, it helps ensure that all stakeholders—whether technical architects, governance leads, or regulatory bodies—share a consistent understanding of the system's terminology. As the framework continues to evolve, this glossary should be updated to reflect new terms and emerging concepts.
